Measuring Privacy-Preserving Computation in One Place: MPC, FHE, ZKP and Their Hybrids

(author names elided for review)

Abstract

Privacy-preserving computation is not one field but four—secure multi-party computation (MPC), fully homomorphic encryption (FHE), garbled circuits, and zero-knowledge proofs (ZKP)—each with its own literature, benchmarks and accounting conventions. A practitioner choosing between them, or a reviewer assessing a claim that one beats another, must assemble numbers from different papers, measured on different hardware, over different networks, counting different things. Cross-paradigm comparison is therefore largely *citation assembly* not measurement.

We take a different route. We survey the paradigms through a single artifact, Baru, in which one source program runs on all of them—a plain interpreter, three-party replicated-secret-sharing MPC, garbled circuits, BFV/BGV/TFHE homomorphic encryption, a Binius64 SNARK, and *combinations* in which values cross between representations mid-computation without being exposed. Every backend is driven through the same transport, charged by the same byte and round accounting, and checked bit-exactly against the same plaintext reference. A network cost model is validated against a real socket to within 13.6 % worst case, so a claim about a wide-area deployment is computed from measured counts and not asserted.

Several widely repeated intuitions survive re-measurement; several do not. Among the results that only a single-harness study can produce: the FHE-versus-MPC crossover is a property of *the baseline's preprocessing* not of either cryptosystem, and disappears once MPC is given somewhat-homomorphic preprocessing; at a *matched* corruption model—two parties, dishonest majority—and against an external baseline competitive with the published state of the art, MPC's advantage over FHE widens with network depth, from 10× to 47× with preprocessing hoisted on both sides and from a dead tie to 3.2× without, until the levelled homomorphic pipeline stops working altogether at the depth where it would have to bootstrap; active security on the nonlinear surface of private inference costs 2.63× bytes but only 1.06× rounds—and 9.70× bytes once the semi-honest baseline it is measured against is allowed to pack, which only the unverified side can do. We also report our own withdrawn claims—a ZKP backend that proved nothing, a bit-decomposition that leaked its inputs, and a comparison-protocol crossover whose location we got wrong three times before measuring it—because a study that reports only what worked is not reporting.

Contents

1 The comparability problem **4**

2 Scope **6**

3 Mathematical groundwork **6**

3.1 Notation 7

3.2 Asymptotics, negligible functions and the two kinds of security 7

3.3 Algebraic structures, and why the choice of ring is not cosmetic . . . 9
3.4 Fast polynomial multiplication . . . 10
3.5 Fixed-point representation and its two error terms . . . 11
3.6 Communication complexity and the cost model . . . 12

4 Foundations of the protocols **13**
4.1 The real/ideal paradigm . . . 13
4.2 Secret sharing . . . 15
4.3 Multiplication: from correlated randomness, and from triples . . . 16
4.4 Comparison, bit decomposition and truncation . . . 17
4.5 Oblivious transfer and its extensions . . . 18
4.6 Garbling schemes . . . 19
4.7 Lattices, LWE and the hardness assumptions . . . 20
4.8 Homomorphic encryption: noise, depth and the operations that manage them . . . 20
4.9 Packing: two encodings and the operations they admit . . . 22
4.10 Succinct proofs . . . 24

5 The workload: foundations of neural inference **25**
5.1 Classification and the inference function . . . 25
5.2 Nonlinearities, and which ones a cryptosystem can evaluate . . . 26
5.3 Structured layers: convolution and pooling . . . 27
5.4 The decision: why classification does not need a softmax . . . 28
5.5 Quantisation: from real networks to the arithmetic a protocol offers . . . 28
5.6 What an accuracy figure means . . . 29
5.7 Threat models for private inference . . . 30

6 Cryptographic preliminaries **30**
6.1 Security models . . . 31
6.2 Secret-sharing MPC . . . 32
6.3 Garbled circuits . . . 34
6.4 Lattices and homomorphic encryption . . . 35
6.5 A composability theorem, and what it costs . . . 37
6.6 Crossing between the paradigms . . . 41
6.7 Succinct proofs . . . 42

7 Four paradigms, four cost structures **42**
7.1 The hybrid, and why it exists . . . 43

8 The Baru artifact **43**
8.1 Baru is a CEK machine . . . 43
8.2 One program, many backends . . . 45
8.3 Composability . . . 45
8.4 What is implemented . . . 45

9 Compilation: flattening, scheduling and backend-aware optimisation **46**
9.1 Two intermediate representations, and why there are two . . . 46
9.2 Flattening and the classical passes . . . 47
9.3 Scheduling: rounds as a graph height . . . 48
9.4 Backend-aware rewriting . . . 49
9.5 Protocol assignment, and why the objective is not separable . . . 49
9.6 Summary of the compilation pipeline . . . 51

10 The paradigms, surveyed and measured **51**
10.1 Secret-sharing MPC . . . 51
10.2 Garbled circuits . . . 52

10.3 Fully homomorphic encryption . . . 53
10.4 The hybrid and the no-bootstrap result . . . 54
10.5 Zero-knowledge proofs . . . 54
10.6 Preprocessing: OT versus SHE . . . 55

11 Related work, and where this study sits **55**
11.1 Frameworks that run many protocols . . . 55
11.2 Three-party honest-majority protocols . . . 62
11.3 FHE-based and hybrid inference . . . 62
11.4 Homomorphic encryption schemes and libraries . . . 63
11.5 What is distinctive here . . . 64

12 Methodology **64**
12.1 Identical transport, uniform accounting . . . 64
12.2 A validated network model . . . 64
12.3 Bit-exactness as a gate on quoting cost . . . 65
12.4 Machine-checked security attestations . . . 65
12.5 Gating an external baseline the way we gate our own . . . 65
12.6 Recording withdrawn results—and deleting the component . . . 66

13 Cross-paradigm findings **66**
13.1 The FHE-versus-MPC crossover belongs to the baseline's preprocessing . . . 66
13.2 MPC versus FHE at a matched threat model, against an external baseline . . . 67
13.3 Protocol assignment can be derived, not annotated . . . 69
13.4 The nonlinearity crossover, and three wrong answers on the way to it . . . 69
13.5 The convolution crossover, and a trick that did not need its scheme . . . 72

14 Case study: private medical image diagnosis **73**

15 Threats to validity **75**

16 Open problems **78**

17 Conclusion **79**

A Experimental report: what the methodology caught **79**

1 The comparability problem

If we ask how fast private inference is, the literature answers in units that cannot be compared. Gazelle [40], Delphi [52] and Cheetah [37] report two-party hybrid systems in which the server holds the model in plaintext. Falcon [62] reports three-party replicated sharing on a LAN. Papers on CKKS [17] report amortized throughput over batched slots, and papers on SNARKs report prover time for a circuit that another paper would express with a different constraint count. Each of these measurements is internally rigorous, but taken together they do not compose. In this section we describe the obstacles that prevent them from composing, and we state what we propose to do about it.

We remark that four obstacles recur.

1. **Different hardware.** A 5× difference between two systems is unfalsifiable when one ran on a server-class Xeon and the other on a laptop.

2. **Different accounting.** "Communication" may mean payload bytes, serialized bytes, or bytes per party. "Rounds" may mean protocol rounds, messages, or interactive calls. These differ by more than the effects being measured.

3. **Different security models.** Semi-honest and malicious figures appear in adjacent columns of secondary literature without the distinction surviving the citation.

4. **Different functions.** Two ReLU costs may be for different bit widths, different fixed-point scales, or—in the worst case we found in our own work—subtly different functions that nobody checked against a common reference.

A survey that is assembled from such numbers inherits all four obstacles. The alternative that we pursue in this work is to re-measure the paradigms inside a single harness, in which the obstacles are controlled by construction: the same machine, the same accounting code, machine-checked security levels, and a bit-exactness assertion against a shared plaintext reference before any cost is quoted.

The cost of this approach is equally clear, and we state it at the outset. Our implementations are research-quality Rust and not the tuned C++ of the systems that we discuss, and therefore **the absolute figures given here are not competitive with optimized implementations and should not be read as such**. What a single harness buys us is *internally valid ratios*, that is, comparisons in which the confounding factors listed above are held fixed. It means that our internal comparisons are stronger evidence than our external ones, and where we compare against a published number we say so explicitly and treat the comparison accordingly.

Organization. In Section 2 we delimit the scope of the survey. Sections 3 to 6 are the foundations, and we have made them deliberately self-contained: Section 3 fixes notation and develops the mathematics—negligible functions and the two indistinguishability notions, the algebra of the rings we compute in and why the choice between a power-of-two modulus and a prime is not cosmetic, fast polynomial multiplication, fixed-point error analysis, and the cost model that converts our measured counts into latency. Section 4 builds the protocol machinery on it, with the real/ideal definition our composability argument quantifies over, secret sharing and its multiplication protocols, the comparison and truncation primitives whose cost dominates our secret-sharing measurements, oblivious transfer, garbling, the noise arithmetic of homomorphic encryption, packing, and succinct proofs. A reader who knows this material can begin at Section 6, which gives the cryptographic

preliminaries in the specific form our artifact implements them—and we remark that several of our results turn on the difference between that form and the textbook one. Section 5 completes the foundations on the workload side, developing the neural inference our experiments compute: the arithmetic of dense and convolutional layers, which nonlinearities each cryptosystem can evaluate and at what cost, quantisation and its error, the metrics an accuracy figure is quoted in, and the threat models a private inference may target—the last of which is what makes two published latencies incomparable when one hides the model and the other does not.

In Section 7 we organize the four paradigms by what each one is expensive in, and in Section 8 we describe our artifact and the abstract machine at its centre. Section 9 presents the compiler as a contribution in its own right: the two intermediate representations and the faithful lowering between them, the classical passes and their sound order, scheduling—under which round complexity is a graph height—backend-aware rewriting, in which one transformation is profitable under one protocol and unprofitable under another, and protocol assignment, whose objective we show is not separable and therefore not faithfully expressible as an integer program. In Section 10 we survey and measure each paradigm in turn. In Section 11 we position this study with respect to the literature, and in particular with respect to MP-SPDZ. In Section 12 we present the methodology, and in Section 13 the cross-paradigm findings that motivate it. In Section 14 we give a case study on private medical image diagnosis. In Section 15 we state the threats to validity, and in Section 16 the problems that remain open. Appendix A is a detailed experimental report of the errors this methodology detected in our own work; it sits outside the body because its interest is methodological instead of scientific, and no result of the survey depends on it.

Contributions.

- **Foundations** for cross-paradigm comparison that are developed not cited: the mathematics the paradigms share and the points at which their algebra diverges (Section 3), the protocol machinery built on it (Section 4), and the arithmetic of the workload itself, since a comparison of privacy-preserving inference is in the end a comparison of how each paradigm evaluates one nonlinearity (Section 5).
- A **systematization** of four paradigms organized by what they actually cost, with the cost dimensions (rounds, bytes, compute, preprocessing) separated rather than collapsed into one "performance" number (Section 7).
- A **single-artifact measurement study**: roughly 70,000 lines of Rust in which one program runs on every paradigm, with a validated network model and uniform accounting (Sections 8 and 12).
- **Cross-paradigm findings** not visible from any single paradigm's literature (Section 13), including two crossovers whose location depends on parameters the original claims left implicit—one of which we ourselves mislocated three times before measuring it (Section 13.4).
- A **compiler whose optimiser takes the protocol as a parameter** (Section 9). One source program is flattened, lowered to a vectorised bytecode carrying domain and secrecy, rewritten against the cost model of the chosen protocol, and scheduled so that round complexity is the height of a dependence graph. We show that a single rewrite rule can be profitable under one protocol and unprofitable under another (Proposition 9.2), and that the protocol-assignment objective is *not separable* over operations and therefore not faithfully expressible as an integer program (Proposition 9.3 and Corolary 9.3)—which distinguishes our formulation from the prior work on the problem.

- A **methodology for honest benchmarking** (Section 12), together with the record, in Section A, of the errors it detected in our own work.

2 Scope

In this section we delimit what the survey covers, and we state the workload that we use throughout.

In scope. Semi-honest and malicious three-party MPC over rings; garbled circuits in the honest-majority setting; RLWE-based FHE (BFV [32], BGV [10]) and TFHE [19]; FHE↔MPC scheme switching; SNARKs over 64-bit words (Binius64 [27]); and the supporting primitives these need—oblivious transfer, private set intersection, and Beaver-triple [6] preprocessing from both OT and somewhat-homomorphic encryption (SHE).

Out of scope. Differential privacy and federated learning (a different threat model: they protect an *aggregate*, not a computation); trusted execution environments (hardware trust assumption); ORAM (surveyed but not implemented, see Section 16); and transformer nonlinearities such as softmax and LayerNorm, which need exponential, reciprocal and inverse-square-root subprotocols we do not have.

Application focus. We use private neural-network inference as the running workload, because it is the one application that exercises every paradigm at once. It requires dense linear algebra, where the packing techniques of homomorphic encryption are effective; it requires comparisons, where secret sharing and garbled circuits compete; and it admits verifiable computation, where succinct proofs apply. We remark that some of our results are specific to this workload, and where that is the case we say so.

3 Mathematical groundwork

This section fixes notation and collects the mathematics the rest of the survey uses. We have written it to be self-contained for a reader who knows undergraduate algebra and probability but not cryptography, because the comparison this paper makes is between paradigms whose costs are determined by quite different mathematical objects—a carry chain in one case, the growth of a noise term in another—and a reader cannot weigh our measurements without both. Where a statement is standard we give it with a proof anyway when the proof is short and the constant matters to us later; where the proof is long or the result is purely cited, we say so and give the reference.

3.1 Notation

Notation 3.1. We write $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$ and $\mathbb{C}$ for the naturals, integers, rationals, reals and complex numbers. For $m \in \mathbb{N}$ we write $\mathbb{Z}_m$ for the ring of integers modulo m; we use $\mathbb{Z}_{2^k}$ for the machine-word rings and $\mathbb{Z}_p$ or $\mathbb{F}_p$ for prime fields, writing the latter as a field when we rely on invertibility. For a finite set S we write $s \leftarrow S$ for a draw from the uniform distribution on S, and $s \leftarrow D$ for a draw from a distribution D. Vectors are lower-case bold, $\mathbf{a}$, and matrices upper-case bold, $\mathbf{A}$. For $x \in \mathbb{R}$ we write $\lfloor x \rfloor$, $\lceil x \rceil$ and $\lfloor x \rceil$ for rounding down, up, and to the nearest integer with ties resolved upward. For $a \in \mathbb{Z}$ and odd or even modulus m we write $[a]_m$ for the representative of $a + m\mathbb{Z}$ in the ***centred*** interval $[-m/2, m/2)$, and $a \bmod m$ for the representative in $[0, m)$; the distinction matters in Section 3.5, because a fixed-point value is a centred representative while a machine word is the other one. The security parameter is λ, the statistical parameter is κ, and log is base two unless written otherwise. We write $|S|$ for the cardinality of a set and $\|\cdot\|_\infty$ for the maximum-absolute-value norm on a vector or on the coefficient vector of a polynomial. Parties are $P_0, \ldots, P_{n-1}$ and indices into parties are always modulo n, so that P_{i+1} makes sense for $i = n - 1$.

We reserve $[\![x]\!]$ for a secret sharing of x in whichever scheme is in force, and ct for a ciphertext. Overloading $[\![\cdot]\!]$ is deliberate: one of the points of this survey is that a program can be written once against an abstract sharing and executed under several concrete ones, and the notation should not pretend otherwise. Where the concrete scheme matters we subscript it.

3.2 Asymptotics, negligible functions and the two kinds of security

Cryptographic statements are asymptotic: they say that an adversary's advantage shrinks faster than any inverse polynomial in a parameter the honest parties may increase. Making this precise is what lets us distinguish claims that are *unconditional* from claims that rest on a computational assumption, and the distinction runs right through our comparison, because secret sharing with an honest majority is information-theoretic while homomorphic encryption is not.

Definition 3.1. A function $\varepsilon : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is ***negligible*** when for every $c \in \mathbb{N}$ there is an n_0 such that $\varepsilon(n) < n^{-c}$ for all $n > n_0$. We write $\mathrm{negl}(n)$ for an unspecified negligible function and $\mathrm{poly}(n)$ for an unspecified positive polynomial. A function is ***overwhelming*** when it is $1 - \mathrm{negl}(n)$.

Proposition 3.1. Negligible functions are closed under addition and under multiplication by a polynomial: if $\varepsilon_1, \varepsilon_2$ are negligible and p is a polynomial, then $\varepsilon_1 + \varepsilon_2$ and $p \cdot \varepsilon_1$ are negligible.

Proof. Fix c. Choose n_1 with $\varepsilon_1(n) < n^{-(c+1)}$ for $n > n_1$ and n_2 likewise for ε_2. For $n > \max(n_1, n_2, 2)$ we have $\varepsilon_1(n) + \varepsilon_2(n) < 2n^{-(c+1)} \leq n^{-c}$, which proves the first claim. For the second, let p have degree d and leading behaviour bounded by $p(n) \leq n^{d+1}$ for large n; choose n_3 with $\varepsilon_1(n) < n^{-(c+d+1)}$ for $n > n_3$, so that $p(n)\varepsilon_1(n) < n^{d+1} n^{-(c+d+1)} = n^{-c}$. □

Proposition 3.1 is the reason a protocol may invoke a subprotocol polynomially many times without losing security, and it is used implicitly in every hybrid argument below.

Definition 3.2. The ***statistical distance*** between distributions D_0, D_1 over a countable set S is

$$\Delta(D_0, D_1) \;=\; \tfrac{1}{2}\sum_{s\in S} \left|\Pr[D_0 = s] - \Pr[D_1 = s]\right| \;=\; \max_{T\subseteq S} \left(\Pr[D_0 \in T] - \Pr[D_1 \in T]\right).$$

Two families of distributions are ***statistically indistinguishable*** when their statistical distance is negligible in λ, and ***perfectly indistinguishable*** when it is zero.

The two expressions in Definition 3.2 agree because the maximising T is $\{s : \Pr[D_0 = s] > \Pr[D_1 = s]\}$, and the positive and negative parts of the difference each sum to the same value since both distributions have total mass one. The second form is the operational one: it says that statistical distance is exactly the advantage of the best possible distinguisher, *including computationally unbounded ones*. This is why a statistical guarantee needs no assumption, and why we quote κ separately from λ throughout.

Definition 3.3. Distribution ensembles $\{D_0(\lambda)\}$ and $\{D_1(\lambda)\}$ are ***computationally indistinguishable***, written $D_0 \approx_c D_1$, when for every probabilistic polynomial-time distinguisher $\mathcal{D}$,

$$\Big|\Pr[\mathcal{D}(1^\lambda, D_0(\lambda)) = 1] - \Pr[\mathcal{D}(1^\lambda, D_1(\lambda)) = 1]\Big| \;=\; \mathrm{negl}(\lambda).$$

Statistical indistinguishability implies computational indistinguishability, and the converse fails; the gap is where all of lattice cryptography lives.

Lemma 3.1 (Hybrid argument)**.** Let $D_0, D_1, \ldots, D_m$ be distribution ensembles with $m = \mathrm{poly}(\lambda)$, and suppose $D_{j-1} \approx_c D_j$ for every j. Then $D_0 \approx_c D_m$.

Proof. Let $\mathcal{D}$ be a distinguisher and write $p_j = \Pr[\mathcal{D}(D_j) = 1]$. By the triangle inequality, $|p_0 - p_m| \le \sum_{j=1}^{m} |p_{j-1} - p_j|$. Each term is negligible by hypothesis, and there are polynomially many of them, so the sum is negligible by Proposition 3.1. □

Lemma 3.1 is the workhorse of every simulation proof in this survey: a simulator is exhibited by replacing one real message at a time with a simulated one and arguing that each replacement is undetectable.

We will need one further probabilistic tool, for the arguments in which a mask drawn from a wide interval hides a value drawn from a narrow one. It complements Theorem 6.1, which treats masking over a whole ring; the lemma below treats masking by an interval, which is the situation in threshold decryption of a lattice ciphertext (Section 6.4) and in the masked openings of Section 4.4.

Lemma 3.2 (Smudging)**.** Let $B, B' \in \mathbb{N}$ with $B' \ge 2^\kappa B$, let e be any fixed integer with $|e| \le B$, and let $E \leftarrow [-B', B']$ be uniform. Then

$$\Delta\big(E,\; E + e\big) \;\le\; \frac{|e|}{2B' + 1} \;\le\; 2^{-\kappa}.$$

Consequently, for any two integers e_0, e_1 of magnitude at most B, the distributions $E + e_0$ and $E + e_1$ are at statistical distance at most $2^{-\kappa+1}$.

Proof. Both E and $E + e$ are uniform on integer intervals of the same cardinality $2B' + 1$, namely $[-B', B']$ and $[-B' + e, B' + e]$. Two uniform distributions on finite sets S_0, S_1 of equal cardinality N have statistical distance $|S_0 \setminus S_1| / N$: each point of $S_0 \setminus S_1$ contributes $1/N$ to the sum in Definition 3.2 and each point of $S_1 \setminus S_0$ contributes the same amount, and the factor $\frac{1}{2}$ halves the total. Shifting an interval of length $2B' + 1$ by e leaves $|e|$ points uncovered at one end, so $\Delta(E, E+e) = |e| / (2B'+1) \leq B/(2 \cdot 2^{\kappa} B) \leq 2^{-\kappa}$. The final claim follows from the triangle inequality applied through E. □

Remark 3.1. Lemma 3.2 is cheap to state and expensive to use. It says that hiding a B-bounded quantity statistically costs κ *additional* bits of range in the mask, and in a lattice ciphertext that range is subtracted from the noise budget that also has to accommodate the evaluation. This is the exact mechanism by which the soundness slack of a proof about a ciphertext is paid for out of the depth available for computing on it, and we report the measured consequence in Section 15.

3.3 Algebraic structures, and why the choice of ring is not cosmetic

Two ambient structures appear throughout: the ring $\mathbb{Z}_{2^k}$ of machine words and the field $\mathbb{F}_p$ of integers modulo a prime. They are not interchangeable, and the differences propagate all the way into our cost measurements, so we set them out explicitly.

Definition 3.4. A ***commutative ring*** $(\mathcal{R}, +, \cdot)$ is an abelian group under $+$ with an associative, commutative multiplication distributing over $+$ and possessing an identity 1. An element $u \in \mathcal{R}$ is a ***unit*** when $uv = 1$ for some v, and a ***zero divisor*** when $u \neq 0$ and $uv = 0$ for some $v \neq 0$. A ***field*** is a commutative ring in which every nonzero element is a unit.

Proposition 3.2. In $\mathbb{Z}_m$, an element a is a unit if and only if $\gcd(a, m) = 1$. In particular $\mathbb{Z}_{2^k}$ has exactly 2^{k-1} units, namely the odd residues, and every even nonzero residue is a zero divisor; $\mathbb{Z}_p$ for p prime is a field.

Proof. If $\gcd(a, m) = 1$ then by Bézout there are u, v with $au + mv = 1$, so $au \equiv 1 \pmod{m}$. Conversely if $au \equiv 1 \pmod{m}$ then $au - 1 = mv$ for some v, so any common divisor of a and m divides 1. For $m = 2^k$ the condition $\gcd(a, 2^k) = 1$ says exactly that a is odd, and there are 2^{k-1} odd residues; if $a = 2^s b$ with $1 \leq s < k$ then $a \cdot 2^{k-s} \equiv 0$, so a is a zero divisor. For $m = p$ prime, every $a \in \{1, \ldots, p-1\}$ is coprime to p. □

Proposition 3.2 has three consequences that recur in this survey, and we state them together because each is a place where a textbook protocol has to be modified before it can run over machine words.

- **Shamir sharing does not transfer.** Reconstruction interpolates a polynomial by Lagrange's formula, whose coefficients are differences of evaluation points inverted. Over $\mathbb{Z}_{2^k}$ those differences are frequently even and therefore not invertible, so the scheme is not well defined for arbitrary evaluation points. This is why our ring-based layer uses *replicated* instead of threshold sharing (Definition 6.3), and why the field-based engines in our compiler use a Mersenne prime instead.
- **Statistical masking must respect the power-of-two structure.** A mask uniform on $\mathbb{Z}_{2^k}$ hides perfectly (Theorem 6.1), but a mask uniform on a subinterval does not, and the failure is not visible to any correctness test, since the arithmetic is unaffected. An instance is recorded in Appendix A.

- **Truncation is natural.** Division by 2^d with rounding is a shift on a machine word, and fixed-point arithmetic wants precisely that operation. Over $\mathbb{F}_p$ the corresponding operation needs a comparison against $p/2$ to recover the sign, which is why the field engines pay for a wider statistical margin. Section 3.5 makes the trade quantitative.

Theorem 3.1 (Chinese remainder)**.** Let $m = m_1 m_2 \cdots m_r$ with the m_i pairwise coprime. Then the map $x \mapsto (x \bmod m_1, \ldots, x \bmod m_r)$ is a ring isomorphism $\mathbb{Z}_m \cong \mathbb{Z}_{m_1} \times \cdots \times \mathbb{Z}_{m_r}$.

Proof. The map is a ring homomorphism because reduction modulo each m_i is. It is injective: if $x \equiv y$ modulo every m_i then each m_i divides $x - y$, and since the m_i are pairwise coprime their product does, so $x = y$ in $\mathbb{Z}_m$. Both sides have cardinality m, so an injective map between them is bijective. □

Theorem 3.1 is used in two distinct ways below, and conflating them is a common source of confusion. In the *implementation* of lattice arithmetic it is a representation device: a large modulus q is stored as a residue system over machine-word primes, which is what makes a number-theoretic transform practical (Section 3.4). In the *plaintext* space of a homomorphic scheme it is a parallelism device: the plaintext ring factors into independent slots, so one ciphertext carries many independent values (Section 4.9). The first is invisible to the protocol; the second changes its asymptotics.

Definition 3.5. For a commutative ring $\mathcal{R}$ we write $\mathcal{R}[X]$ for the ring of polynomials with coefficients in $\mathcal{R}$. For $N \in \mathbb{N}$ the ***negacyclic*** quotient is $\mathcal{R}[X]/(X^N + 1)$, and the ***cyclic*** quotient is $\mathcal{R}[X]/(X^N - 1)$. We write $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ and $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$ for the rings used by the homomorphic schemes of Section 6.4, with N a power of two.

For N a power of two, $X^N + 1$ is the $2N$-th cyclotomic polynomial and is irreducible over $\mathbb{Q}$, so $\mathbb{Q}[X]/(X^N + 1)$ is a field and $\mathcal{R}$ is the ring of integers of a cyclotomic number field. Two properties of the negacyclic quotient are used constantly. First, reduction is free: $X^N \equiv -1$, so a product of two degree-$(N-1)$ polynomials is reduced by subtracting the high half from the low half not by long division. Second, multiplication is a ***negacyclic convolution***,

$$(a \cdot b)_k \;=\; \sum_{i+j=k} a_i b_j \;-\; \sum_{i+j=k+N} a_i b_j, \tag{1}$$

which is the identity that makes coefficient packing work (Section 4.9) and which our convolution benchmark exploits.

3.4 Fast polynomial multiplication

Homomorphic evaluation is dominated by multiplication in $\mathcal{R}_q$, so the cost of Equation (1) determines the cost of everything above it. Done directly it is $\Theta(N^2)$ coefficient products. The number-theoretic transform reduces it to $\Theta(N \log N)$, and since we report a $40.6\times$ speed-up from implementing it, we give the construction.

Definition 3.6. Let q be a prime with $q \equiv 1 \pmod{2N}$, let $\psi \in \mathbb{Z}_q$ be a primitive $2N$-th root of unity, and put $\omega = \psi^2$, a primitive N-th root. The ***negacyclic number-theoretic transform*** of $\mathbf{a} = (a_0, \ldots, a_{N-1})$ is

$$\widehat{a}_j \;=\; \sum_{i=0}^{N-1} a_i \, \psi^i \, \omega^{ij} \qquad (0 \le j < N),$$

that is, the N-point discrete transform applied to the sequence $(a_i \psi^i)_i$.

Theorem 3.2. With the notation of Definition 3.6, for all $a, b \in \mathcal{R}_q$ the negacyclic product satisfies $\widehat{(a \cdot b)}_j = \widehat{a}_j \, \widehat{b}_j$ for every j, and the transform is invertible with $a_i = N^{-1} \psi^{-i} \sum_j \widehat{a}_j \omega^{-ij}$. Hence a negacyclic product costs two forward transforms, N coefficient products and one inverse transform.

Proof. Write $A(X) = \sum_i a_i X^i$ and let $c = a \cdot b$ in $\mathbb{Z}_q[X]/(X^N + 1)$. Because $\psi^N = -1$, the map $X \mapsto \psi$ sends $X^N + 1$ to 0, so evaluation at $\psi\omega^j$ is a ring homomorphism from $\mathbb{Z}_q[X]/(X^N + 1)$ to $\mathbb{Z}_q$ for each j: indeed $(\psi\omega^j)^N = \psi^N \omega^{jN} = -1$. Now $\widehat{a}_j = \sum_i a_i \psi^i \omega^{ij} = A(\psi\omega^j)$, so $\widehat{c}_j = C(\psi\omega^j) = A(\psi\omega^j) B(\psi\omega^j) = \widehat{a}_j \widehat{b}_j$, which is the first claim. The points $\{\psi\omega^j\}_{j<N}$ are distinct and are exactly the N roots of $X^N + 1$ in $\mathbb{Z}_q$, which exist because $q \equiv 1 \pmod{2N}$; a polynomial of degree less than N is determined by its values at N distinct points, so the transform is a bijection, and the stated inversion formula follows from the orthogonality relation $\sum_j \omega^{j(i-i')} = N \, [i = i']$ in $\mathbb{Z}_q$. □

Remark 3.2. Theorem 3.2 is a statement about $\mathbb{Z}_q$ for prime q. Deployed parameters use a composite q, and the standard resolution is Theorem 3.1: represent $\mathbb{Z}_q$ as a residue number system over machine-word primes each satisfying $q_i \equiv 1 \pmod{2N}$, and transform independently in each. What remains after that is the cost of a single butterfly, and the engineering lesson we report is that this cost is dominated by modular reduction not by memory traffic, which is why a Shoup-preconditioned multiply is worth more than a better loop order.

3.5 Fixed-point representation and its two error terms

Neither secret sharing over $\mathbb{Z}_{2^k}$ nor an integer homomorphic scheme offers real arithmetic, and the networks we evaluate are specified over the reals. The bridge is fixed point, and because two of our findings are about the cost of the rescaling it forces, we set out its arithmetic precisely.

Definition 3.7. Fix a ***scale*** $f \in \mathbb{N}$. The ***fixed-point encoding*** of $x \in \mathbb{R}$ at scale f is $\lfloor x \cdot 2^f \rceil \in \mathbb{Z}$, and a stored integer X decodes to $X \cdot 2^{-f}$. We say X ***represents*** x with error $\left| X 2^{-f} - x \right| \le 2^{-f-1}$.

Addition of encodings at a common scale is exact and needs no adjustment. Multiplication does not preserve the scale: if X and Y encode x and y at scale f then XY encodes xy at scale $2f$, so a ***rescale*** by 2^{-f} is required before the value can be added to another scale-f quantity. Two error terms therefore accumulate, and they behave differently.

Proposition 3.3. Let X_i encode x_i and W_i encode w_i at scale f, for $i = 1, \dots, n$, with $|x_i|, |w_i| \le M$. Let $S = \left\lfloor 2^{-f} \sum_i W_i X_i \right\rceil$ be the rescaled inner product. Then

$$\left| S\, 2^{-f} - \textstyle\sum_i w_i x_i \right| \;\le\; \underbrace{n\, M\, 2^{-f}}_{\text{input rounding}} \;+\; \underbrace{2^{-f-1}}_{\text{one rescale}} \;+ O(n 2^{-2f}).$$

Proof. Write $X_i = 2^f x_i + \delta_i$ and $W_i = 2^f w_i + \epsilon_i$ with $|\delta_i|, |\epsilon_i| \le \frac{1}{2}$. Expanding the product pairs each encoded value with the *other* factor's rounding error,

$$W_i X_i \;=\; 2^{2f} w_i x_i \;+\; 2^f (w_i \delta_i + x_i \epsilon_i) \;+\; \delta_i \epsilon_i.$$

Summing over i and multiplying by 2^{-f},

$$2^{-f} \sum_i W_i X_i \;=\; 2^f \sum_i w_i x_i \;+\; \sum_i (w_i \delta_i + x_i \epsilon_i) \;+\; 2^{-f} \sum_i \delta_i \epsilon_i.$$

The second term is at most nM in absolute value, since $|w_i|, |x_i| \le M$ and each error is at most $\frac{1}{2}$; the third is at most $n 2^{-f}/4$. The final rounding to an integer contributes at most $\frac{1}{2}$. Dividing throughout by 2^f gives the claim. □

Proposition 3.3 is the reason the two paradigms diverge on nonlinearity cost, and it is worth stating why explicitly. The dominant term is input rounding, which is inherited from quantisation and is paid once per value regardless of protocol. The rescale term is a single half-unit, negligible in accuracy—and yet, in a secret-sharing protocol, the rescale is the expensive operation, because a right shift of a *secret* value is not a local operation over $\mathbb{Z}_{2^k}$: the shares of X do not shift independently, since the sum of the shifted shares differs from the shift of the sum by the carries lost at the boundary. Section 4.4 gives the protocol and its round cost. In a homomorphic scheme, by contrast, the rescale is either free (BFV/BGV at integer scale, where the plaintext modulus absorbs it) or a native operation on the modulus chain (CKKS, where it is exactly modulus switching). This asymmetry, and not the multiplication count, is what our measurements keep finding.

Remark 3.3. An activation choice interacts with the same arithmetic. For a comparison-free scheme the natural nonlinearity is $x \mapsto x^2$, which is exact in both paradigms. But squaring a scale-f encoding produces a scale-$2f$ value, so it forces a rescale of its own, and in a secret-sharing protocol that rescale costs exactly what a comparison would have cost. It also has a dynamic-range consequence: if a layer's linear map has gain g, then the magnitude of the activations obeys $M_{\ell+1} \approx g^2 M_\ell^2$, which for $gM > 1$ grows doubly exponentially in depth and overflows any fixed word length after a few layers. Scaling weights so that $gM \approx 1$—which is what standard initialisation does—puts the recursion at its fixed point. We report both the failure and the fix in Section 13.2.

3.6 Communication complexity and the cost model

Our comparisons are quoted in wall-clock time computed from measured counts, so the model that performs that conversion is part of the foundations and not a detail of the harness.

Definition 3.8. Fix a protocol execution. Its ***communication*** is the total number of bytes a single party sends, and its ***round complexity*** is the length of the longest chain of messages in which each depends on the previous one. Formally, orient the message graph by causal dependence; the round complexity is the height of that directed acyclic graph.

Definition 3.9. For a link with one-way latency L and per-party bandwidth B, the ***modelled latency*** of an execution with R rounds, C bytes of communication and T seconds of local computation is

$$\text{Time} \;=\; R \cdot L \;+\; C/B \;+\; T.$$

Definition 3.9 is a lower bound made into an estimate. It is a lower bound because a party cannot receive a causally dependent message sooner than L after sending its predecessor, and cannot transfer C bytes faster than B; it becomes an estimate by assuming perfect overlap within a round and no overlap between rounds. We validate it against a socket to within 13.6 % worst case (Section 12.2), and we record in Section 15 the one place where dropping T made a local-area comparison wrong by a factor of nineteen.

Proposition 3.4. Let a program be a directed acyclic graph of operations in which each operation is either ***local*** or ***interactive***, and suppose every set of interactive operations that are mutually independent can be executed in a single round. Schedule each operation at the earliest level permitted by its dependences. Then the number of rounds equals the maximum number of interactive operations on any directed path, and no schedule achieves fewer.

Proof. Write $d(v)$ for the maximum number of interactive operations on a path ending at v, so that d is computed by the recurrence $d(v) = \max_{u \to v} d(u) + [v \text{ interactive}]$. Assign v to level $d(v)$. Two interactive operations at the same level cannot be connected by a path, since a path between them would strictly increase d; hence each level's interactive operations are mutually independent and by hypothesis occupy one round, giving $\max_v d(v)$ rounds. Conversely, along a path carrying m interactive operations, each must complete before the next begins, so any schedule uses at least m rounds. □

Proposition 3.4 is why we report *dependence depth* not operation counts as the predictor of latency, and why batching a layer's comparisons is worth more than reducing their number: a layer of n independent comparisons costs one comparison's rounds. It also explains a structural feature of the assignment problem in Section 13.3: because a level costs the maximum over its members not the sum, the objective is not separable across operations, and an integer-linear formulation of it is not faithful.

4 Foundations of the protocols

Section 3 fixed the mathematics; this section builds the protocol machinery on it. Section 6 then presents the specific constructions our artifact implements, in the form it implements them. The division is that everything here holds for any implementation, whereas Section 6 records our choices.

4.1 The real/ideal paradigm

Definition 6.1 named the corruption models; we now give the security definition itself, because our composability argument (Theorem 6.5) quantifies over it and a reader cannot check that argument against an informal notion.

Definition 4.1. An ***ideal functionality*** $\mathcal{F}$ for n parties is a probabilistic interactive machine that receives an input x_i from each P_i, computes $(y_1, \ldots, y_n) \leftarrow \mathcal{F}(x_1, \ldots, x_n)$, and delivers y_i to P_i. A functionality may additionally be given a ***leakage*** function $\mathcal{L}$, whose value is delivered to the adversary; a functionality with no leakage function is ***leakage-free***.

Making leakage explicit in the functionality is not pedantry. A protocol that opens an intermediate value is not insecure if the functionality says the value is opened; it is insecure if the functionality does not. Our own broken truncation (Section 15) is exactly a case where the protocol opened slightly more than the functionality we had written down, and stating the functionality with its leakage clause is what made the discrepancy visible.

Definition 4.2. Let Π be an n-party protocol and $\mathcal{F}$ a functionality. For an adversary $\mathcal{A}$ corrupting a set $I \subseteq \{P_0, \ldots, P_{n-1}\}$ with $|I| \le t$, let $\mathrm{REAL}_{\Pi,\mathcal{A},I}(\mathbf{x}, \lambda)$ denote the joint distribution of $\mathcal{A}$'s view and the honest parties' outputs in an execution of Π on inputs $\mathbf{x}$. For a simulator $\mathcal{S}$, let $\mathrm{IDEAL}_{\mathcal{F},\mathcal{S},I}(\mathbf{x}, \lambda)$ denote the same joint distribution in an ideal execution where the parties hand their inputs to $\mathcal{F}$ and $\mathcal{S}$ plays the corrupted parties given only $\{x_i\}_{i \in I}$, $\{y_i\}_{i \in I}$ and any leakage. We say Π ***securely realises*** $\mathcal{F}$ against t static corruptions when for every such $\mathcal{A}$ there exists a probabilistic polynomial-time $\mathcal{S}$ with

$$\{\mathrm{REAL}_{\Pi,\mathcal{A},I}(\mathbf{x}, \lambda)\} \;\approx\; \{\mathrm{IDEAL}_{\mathcal{F},\mathcal{S},I}(\mathbf{x}, \lambda)\}$$

for all $\mathbf{x}$ and all admissible I, where $\approx$ is perfect, statistical or computational indistinguishability as specified.

Three features of Definition 4.2 matter to us. It is a *joint* distribution over the adversary's view and the honest outputs, so a protocol cannot be excused for letting the adversary influence the result while learning nothing. The simulator is given the corrupted parties' outputs, which is what makes simulation possible at all: whatever the adversary sees must be reconstructible from what it is entitled to know. And corruption here is ***static***, fixed before the execution; ***adaptive*** corruption, where the adversary chooses whom to corrupt as it goes, is strictly harder and we do not claim it.

Remark 4.1. Definition 4.2 says nothing about whether honest parties obtain output. Three increasingly strong guarantees are distinguished in the literature: ***security with abort***, where honest parties may halt without output; ***fairness***, where the adversary obtains output only if honest parties do; and ***guaranteed output delivery***. With a dishonest majority the last two are unachievable in general—Cleve's impossibility result for coin flipping applies already to two parties—so a dishonest-majority protocol offering abort is not making a weak choice but the only available one. Our artifact offers abort in an honest-majority setting, where the stronger guarantees are achievable and we do not achieve them; we say so rather than imply otherwise.

4.2 Secret sharing

Definition 4.3. Let $\mathcal{R}$ be a finite commutative ring and let $\mathcal{A}$ be a monotone collection of subsets of $\{0, \dots, n-1\}$, the ***authorised sets***. A ***secret-sharing scheme*** is a pair of algorithms $(\mathsf{Share}, \mathsf{Rec})$ where $\mathsf{Share}(x)$ outputs $(s_0, \dots, s_{n-1})$ and Rec maps the shares held by any authorised set back to x. It is ***correct*** when Rec succeeds for every authorised set, and ***perfectly private*** when for every unauthorised set $U \notin \mathcal{A}$ and every pair of secrets x, x', the distributions of $\{s_i\}_{i \in U}$ under $\mathsf{Share}(x)$ and $\mathsf{Share}(x')$ are identical. It is ***linear*** when $\mathsf{Share}(x) + \mathsf{Share}(y)$ is a valid sharing of $x + y$ under componentwise addition.

Linearity is the property that makes secret sharing useful for computation: addition and multiplication by a public constant are local, so only multiplication of two secrets needs interaction, and the entire cost structure of the paradigm follows from that split.

Proposition 4.1. Additive sharing over $\mathcal{R}$, in which $s_0, \dots, s_{n-2}$ are uniform and $s_{n-1} = x - \sum_{i<n-1} s_i$, is correct for the single authorised set $\{0, \dots, n-1\}$ and perfectly private against every proper subset.

Proof. Correctness is immediate. For privacy, let U be a proper subset, so that U omits at least one index. There are two cases. If $n-1 \notin U$, then $\{s_i\}_{i \in U}$ is a subset of the independent uniform values $s_0, \dots, s_{n-2}$, whose joint distribution does not involve x at all. If $n-1 \in U$, pick $j \leq n-2$ with $j \notin U$, which exists because U is proper. Condition on $\{s_i\}_{i \in U \setminus \{n-1\}}$, all of which are uniform and independent of x. Then $s_{n-1} = x - \sum_{i \leq n-2} s_i$ contains the term $-s_j$ with s_j uniform, unobserved and independent of everything conditioned on, so s_{n-1} is uniform on $\mathcal{R}$ and independent of x. In both cases the observed tuple is distributed identically for every secret. We remark that this argument uses only that $\mathcal{R}$ is an abelian group under addition, so it holds over $\mathbb{Z}_{2^k}$ despite Proposition 3.2—which is exactly why additive and replicated sharing transfer to machine words while Shamir sharing does not. □

Proposition 4.2. Replicated sharing among three parties (Definition 6.3), in which P_i holds (x_i, x_{i+1}) for an additive sharing $x_0 + x_1 + x_2 = x$, is correct for any two parties and perfectly private against any one.

Proof. Any two parties jointly hold all three additive shares: P_i and P_{i+1} hold $\{x_i, x_{i+1}\}$ and $\{x_{i+1}, x_{i+2}\}$, whose union is all of them, so reconstruction succeeds. A single party P_i holds (x_i, x_{i+1}) and is missing x_{i+2}, which is uniform and independent of the pair it holds; by the argument of Proposition 4.1 the observed pair is identically distributed for every secret. □

Definition 4.4. Let $\mathbb{F}$ be a finite field with $|\mathbb{F}| > n$ and fix distinct nonzero evaluation points $\alpha_0, \dots, \alpha_{n-1}$. ***Shamir sharing*** of x with threshold t samples a uniform polynomial p of degree at most t subject to $p(0) = x$ and sets $s_i = p(\alpha_i)$. The authorised sets are those of size at least $t+1$.

Proposition 4.3. Shamir sharing is correct and perfectly private, and it is linear.

Proof. Correctness is Lagrange interpolation: given $t+1$ points, the unique polynomial of degree at most t through them is $p(X) = \sum_j s_{i_j} \prod_{l \neq j} \frac{X - \alpha_{i_l}}{\alpha_{i_j} - \alpha_{i_l}}$, and $x = p(0)$. This is where a field is

needed: the denominators are differences of distinct evaluation points and must be invertible, which Proposition 3.2 shows can fail over $\mathbb{Z}_{2^k}$. For privacy, fix an unauthorised set of size t and any candidate secret x'; the t observed values together with $p(0) = x'$ determine $t + 1$ conditions on a polynomial with $t + 1$ free coefficients, so exactly one polynomial is consistent, and the map from the sampling randomness to the observed shares is a bijection for every x'. Linearity holds because the sum of two polynomials of degree at most t has degree at most t and evaluates pointwise. □

Remark 4.2. Multiplying two Shamir sharings pointwise yields a sharing of the product on a polynomial of degree up to $2t$, which is not a valid degree-t sharing. The standard repair, ***degree reduction***, has each party reshare its product and the parties take a fixed linear combination; it costs one round and is why a Shamir multiplication is interactive even with an honest majority. Replicated sharing avoids the degree problem entirely at the cost of storing two shares per party, which is the trade Definition 6.3 makes.

4.3 Multiplication: from correlated randomness, and from triples

Definition 4.5. A ***Beaver triple*** over $\mathcal{R}$ is a sharing $(\llbracket a \rrbracket, \llbracket b \rrbracket, \llbracket c \rrbracket)$ with a, b uniform and $c = ab$.

Theorem 4.1. Given a Beaver triple and sharings $\llbracket x \rrbracket, \llbracket y \rrbracket$, the protocol that opens $d = x - a$ and $e = y - b$ and outputs $\llbracket z \rrbracket = \llbracket c \rrbracket + d\llbracket b \rrbracket + e\llbracket a \rrbracket + de$ is correct, uses one round and two openings, and leaks nothing: the opened values are uniform and independent of x, y.

Proof. For correctness, expand using $c = ab$:

$$c+db+ea+de = ab+(x-a)b+(y-b)a+(x-a)(y-b) = ab+xb-ab+ya-ab+xy-xb-ya+ab = xy.$$

Each term on the left is either a sharing scaled by a public value or a public constant, so the combination is local and the result is a valid sharing by linearity. For privacy, a is uniform and independent of x, so $d = x - a$ is uniform on $\mathcal{R}$ and independent of x; likewise e. A simulator therefore samples d, e uniformly, which is exactly the real distribution, and the simulation is perfect. Both openings are independent and occupy one round. □

Theorem 4.1 explains the shape of the dishonest-majority cost we measure: the online phase is two openings and is cheap, and the entire expense is producing $(\llbracket a \rrbracket, \llbracket b \rrbracket, \llbracket c \rrbracket)$, which needs either oblivious transfer or a homomorphic scheme. Section 6.2 gives the honest-majority alternative, Theorem 6.2, in which correlated randomness from a pairwise pseudorandom function replaces the triple and the multiplication needs no preprocessing at all. That contrast—preprocessing volume against threat model—is the mechanism behind the crossover of Section 13.1.

Proposition 4.4. Let each party P_i contribute a uniform bit $b_i \in \{0, 1\}$ as a private input. Then $b = \bigoplus_i b_i$ is a uniform bit unknown to any proper subset of the parties, and it can be computed from the $\llbracket b_i \rrbracket$ by $\lceil \log n \rceil$ rounds of multiplication using the identity $u \oplus v = u + v - 2uv$ over $\mathcal{R}$ with $u, v \in \{0, 1\}$.

Proof. The identity holds by inspection on the four cases. Uniformity: conditioned on any proper subset of the b_i, at least one b_j remains uniform and independent, and exclusive-or with a uniform independent bit yields a uniform bit. Balanced binary reduction of n terms has depth $\lceil \log n \rceil$, and all exclusive-ors at one level are independent, so by Proposition 3.4 each level costs one round. □

Proposition 4.4 is how our engines obtain shared random bits without a trusted dealer, and it prices them: one multiplication per bit per level, hence one Beaver triple per bit at two parties. We record in Section 15 that treating these bits as free—the usual "fake offline" simplification—understated the cost of a comparison by a large factor, because a comparison consumes tens of them.

4.4 Comparison, bit decomposition and truncation

Every nonlinearity we evaluate reduces to the sign of a shared value: $\mathrm{ReLU}(x) = x \cdot [x > 0]$, a maximum is a mux on a comparison, and an argmax is a fold of maxima. Over $\mathbb{Z}_{2^k}$ the sign is the top bit, so the primitive is bit decomposition, and its cost dominates our secret-sharing measurements.

Definition 4.6. Given $[\![x]\!]$ with x known to lie in $[-2^{\ell-1}, 2^{\ell-1})$, a ***bit-decomposition*** protocol outputs sharings $[\![x_0]\!], \dots, [\![x_{\ell-1}]\!]$ of the two's-complement bits of x.

The construction we use is the masked-opening one. Draw $\ell + \kappa$ shared random bits r_j (Proposition 4.4), form $[\![r]\!] = \sum_j 2^j [\![r_j]\!]$ locally, open $c = x + 2^{\ell-1} + r$, and recover the bits of $u = x + 2^{\ell-1}$ as the binary difference $c - r$ computed on the shared bits of r against the public bits of c. Two facts make this work, and we prove both because the constants are ones we quote.

Lemma 4.1. With $r = \sum_{j<\ell+\kappa} 2^j r_j$ for independent uniform bits r_j, and $u \in [0, 2^\ell)$, the opened value $c = u + r$ has statistical distance at most $2^{-\kappa}$ from a distribution independent of u.

Proof. r is uniform on $[0, 2^{\ell+\kappa})$. By the argument of Lemma 3.2 applied to integer intervals, shifting a uniform interval of cardinality $2^{\ell+\kappa}$ by $u \le 2^\ell - 1$ leaves at most u points uncovered at each end, so $\Delta(r, r+u) \le u/2^{\ell+\kappa} < 2^\ell/2^{\ell+\kappa} = 2^{-\kappa}$. Taking T as in Definition 3.2 gives the claim for any distinguisher. □

Remark 4.3. Lemma 4.1 is why a comparison at ℓ bits consumes $\ell + \kappa$ random bits rather than ℓ, and therefore why the arithmetic domain must be at least $\ell + \kappa + 1$ bits wide: the masked sum must not wrap. With $\ell = 33$ and $\kappa = 26$ this is 60 bits, which is precisely the constraint that forces the field of our somewhat-homomorphic preprocessing engine to be 61 bits and hence forces its ciphertext modulus to be large. A reader tracking why those parameters are what they are should start here.

Theorem 4.2 (Parallel-prefix borrow)**.** Let c be public and let the bits r_j of r be shared. The bits of $u = c - r$ can be computed in $\lceil \log \ell \rceil$ rounds of shared multiplication, independently of ℓ's value beyond that logarithm.

Proof. Write the borrow-out of position j as β_j, with $\beta_{-1} = 0$ and

$$\beta_j \;=\; g_j \;\vee\; (p_j \wedge \beta_{j-1}), \qquad g_j = \neg c_j \wedge r_j, \qquad p_j = \neg(c_j \oplus r_j),$$

and the difference bit $u_j = c_j \oplus r_j \oplus \beta_{j-1}$. Because c is *public*, g_j and p_j are local: each collapses to r_j or $1 - r_j$ according to the public bit c_j, so no interaction is needed to form them. Define the associative operator $(g, p) \circ (g', p') = (g \vee (p \wedge g'),\ p \wedge p')$, whose associativity is checked by expanding both bracketings. Then β_j is the first component of $(g_j, p_j) \circ \cdots \circ (g_0, p_0)$, and a balanced prefix network (Kogge–Stone) computes all ℓ prefixes in $\lceil \log \ell \rceil$ levels. It remains to observe that $g_j p_j = 0$ always—if $c_j = 0$ then $g_j = r_j$ and $p_j = 1 - r_j$, and if $c_j = 1$ then $g_j = 0$—so the disjunction in $\circ$ may be replaced by addition, making each application two shared multiplications. All applications at one level are independent, so by Proposition 3.4 each level is one round. □

Corolary 4.1. A comparison of two ℓ-bit shared values costs one opening, $\ell + \kappa$ shared random bits, and $\lceil \log(\ell + 1) \rceil$ rounds of multiplication; a batch of any number of independent comparisons costs the same number of rounds as one.

Proof. Compare x against y by decomposing $x - y$ and taking the sign bit, which is the top bit of the two's-complement representation; the counts are those of Lemma 4.1 and Theorem 4.2 with $\ell + 1$ bits to accommodate the difference. The batching claim is Proposition 3.4 applied to a set of mutually independent decompositions. □

Corollary 4.1 is the single most consequential statement in this section for our measurements. It says that the nonlinear surface of a network costs *rounds proportional to depth and not to width*, which is why our latency figures scale with the number of layers and are almost independent of the number of neurons per layer, and why the dominant term in a comparison's cost is the random bits instead of the prefix network.

Proposition 4.5 (Truncation)**.** Let $[\![x]\!]$ be given with the bits of x available as $[\![x_j]\!]$, and let $d < \ell$. Then $[\![\lfloor x/2^d \rfloor]\!]$ is obtained locally as

$$[\![\lfloor x/2^d \rfloor]\!] \;=\; \sum_{j=d}^{\ell-1} 2^{\,j-d}\,[\![x_j]\!] \;-\; 2^{\,\ell-d}\,[\![x_{\ell-1}]\!],$$

that is, by recomposing the high bits and then correcting the sign; no interaction is required beyond the decomposition.

Proof. In two's complement over ℓ bits, $x = -2^{\ell-1}x_{\ell-1} + \sum_{j<\ell-1} 2^j x_j$. An arithmetic shift right by d is floor division by 2^d, and $\lfloor x/2^d \rfloor = -2^{\ell-1-d}x_{\ell-1} + \sum_{d\le j<\ell-1} 2^{j-d}x_j$, because discarding the low d bits of a two's-complement representation is exactly flooring. The displayed expression recomposes $\sum_{d\le j<\ell} 2^{j-d}x_j$, which includes $+2^{\ell-1-d}x_{\ell-1}$, and then subtracts $2^{\ell-d}x_{\ell-1} = 2\cdot 2^{\ell-1-d}x_{\ell-1}$, which converts that term to $-2^{\ell-1-d}x_{\ell-1}$. All operations are additions and public scalings, so they are local. □

Remark 4.4. The sign correction in Proposition 4.5 is easy to omit, and omitting it yields a *logical* and not an arithmetic shift: the result is correct for non-negative inputs and wrong for every negative one. Since roughly half of a network's pre-activations are negative, the error is not subtle in effect—but it is invisible to any test whose fixtures are non-negative. This is a case where the natural test set and the failure set are disjoint, which is why the correctness condition is worth stating as a proposition not left to the implementation.

4.5 Oblivious transfer and its extensions

Dishonest-majority preprocessing rests on oblivious transfer, so we define it and give the two constructions whose costs we measure.

Definition 4.7. The functionality $\mathcal{F}_{\mathrm{OT}}$ takes messages (m_0, m_1) from a sender and a bit b from a receiver, and delivers m_b to the receiver and nothing to the sender. In ***random OT*** the sender has no input and receives uniform (m_0, m_1) as output; in ***correlated OT*** with correlation Δ the sender inputs Δ, receives uniform m_0, and the receiver obtains $m_b = m_0 + b\Delta$.

Proposition 4.6 (Gilboa)**.** Let P_0 hold $x \in \mathbb{Z}_{2^k}$ and P_1 hold $y \in \mathbb{Z}_{2^k}$. Using k correlated OTs in which P_0 is the receiver with choice bits the binary digits of x and P_1 offers correlation $2^i y$ on the i-th transfer, the parties obtain an additive sharing of xy.

Proof. On transfer i, P_1 holds a uniform r_i and P_0 obtains $r_i + x_i 2^i y$ where x_i is the i-th bit of x. Summing, P_0 holds $\sum_i r_i + y \sum_i x_i 2^i = \sum_i r_i + xy$ and P_1 holds $-\sum_i r_i$; the two sum to xy. Privacy is inherited from the OT: P_0 sees only its selected outputs, which are uniform given the unselected ones, and P_1 learns nothing about the choice bits. □

Proposition 4.6 prices a Beaver triple over $\mathbb{Z}_{2^k}$ at $2k$ correlated OTs, since a triple needs the two cross terms $a_0 b_1$ and $a_1 b_0$. With $k = 64$ and a per-OT cost of κ bits plus a correction, this is the ~ 3 kilobytes per triple that our measurements confirm to within one per cent, and it is the quantity against which somewhat-homomorphic preprocessing competes.

Remark 4.5. Base oblivious transfer costs public-key operations, so a protocol needing millions of them uses an ***extension***: a small number of base OTs are turned into arbitrarily many by symmetric-key work, at the cost of a correlation the receiver's choices induce across the outputs. The construction transposes a matrix of pseudorandom columns so that the sender learns, for each row, one of two values determined by a global secret Δ, and hashing breaks the correlation. A malicious receiver can use inconsistent choice bits across columns, which leaks bits of Δ; a consistency check based on a random linear combination over $\mathbb{F}_{2^{128}}$ closes this at $O(1)$ additional communication. Our layer implements the extension and the check, and we report that the extension is $27\times$ cheaper per transfer than the base protocol at batch size 8192.

4.6 Garbling schemes

Definitions 6.4–6.6 and Theorem 6.3 give the constructions we implement; here we give the security notions they must satisfy, because the cheapest optimisations are exactly the ones that strengthen the assumption.

Definition 4.8. A garbling scheme $(\mathsf{Gb}, \mathsf{En}, \mathsf{Ev}, \mathsf{De})$ is ***correct*** when $\mathsf{De}(d, \mathsf{Ev}(F, \mathsf{En}(e, x))) = f(x)$ for all x. It is ***private*** when there is a simulator that, given f and $f(x)$, produces $(F, \mathsf{En}(e, x), d)$ indistinguishable from a real garbling; ***oblivious*** when a simulator given only f produces $(F, \mathsf{En}(e, x))$ indistinguishably, that is, the evaluator learns not even the output; and ***authentic*** when no adversary given $(F, \mathsf{En}(e, x))$ can produce an output label that decodes to anything other than $f(x)$.

Privacy is what a semi-honest evaluator requires. Authenticity is what makes a malicious evaluator unable to forge a result, and it is why the garbled-circuit route can be given active security cheaply on the evaluator's side. Obliviousness matters in our setting because the garbled output is kept as a sharing not decoded, which is exactly how a garbled comparison composes with an arithmetic protocol.

Remark 4.6. Free exclusive-or replaces a garbled table by a wire-label offset, so that an XOR gate costs nothing. The price is the assumption: security of the scheme now requires the hash to be ***circular correlation robust***, because labels related by the global offset R are hashed together, whereas the plain construction needs only a pseudorandom function. Half-gates then reduce an AND gate from three ciphertexts to two while remaining compatible with free exclusive-or. The lesson we take from this for our cost comparison is that the garbled route's headline figure of two ciphertexts per AND is not assumption-free, and a fair comparison against an arithmetic protocol whose security is information-theoretic should say so.

4.7 Lattices, LWE and the hardness assumptions

Definition 6.7 introduces lattices and Definition 6.8 the ring problem our schemes rest on. Here we state the assumptions in the form that parameter selection uses, since our security claims about the homomorphic layer are exactly claims about these.

Definition 4.9. Let $n, q \in \mathbb{N}$ and let χ be a distribution over $\mathbb{Z}$ concentrated on small values. The ***search LWE*** problem is: given samples $(\mathbf{a}_i, \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i \bmod q)$ with $\mathbf{a}_i \leftarrow \mathbb{Z}_q^n$, $e_i \leftarrow \chi$ and a fixed secret $\mathbf{s} \in \mathbb{Z}_q^n$, recover $\mathbf{s}$. The ***decision LWE*** problem is to distinguish such samples from uniform samples over $\mathbb{Z}_q^n \times \mathbb{Z}_q$.

Definition 4.10. The ***ring*** variant replaces $\mathbb{Z}_q^n$ by $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$: samples are $(a_i, a_i s + e_i)$ with $a_i \leftarrow \mathcal{R}_q$, $s \in \mathcal{R}_q$ fixed and e_i drawn coefficientwise from χ. A single ring sample carries N scalar samples, so the public key is smaller by a factor of N and multiplication becomes a ring product instead of a matrix–vector product.

The assumptions we rely on are that decision LWE and decision ring-LWE are hard for suitable parameters. Two facts about them shape our practice and not our proofs. First, search-to-decision and worst-case-to-average-case reductions are known, which is why these problems are used at all; we cite not reprove them. Second, and this is what a parameter table actually depends on, the *concrete* hardness is estimated by running the best known attacks—lattice reduction with a sieving or enumeration subroutine, plus algebraic and hybrid attacks—and reading off the estimated cost. Concrete security is therefore a moving target, and the honest way to state a parameter set is to name the estimator and its version. We do that in Section 6.4 and we do not claim security beyond what the standard tables assert.

Remark 4.7. The distinction between "a problem is hard" and "these parameters are hard" is not academic for us. We implemented a scheme whose security rested on a variant assumption, and the variant admitted a subfield-lattice attack for the parameter shape we had chosen; the construction was removed not reparameterised. We record such a case in Appendix A. A survey that lists only the primitives that survived would give a misleading picture of how the choice between them is made.

4.8 Homomorphic encryption: noise, depth and the operations that manage them

Definition 6.9 gives the two integer schemes we implement. This subsection derives the noise arithmetic, because every structural claim we make about homomorphic evaluation—why depth is

bounded, why a bootstrap is needed, why a scheme switch can replace one—is a statement about the growth of a single quantity.

Let $\mathsf{ct} = (c_0, c_1) \in \mathcal{R}_q^2$ be an encryption of m under secret s, with the invariant, for the BFV encoding,

$$c_0 + c_1 s \;=\; \Delta m + e \pmod{q}, \qquad \Delta = \lfloor q/t \rfloor, \tag{2}$$

where t is the plaintext modulus and e is the noise. Decryption computes $c_0 + c_1 s$, divides by Δ and rounds.

Proposition 4.7. Decryption of Equation (2) is correct provided $\|e\|_\infty < \Delta/2$.

Proof. We have $c_0 + c_1 s = \Delta m + e$ over $\mathcal{R}_q$, and lifting to centred representatives, $\lfloor (\Delta m + e)/\Delta \rceil = m + \lfloor e/\Delta \rceil = m$ whenever every coefficient of e has magnitude below $\Delta/2$, since then $|e_i/\Delta| < 1/2$ and rounds to zero. If any coefficient exceeds the bound the corresponding plaintext coefficient is off by at least one, and because plaintexts are taken modulo t the error is not detectable from the ciphertext. □

Proposition 4.7 is the entire budget. Everything homomorphic evaluation does either consumes it or restores it.

Proposition 4.8. Let $\mathsf{ct}_1, \mathsf{ct}_2$ encrypt m_1, m_2 with noise bounded by B_1, B_2. Then

1. addition yields noise at most $B_1 + B_2$;
2. multiplication by a plaintext of infinity-norm γ and Hamming-weight at most h yields noise at most $\gamma h B_1$;
3. ciphertext multiplication yields a degree-two ciphertext whose noise is bounded by approximately $tN(B_1 + B_2) + (\text{terms independent of } B_i)$, that is, it grows *multiplicatively* in the plaintext modulus and the ring dimension.

Proof. For (1), add the invariants: $(c_0 + c_0') + (c_1 + c_1')s = \Delta(m_1 + m_2) + (e_1 + e_2)$, and the triangle inequality bounds the sum. For (2), multiply the invariant by the plaintext p; each output coefficient is a sum of at most h products of a coefficient of p with one of e_1, whence the bound $\gamma h B_1$ by Equation (1) and the triangle inequality. For (3), multiply the two invariants:

$$(c_0 + c_1 s)(c_0' + c_1' s) = \Delta^2 m_1 m_2 + \Delta(m_1 e_2 + m_2 e_1) + e_1 e_2,$$

whose left side expands to a quadratic in s, that is, a degree-two ciphertext (d_0, d_1, d_2) with $d_0 + d_1 s + d_2 s^2$ equal to the right side. Dividing by Δ to restore the invariant scale introduces the factor t, and the convolution of two degree-$(N-1)$ polynomials contributes up to N terms per coefficient, giving the stated shape. The dominant term is $\Delta(m_1 e_2 + m_2 e_1)/\Delta = m_1 e_2 + m_2 e_1$, bounded by $tN(B_1 + B_2)/2$ since plaintext coefficients are bounded by $t/2$. □

Three consequences follow, and they are the structural facts our measurements keep meeting.

Corolary 4.2. With fresh noise B_0 and the multiplicative growth of Proposition 4.8(3), the number of sequential ciphertext multiplications before decryption fails—the ***multiplicative depth***—is $O(\log_{tN}(\Delta/B_0))$, and is therefore *logarithmic* in the ciphertext modulus and *linear* in $\log q$ for fixed t, N.

Proof. Each multiplication multiplies the noise bound by approximately tN, so after L of them the bound is roughly $B_0(tN)^L$. Decryption requires this to stay below $\Delta/2 \approx q/2t$, giving $L \leq \log_{tN}(q/(2tB_0))$. □

Remark 4.8. A ciphertext multiplication raises the degree in s from one to two, so a second multiplication would raise it to four and decryption would need s^4. ***Relinearisation*** restores degree one using an evaluation key that encrypts s^2 under s; it costs a key-switching operation and adds noise. A scheme that declines to relinearise is confined to *one* ciphertext multiplication, and that is exactly the regime our depth-one homomorphic layer occupies: it has no evaluation keys at all, which removes their generation, their storage and their noise, at the price of being unable to multiply twice. Whether that price is payable is the question our scheme-switching result answers.

Remark 4.9. ***Modulus switching*** rescales a ciphertext from modulus q to $q' < q$, dividing both the ciphertext and its noise by approximately q/q'. It does not improve the *ratio* of noise to modulus by itself, but in BGV it is used to keep that ratio constant after each multiplication, which converts multiplicative noise growth into a linear consumption of a precomputed ***modulus chain*** $q_L > q_{L-1} > \cdots > q_0$. The number of links is the depth budget, and this is why a levelled scheme's parameters are chosen from the depth of the circuit before evaluation begins.

Definition 4.11. ***Bootstrapping*** is homomorphic evaluation of the decryption circuit, taking a ciphertext whose noise is near the bound to a fresh ciphertext of the same plaintext with noise near B_0. A scheme with bootstrapping is ***fully*** homomorphic; one without is ***levelled***.

Bootstrapping is what makes unbounded depth possible and is, by a wide margin, the most expensive operation in the paradigm: it requires evaluating a nontrivial arithmetic circuit under the encryption it is repairing, at parameters chosen to accommodate that circuit. The measurements we cite from the literature put a single CKKS bootstrap at seconds to tens of seconds on a CPU, dominating the inference that contains it. That cost, and the fact that a levelled pipeline which declines to pay it simply stops at its depth bound, is the discontinuity our comparison in Section 13.2 is organised around.

Remark 4.10. Definition 4.11 refreshes noise by computing decryption homomorphically. A scheme switch achieves the same effect by a different route: the ciphertext is decrypted *under a sharing*, so the plaintext is never exposed, and re-encrypted fresh. The cost is a round of interaction with parties who hold the key in shares, instead of a homomorphic evaluation of decryption. Whether this is a better trade is precisely a measurement question, and it is the one our hybrid result answers: the switch is milliseconds where the bootstrap is seconds, and the price is that the parties must be online.

4.9 Packing: two encodings and the operations they admit

A ciphertext holds a ring element with N coefficients, and using it to carry one scalar wastes a factor of N. There are two ways to fill it, they support different operations, and choosing between them changes the asymptotics of a layer, so we set them out together.

Definition 4.12. ***Coefficient packing*** encodes a vector $\mathbf{a} \in \mathbb{Z}_t^N$ as the polynomial $\sum_i a_i X^i$. ***Slot packing*** fixes a plaintext modulus t with $t \equiv 1 \pmod{2N}$, so that $X^N + 1$ splits into distinct linear factors over $\mathbb{Z}_t$, and encodes $\mathbf{a}$ as the unique polynomial with $a(\zeta_j) = a_j$ at the N roots ζ_j; by Theorem 3.1 this is a ring isomorphism $\mathbb{Z}_t[X]/(X^N + 1) \cong \mathbb{Z}_t^N$.

Proposition 4.9. Under coefficient packing, the product of two encodings is the negacyclic convolution of the vectors, Equation (1). Under slot packing, the product is the componentwise product. Consequently a single ciphertext multiplication computes, respectively, all N shifted inner products of two vectors, or N independent scalar products.

Proof. The first claim is Definition 3.5 and Equation (1). The second is that evaluation at each ζ_j is a ring homomorphism, so $(ab)(\zeta_j) = a(\zeta_j)b(\zeta_j)$, which is exactly componentwise multiplication of the slot vectors; the isomorphism of Theorem 3.1 makes the correspondence bijective. □

Proposition 4.9 is the reason the two encodings suit different layers. A convolution is a shifted inner product, so coefficient packing computes a whole convolutional layer in one ciphertext multiplication with no rotations at all: the k^2 taps of a kernel land in the coefficients of a single product, and the desired outputs are read off at a stride. A matrix–vector product, by contrast, needs sums *across* positions, which coefficient packing does not provide and slot packing provides only through rotations.

Definition 4.13. A ***rotation*** by k maps the slot vector $(a_0, \ldots, a_{N-1})$ to its cyclic shift. Homomorphically it is realised by the Galois automorphism $X \mapsto X^g$ for a suitable g, followed by a key switch under a ***Galois key***, and costs one key-switching operation.

Proposition 4.10. A matrix–vector product with an $N \times N$ plaintext matrix on a slot-packed ciphertext can be computed with N plaintext multiplications and $N - 1$ rotations by summing the ***diagonals*** of the matrix, and with $O(\sqrt{N})$ rotations by the baby-step giant-step decomposition.

Proof. Write $\mathbf{y} = \mathbf{A}\mathbf{x}$ and let $\mathbf{d}_k$ be the k-th generalised diagonal, $d_{k,i} = A_{i,\, i+k \bmod N}$. Then $y_i = \sum_k d_{k,i} x_{i+k}$, so $\mathbf{y} = \sum_k \mathbf{d}_k \odot \mathrm{rot}_k(\mathbf{x})$, a sum of N componentwise products of a plaintext with a rotation. For the second bound, write $k = k_1\sqrt{N} + k_2$ and factor $\mathrm{rot}_k = \mathrm{rot}_{k_1\sqrt{N}} \circ \mathrm{rot}_{k_2}$: precompute the $\sqrt{N}$ inner rotations once, and apply $\sqrt{N}$ outer rotations to the accumulated partial sums, giving $2\sqrt{N}$ rotations in total. Correctness is the associativity of the shift action. □

Proposition 4.10 is the technique the compilers we compare against implement and optimise. We record it here because it makes the comparison legible: when we report that a homomorphic layer costs a fixed number of ciphertext operations while a secret-sharing layer costs traffic proportional to width, the homomorphic side's flatness is exactly this packing, and it is also why the homomorphic side's advantage grows with width in our measurements.

Remark 4.11. CKKS encodes *approximate* complex or real vectors rather than exact integers, using the canonical embedding: a vector is scaled by a factor Δ, rounded into the ring, and the rounding error is absorbed as additional noise. Its distinguishing operation is ***rescaling***, which divides a ciphertext by Δ after a multiplication and is implemented as modulus switching; the consequence is that CKKS's "noise" and its "precision" are the same quantity, and depth is consumed by the scale management rather than tracked separately. Two properties matter for our comparison. The scheme has no comparison operation and cannot compute a sign, so a nonlinearity must be a polynomial, which is why the only activation both paradigms evaluate exactly is a monomial (Remark 3.3). And precision degrades along the modulus chain, so a classification margin narrower than the accumulated error is not recoverable—a failure mode that presents as a wrong label and not as as an error signal, which is why our harness compares against a plaintext reference before quoting any timing.

Remark 4.12. In a ***threshold*** homomorphic scheme the secret key is shared and decryption is a protocol. The subtlety is that a partial decryption reveals a noise term that depends on the key share, so partial decryptions must be flooded by noise drawn from a range at least 2^{κ} times the bound—Lemma 3.2—and that flooding is subtracted from the same budget as the evaluation. The practical consequence, which we report as a disclosed gap and not a solved problem, is that a statistical security level for the decryption and a depth budget for the computation are drawn on the same account.

4.10 Succinct proofs

Section 6.7 describes the system we implement; we give the definitions here so that the claims in that section are checkable.

Definition 4.14. Let R be a relation and $L = \{x : \exists w,\ (x, w) \in R\}$. An ***interactive argument*** for R is a pair (P, V) of interactive machines with V polynomial-time, satisfying ***completeness***—for $(x, w) \in R$, V accepts with probability $1 - \mathrm{negl}$—and ***computational soundness***: for $x \notin L$ and every polynomial-time P^*, V accepts with negligible probability. It is an ***argument of knowledge*** when there is a polynomial-time extractor that, given oracle access to a prover convincing V with noticeable probability, outputs a witness. It is ***zero-knowledge*** when there is a simulator producing accepting transcripts indistinguishable from real ones without a witness.

Definition 4.15. A ***SNARK*** is a non-interactive argument of knowledge whose proof length and verification time are $o(|w|)$, typically polylogarithmic in the circuit size.

Two points of principle bear on how we report our zero-knowledge measurements. First, soundness in Definition 4.14 is *computational*: an unbounded prover can cheat, which is the price of succinctness, and it is why these objects are called arguments not proofs. Second, non-interactivity is normally obtained by the Fiat–Shamir transform, replacing the verifier's random challenges with a hash of the transcript; the transform is provable in the random-oracle model and its concrete instantiation is an assumption, so a proof system's security cannot be quoted independently of the hash it is instantiated with.

Remark 4.13. The gap between "a proof verifies" and "a proof proves the intended statement" is where our own zero-knowledge error lived: a backend can be complete and internally consistent while the constraint system it emits fails to bind the witness to the computation one intended, and no amount of successful verification detects it. The methodological consequence is that a proof system requires a ***negative*** test—a witness that ought to fail and does—and that a report of successful verifications alone is not evidence of soundness. Appendix A records an instance.

5 The workload: foundations of neural inference

Every measurement in this survey is of the same computation: a forward pass through a trained neural network in which the input, and in our case also the weights, are hidden from the party performing it. A reader cannot judge those measurements without knowing precisely which function is being computed, which of its parts are expensive under which cryptographic paradigm, and what an accuracy figure for it means. This section supplies that. It is deliberately elementary in the machine-learning direction and precise in the arithmetic direction, because the arithmetic is what the cryptography charges for.

5.1 Classification and the inference function

Definition 5.1. Let $\mathcal{X} \subseteq \mathbb{R}^{n_0}$ be an input space and $\mathcal{Y} = \{0, \dots, C-1\}$ a finite label set. A ***classifier*** is a map $h : \mathcal{X} \to \mathcal{Y}$. It is usually presented as $h = \arg\max \circ\, g$ where $g : \mathcal{X} \to \mathbb{R}^C$ produces a vector of ***logits*** and $\arg\max$ returns the index of the largest.

Training chooses g from data; ***inference*** evaluates it on one input. This survey measures inference only. The distinction matters because the two have different arithmetic: training needs gradients, and therefore division and accumulation across a batch, whereas inference needs a fixed sequence of affine maps and nonlinearities. Our artifact does implement private training, and we report it, but the comparisons in Section 13 are of inference.

Definition 5.2. A ***feedforward network*** of depth L is a composition

$$g \;=\; A_L \circ \sigma \circ A_{L-1} \circ \cdots \circ \sigma \circ A_1,$$

where each $A_\ell : \mathbb{R}^{n_{\ell-1}} \to \mathbb{R}^{n_\ell}$ is affine, $A_\ell(\mathbf{u}) = \mathbf{W}_\ell \mathbf{u} + \mathbf{b}_\ell$, and σ is a fixed nonlinearity applied coordinatewise. We call n_ℓ the ***width*** of layer ℓ and L the ***depth***. The parameters are $\theta = (\mathbf{W}_\ell, \mathbf{b}_\ell)_{\ell \le L}$.

Definition 5.2 already exposes the structure that governs every cost in this survey. The affine maps are *linear* in the shared or encrypted value, so under any linearly homomorphic representation—an additive secret sharing, or a lattice ciphertext—they require no interaction and no ciphertext multiplication beyond scaling by public constants. The nonlinearity σ is not linear, and it is the only place where a protocol must do something irreducibly expensive. The entire comparison between paradigms is therefore a comparison of how each one implements σ, and how many times it must.

Proposition 5.1. A network as in Definition 5.2 performs $\sum_\ell n_{\ell-1} n_\ell$ multiply–accumulate operations in its affine maps and $\sum_{\ell<L} n_\ell$ applications of σ. The first quantity is quadratic in width and the second is linear; consequently the ratio of linear to nonlinear work grows with width and is independent of depth.

Proof. The matrix $\mathbf{W}_\ell$ has $n_\ell n_{\ell-1}$ entries and each contributes one multiply–accumulate; σ is applied once per coordinate of each hidden layer, of which there are n_ℓ at layer $\ell < L$. The stated asymptotics follow by inspection: doubling every width multiplies the first sum by four and the second by two, while appending a layer adds one term to each. □

Proposition 5.1 is the reason our measurements report both quantities separately, and the reason the same network can look nonlinearity-dominated or arithmetic-dominated depending on its shape: at small width the σ applications dominate the cost even though they are asymptotically fewer, because each costs orders of magnitude more than a multiply–accumulate under secret sharing.

5.2 Nonlinearities, and which ones a cryptosystem can evaluate

Definition 5.3. The ***rectified linear unit*** is $\mathrm{ReLU}(x) = \max(x, 0)$. The ***square*** activation is $\mathrm{sq}(x) = x^2$. A ***polynomial activation*** of degree d is any $p \in \mathbb{R}[x]$ with $\deg p = d$.

Proposition 5.2. ReLU is 1-Lipschitz and non-expansive: $|\mathrm{ReLU}(x) - \mathrm{ReLU}(y)| \le |x - y|$ and $|\mathrm{ReLU}(x)| \le |x|$. Consequently, in a network whose affine maps have operator norm at most γ in the infinity norm, the activations satisfy $\|\mathbf{u}_\ell\|_\infty \le \gamma^\ell \|\mathbf{u}_0\|_\infty + O(\max_\ell \|\mathbf{b}_\ell\|_\infty)$. By contrast sq satisfies $\|\mathbf{u}_{\ell+1}\|_\infty \le (\gamma \|\mathbf{u}_\ell\|_\infty)^2$, so for $\gamma \|\mathbf{u}\|_\infty > 1$ the magnitude grows doubly exponentially in depth.

Proof. Both properties of ReLU follow from $\max(\cdot, 0)$ being a projection onto a convex set, or directly by cases. The propagation bound is then induction on ℓ: $\|\sigma(A_\ell \mathbf{u})\|_\infty \le \|A_\ell \mathbf{u}\|_\infty \le \gamma \|\mathbf{u}\|_\infty + \|\mathbf{b}_\ell\|_\infty$. For the square activation the same induction gives $\|\mathbf{u}_{\ell+1}\|_\infty = \|(A_\ell \mathbf{u}_\ell)^2\|_\infty \le (\gamma \|\mathbf{u}_\ell\|_\infty)^2$, and iterating a map of the form $m \mapsto (\gamma m)^2$ from m_0 gives $\log m_\ell = 2^\ell(\log m_0 + \log \gamma) - \log \gamma$, which diverges when $\gamma m_0 > 1$ and converges to zero when $\gamma m_0 < 1$. □

Proposition 5.2 is the analytic content behind a practical constraint we measure. A fixed-point or lattice representation carries a bounded dynamic range, so an activation whose iterates diverge doubly exponentially exhausts that range within a few layers, and one whose iterates converge to zero erases the signal. The square activation has a neutral point at $\gamma \|\mathbf{u}\|_\infty = 1$ and is unstable on both sides of it, whereas ReLU is neutral everywhere. Standard initialisation, which sets $\|\mathbf{W}\|$ to approximately $1/\sqrt{n_{\ell-1}}$ so that the layer has unit gain, places a square-activation network at precisely that neutral point, which is why such networks are trainable at all and why they must be trained rather than converted.

The cryptographic significance is a dichotomy.

Remark 5.1. Evaluating ReLU requires the *sign* of a hidden value, that is, a comparison. A secret-sharing protocol can compute a comparison exactly, at the cost of Corollary 4.1. An approximate homomorphic scheme over the reals cannot compute a comparison at all: its operations are addition and multiplication of ciphertexts, and the sign function is not a polynomial. Such a scheme must therefore replace σ by a polynomial approximation, paying in multiplicative depth for the degree and in accuracy for the residual error, or the network must be trained with a polynomial activation in the first place. The only activations both paradigms evaluate *exactly* are polynomials, and the cheapest nonconstant one is sq. This dichotomy, and not any difference in raw throughput, determines the shape of our matched comparison in Section 13.2.

5.3 Structured layers: convolution and pooling

Image classifiers do not use dense affine maps at every layer. The structured alternatives matter here because their structure is exactly what a packed ciphertext can exploit.

Definition 5.4. Let $\mathbf{u} \in \mathbb{R}^{c \times h \times w}$ be an input with c channels and let $\mathbf{K} \in \mathbb{R}^{c' \times c \times k \times k}$ be a kernel. The ***convolution*** with unit stride and no padding is

$$(\mathbf{K} * \mathbf{u})_{o,y,x} \;=\; \sum_{i=0}^{c-1} \sum_{a=0}^{k-1} \sum_{b=0}^{k-1} \mathbf{K}_{o,i,a,b}\, \mathbf{u}_{i,\, y+a,\, x+b},$$

for $o < c'$ and $y < h - k + 1$, $x < w - k + 1$. ***Max pooling*** with window s replaces each disjoint $s \times s$ window by its maximum, and ***average pooling*** by its mean.

Proposition 5.3. Convolution is linear in $\mathbf{u}$, and for fixed $\mathbf{K}$ it is a matrix–vector product with a matrix whose entries are the kernel entries, repeated with a shift structure. It performs $c'ck^2(h-k+1)(w-k+1)$ multiply–accumulate operations, and it applies no nonlinearity.

Proof. Linearity is immediate from the definition, which is a finite sum of scalar multiples of coordinates of $\mathbf{u}$. Flattening $\mathbf{u}$ and the output into vectors exhibits the map as a matrix; the count is the number of terms in the triple sum times the number of output positions. □

Proposition 5.3 says that a convolutional layer is free of nonlinearity, so by Proposition 5.1 its cost under secret sharing is traffic and not rounds. Its interaction with packing is sharper: Equation (1) showed that a product in a negacyclic ring is a shifted-and-signed convolution of coefficient vectors, and Definition 5.4 is a shifted inner product. Laying the input forwards and the kernel reversed therefore places an entire convolutional layer in the coefficients of a *single* ring product, with the wanted outputs at a stride and the unwanted ones discarded. That is the mechanism behind the technique we implement and measure, and it explains why the ciphertext cost of a convolution in our harness does not grow with the spatial size of the image.

Max pooling, by contrast, is a nonlinearity: it is a fold of maxima, and a maximum is a mux on a comparison, $\max(x, y) = y + (x - y) \cdot [x > y]$. It therefore falls on the expensive side of the dichotomy of Remark 5.1, and a window of size s costs $s^2 - 1$ comparisons per output. Average pooling is linear and free. This asymmetry is a design lever: our measurements exploit the fact that ReLU and max pooling commute, so pooling first reduces the number of ReLU applications by the pooling factor.

Proposition 5.4. For any s, max over a window commutes with a coordinatewise nondecreasing map. In particular $\mathrm{maxpool} \circ \mathrm{ReLU} = \mathrm{ReLU} \circ \mathrm{maxpool}$, and more generally $\mathrm{maxpool} \circ (\mathrm{ReLU} \circ \rho) = \mathrm{ReLU} \circ \rho \circ \mathrm{maxpool}$ for any nondecreasing rescaling ρ.

Proof. Let f be nondecreasing and S a finite set. Then $\max_{x \in S} f(x) = f(\max_{x \in S} x)$: the right side is attained by some x^* maximising x, and $f(x^*) \geq f(x)$ for every $x \in S$ by monotonicity, so it is also the maximum of the image; conversely it lies in the image. Both ReLU and a nonnegative rescaling are nondecreasing, as is their composition. □

Proposition 5.4 converts directly into a saving, because applying the nonlinearity after pooling evaluates it on s^2 times fewer values. Ordering the two operations is therefore not a matter of taste; it changes the comparison count of the whole network, and we report the measured factor.

5.4 The decision: why classification does not need a softmax

Trained classifiers are usually described as ending in a ***softmax***, $\mathrm{sm}(\mathbf{z})_j = e^{z_j} / \sum_l e^{z_l}$, which converts logits into a probability vector. Under encryption this would be a serious obstacle: it needs an exponential and a reciprocal, neither of which is a low-degree polynomial. It is also unnecessary.

Proposition 5.5. Softmax is strictly increasing in each coordinate relative to the others, in the sense that $z_j > z_l \iff \mathrm{sm}(\mathbf{z})_j > \mathrm{sm}(\mathbf{z})_l$. Consequently $\arg\max \circ \mathrm{sm} = \arg\max$, and a classifier that reveals only its predicted label may omit the softmax entirely. The same holds for any strictly increasing scalar function applied coordinatewise, and for multiplication by any positive constant.

Proof. The denominator of sm is a positive constant common to all coordinates, and $t \mapsto e^t$ is strictly increasing, so $\mathrm{sm}(\mathbf{z})_j > \mathrm{sm}(\mathbf{z})_l$ iff $e^{z_j} > e^{z_l}$ iff $z_j > z_l$. Hence the coordinatewise order, and in particular its maximiser, is preserved. For a general strictly increasing f applied coordinatewise the same argument applies verbatim; for a positive scalar multiple it is immediate. □

Proposition 5.5 is what makes a *classifier* tractable under encryption while a language model is not: the former reveals $\arg\max$ of the logits, which is invariant under the transformation one would otherwise have to compute, whereas the latter must sample from the distribution and therefore needs the normalisation itself. It also fixes what a private inference protocol should output, namely a sharing or ciphertext of the index, and what it should reveal, namely that index alone.

Definition 5.5. Given logits $\mathbf{z} \in \mathbb{R}^C$, the ***argmax fold*** computes $(m_0, i_0) = (z_0, 0)$ and

$$c_j = [\, z_j > m_{j-1} \,], \qquad m_j = m_{j-1} + c_j(z_j - m_{j-1}), \qquad i_j = i_{j-1} + c_j(j - i_{j-1}),$$

returning i_{C-1}. It uses $C-1$ comparisons and $2(C-1)$ multiplications, and its comparisons are sequentially dependent.

The sequential dependence in Definition 5.5 is worth noting because it is the one place in an inference where our round count is not determined by network depth: a C-way argmax contributes $C-1$ dependent comparisons. A tournament fold reduces this to $\lceil \log C \rceil$ levels at the same comparison count, which matters when C is large.

5.5 Quantisation: from real networks to the arithmetic a protocol offers

Networks are trained over the reals; protocols offer $\mathbb{Z}_{2^k}$, $\mathbb{F}_p$ or approximate reals. The bridge is quantisation, and because its error is one of the two error terms in every accuracy figure we report, we state it explicitly.

Definition 5.6. Fix a scale f. The ***quantised network*** $\widehat{g}$ of a network g replaces every parameter by its fixed-point encoding at scale f (Definition 3.7) and every affine map by integer arithmetic followed by a rescale, so that layer ℓ computes $\widehat{\mathbf{u}}_\ell = \sigma\big(\big\lfloor 2^{-f} \widehat{\mathbf{W}}_\ell \widehat{\mathbf{u}}_{\ell-1} \big\rceil + \widehat{\mathbf{b}}_\ell\big)$, with the bias stored at output scale.

Proposition 5.6. Let g have L layers of width at most n, with parameters and activations bounded by M and each σ 1-Lipschitz. Then the quantised network satisfies

$$\left\| \widehat{g}(x)\, 2^{-f} - g(x) \right\|_\infty \;\le\; (nM + \tfrac{1}{2})\, \frac{\gamma^L - 1}{\gamma - 1}\, 2^{-f} \qquad \text{where } \gamma = nM,$$

that is, the per-layer error of Proposition 3.3 amplified by the network's gain and accumulated over depth.

Proof. By Proposition 3.3 each layer introduces error at most $\varepsilon = (nM + \frac{1}{2})2^{-f}$ relative to exact evaluation on its own input. Write η_ℓ for the error at the output of layer ℓ. A 1-Lipschitz σ does not amplify, and the affine map amplifies by at most its infinity-norm operator norm, bounded by nM; hence $\eta_\ell \le \gamma \eta_{\ell-1} + \varepsilon$ with $\eta_0 = 0$. Unrolling the recurrence gives $\eta_L \le \varepsilon(1 + \gamma + \cdots + \gamma^{L-1})$, which is the stated geometric sum. □

Proposition 5.6 is pessimistic—it assumes worst-case sign alignment at every step, whereas real errors partly cancel—and we therefore do not use it to predict accuracy. We use it for its shape, which carries two practical consequences. Error grows geometrically in depth with ratio equal to the network's gain, so a network with unit gain accumulates error linearly in depth while one with gain above unity does not remain usable; and the scale f trades directly against every cost in the protocol, because the arithmetic domain must carry f plus the integer range plus the statistical margin of Remark 4.3. In practice one measures not bounds, which is why our pipeline reports the observed quantisation error of every model it deploys.

Remark 5.2. There is a stronger property than small error, and our harness insists on it wherever it is available: ***bit-exactness***. A private evaluation is bit-exact when its output equals that of the *same* quantised network evaluated in the clear, digit for digit. Bit-exactness says nothing about whether quantisation was a good idea; it says that the cryptography computed what the plaintext reference computed, and it therefore separates a protocol error from an approximation error. Where a paradigm cannot offer it—an approximate homomorphic scheme cannot—the weaker comparison is agreement of the predicted label, and we say which of the two a figure rests on.

5.6 What an accuracy figure means

Two of our case studies are medical, where accuracy alone is misleading, so we fix the metrics.

Definition 5.7. For a binary classifier with true positives TP, false positives FP, true negatives TN and false negatives FN, ***accuracy*** is $(TP + TN)/N$, ***sensitivity*** (recall) is $TP/(TP + FN)$, ***specificity*** is $TN/(TN + FP)$, and ***balanced accuracy*** is the mean of the last two. For C classes, balanced accuracy is the mean of the per-class recalls. The ***area under the receiver operating characteristic***, AUC, is the probability that a uniformly chosen positive receives a higher score than a uniformly chosen negative, ties counting one half.

Proposition 5.7. AUC equals the normalised Mann–Whitney statistic: if $s_1, \dots, s_m$ are the scores of the positives and $t_1, \dots, t_n$ those of the negatives, then

$$\mathrm{AUC} \;=\; \frac{1}{mn} \sum_{i=1}^{m} \sum_{j=1}^{n} \Big(\mathbb{1}[s_i > t_j] + \tfrac{1}{2}\, \mathbb{1}[s_i = t_j] \Big).$$

Proof. The double sum counts, over all mn pairs, the indicator of the event in Definition 5.7 with ties weighted one half; dividing by the number of pairs converts the count into the probability under the uniform choice of a pair. □

Proposition 5.7 is the form an implementation must use, and the tie term is the part that is easy to omit; a quantised network produces exact ties far more often than a real-valued one, because its scores are integers, so an AUC computed without the tie correction is biased for exactly the models we deploy.

Definition 5.8. For k successes in N independent trials with unknown rate p, the ***Wilson score interval*** at level $1-\alpha$ is

$$\frac{1}{1+z^2/N}\left(\widehat{p}+\frac{z^2}{2N} \;\pm\; z\sqrt{\frac{\widehat{p}(1-\widehat{p})}{N}+\frac{z^2}{4N^2}}\right), \qquad \widehat{p}=k/N,$$

where z is the standard normal quantile at $1-\alpha/2$.

We quote Wilson instead of normal-approximation intervals because our evaluation sets are small enough, and our rates close enough to one, that the normal interval can extend past unity; Wilson does not, and it retains nominal coverage at those extremes. An accuracy difference between two configurations is reported as meaningful only when the intervals are disjoint.

5.7 Threat models for private inference

Finally, the same computation admits several confidentiality goals, and the literature's figures are not comparable across them. We name them because our comparison depends on holding one fixed.

Definition 5.9. In ***input privacy*** the client's x is hidden from the server and the parameters θ are public or held in the clear by the server. In ***model privacy*** θ is hidden from the client. In ***joint privacy*** both are hidden from whoever evaluates: the evaluator learns neither x nor θ, and only the label is revealed, to a designated recipient.

Remark 5.3. The distinction is not a detail of framing; it changes the arithmetic. Under input privacy alone, an affine layer is a *plaintext*–ciphertext product, which for a lattice scheme costs no ciphertext multiplication and adds far less noise (Proposition 4.8(2)) and for a secret sharing is a purely local scaling. Under joint privacy it is a ciphertext–ciphertext or sharing–sharing product, which consumes multiplicative depth or a Beaver triple per term. Comparing a joint-privacy system against an input-privacy one on latency therefore compares two different computations, and the hybrid systems most often cited for their speed—in which the server holds the model in the clear—occupy the cheaper setting. Our measurements are of joint privacy throughout, and where we place them beside a published figure we state which setting that figure was obtained in.

6 Cryptographic preliminaries

In this section we give the definitions, theorems and algorithms that the rest of this survey relies on. We present each construction in the form our artifact implements it, and we remark where this form

differs from the textbook one, because several of our results depend precisely on those differences. In Section 6.1 we describe the security models and we define what hiding will mean. In Section 6.2 we present secret-sharing MPC and the three-party multiplication that needs no preprocessing. In Section 6.3 we describe garbled circuits and the three optimizations that determine their cost. In Section 6.4 we give the lattice background and the two encodings, BFV and BGV, that we implement. In Section 6.6 we show how a value can cross between a ciphertext and a sharing, and in Section 6.7 we briefly describe succinct proofs. For further information on the lattice material we point the reader to our previous report on the subject [54] and to the homomorphic encryption standard [2].

6.1 Security models

A protocol among n parties computes $f(x_1, \ldots, x_n)$ revealing nothing beyond the output. Security is defined by simulation: for every adversary in the real protocol there exists a simulator that, given only the corrupted parties' inputs and the output, produces an indistinguishable view. It means that whatever an adversary learns, it could have computed by itself from what it is entitled to know. Three axes determine the cost of achieving this.

Definition 6.1. An adversary is ***semi-honest*** when it follows the protocol and tries to learn from the messages it observes, and it is ***malicious*** when it deviates arbitrarily from the protocol. The ***corruption threshold*** t is the number of parties that the adversary controls; when $t < n/2$ we say that the protocol assumes an ***honest majority***, and when $t \leq n - 1$ we say that it tolerates a ***dishonest majority***. A protocol offers ***security with abort*** when the honest parties either output the correct value or detect the deviation and stop, and it offers ***guaranteed output delivery*** when they are required to finish.

The distance between these settings is not only theoretical. Semi-honest protocols require no consistency checks, and adding them is where most of the cost of active security is spent. An honest majority admits information-theoretic techniques and keeps public-key operations off the critical path, whereas a dishonest majority does not, which is why the SPDZ line [25, 24] needs cryptographic preprocessing. Our artifact is three-party with one corruption, which is the smallest honest-majority setting and also the setting of ABY3 [53] and Falcon [62]. Like them, we implement security with abort.

We also need a precise notion of what it means for a mask to hide a value, because one of our findings turns on it.

Definition 6.2. Let a be a secret in $\mathbb{Z}_{2^\ell}$ and let r be a mask drawn from a distribution over $\mathbb{Z}_{2^\ell}$. The masked value $c = a + r \bmod 2^\ell$ hides a ***perfectly*** when the distribution of c is independent of a, and it hides a ***statistically*** to within $2^{-\kappa}$ when, for every pair of secrets a, a', the distributions of $a + r$ and $a' + r$ are at total variation distance at most $2^{-\kappa}$.

Theorem 6.1. If r is uniform over the whole ring $\mathbb{Z}_{2^\ell}$, then $c = a + r$ is uniform and therefore hides a perfectly. If instead r is uniform over $[0, 2^m)$ with $m < \ell$ and $a \in [0, 2^m)$, then the posterior distribution of a given c is uniform over an interval of size $\min(c, 2^m) - \max(0, c - 2^m)$, which equals 1 when $c = 0$; for uniform a the mutual information is $I(a; c) \approx 1/(2 \ln 2) \approx 0.72$ bits.

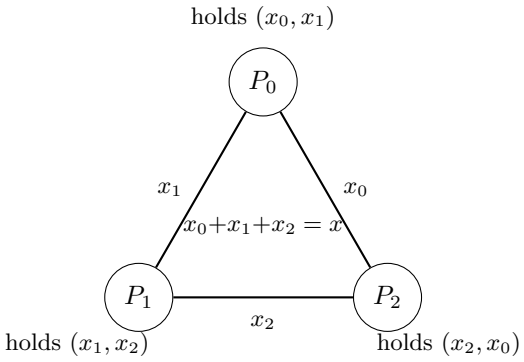


Figure 6.1: Replicated sharing over three parties. Each edge is the component shared by the two parties it connects, so any two parties together determine x and any single party learns nothing.

The second case of Theorem 6.1 can be interpreted as the statement that a mask chosen only to prevent an overflow does not hide anything in particular. We give it as a theorem and not as as a remark because the second condition is easy to satisfy inadvertently while violating the first, and because no correctness test can distinguish the two: a mask that prevents overflow yields correct arithmetic and insufficient hiding. Appendix A records an instance.

6.2 Secret-sharing MPC

In this section we describe the sharing structures and the multiplication protocol that our MPC layer implements. We work over the ring $\mathbb{Z}_{2^k}$ and not over a prime field, because machine arithmetic is already modular and because fixed-point truncation is more natural there.

Definition 6.3. An ***additive sharing*** of $x \in \mathbb{Z}_{2^k}$ among three parties is a triple (x_0, x_1, x_2) with $x_0 + x_1 + x_2 = x$, where party i holds x_i. A ***replicated sharing***, denominated $[x]$, gives party i the pair (x_i, x_{i+1}), with indices taken modulo 3, so that every additive component is held by exactly two parties.

Figure 6.1 shows the structure. The redundancy costs storage and it buys two properties that we use repeatedly. Linear operations remain local, since each party applies them to both of its components. And every value that a party sends can be recomputed by some other party, which is what makes most deviations detectable by comparison; we return to this in the discussion of malicious security below.

Addition of two shared values is local. Multiplication is not, because the product xy expands into nine cross terms $x_i y_j$ that are distributed among the parties. The classical solution consumes a Beaver triple [6], that is, a preprocessed $([a], [b], [ab])$ with a, b random, which converts one multiplication into two openings. Under replicated sharing with three parties this preprocessing is unnecessary, and we consider this the single most important cost fact about the honest-majority setting.

Theorem 6.2 (ABY3 multiplication [53])**.** Let $[x]$ and $[y]$ be replicated sharings over $\mathbb{Z}_{2^k}$, and let $\alpha_0, \alpha_1, \alpha_2$ be a zero-sum triple, $\sum_i \alpha_i = 0$. If party i computes

$$z_i \;=\; x_i y_i \;+\; x_i y_{i+1} \;+\; x_{i+1} y_i \;+\; \alpha_i,$$

then $\sum_i z_i = xy$, and (z_0, z_1, z_2) is an additive sharing of xy that reveals nothing beyond it.

Proof. Summing the three expressions collects each of the nine products $x_i y_j$ exactly once, because the index pairs (i, i), $(i, i+1)$ and $(i+1, i)$ enumerate all of $\{0,1,2\}^2$ as i ranges over the three parties, and the α_i cancel by construction. Moreover, each z_i is masked by α_i, which depends on a key that party i does not hold in full, hence no party learns a cross term that it should not learn. □

The zero-sum randomness is obtained without communication. Party i holds PRF keys k_i and k_{i-1} shared with its two neighbours and computes $\alpha_i = F_{k_i}(\mathsf{id}) - F_{k_{i-1}}(\mathsf{id})$, where id is a counter that all parties advance identically; the sum telescopes to zero. Algorithm 6.1 states the whole protocol. We remark that its cost is one ring element per party and one round, with no preprocessing, and that this is why an honest-majority protocol can be cheaper online than a dishonest-majority one however efficient the latter's preprocessing becomes.

Algorithm 6.1 Replicated-sharing multiplication

INPUT The shares (x_i, x_{i+1}) and (y_i, y_{i+1}) held by party i; the PRF keys k_i and k_{i-1}; the operation counter id. Indices are modulo 3.

OUTPUT The replicated sharing (z_i, z_{i+1}) of the product xy.

Compute $\alpha_i = F_{k_i}(\mathsf{id}) - F_{k_{i-1}}(\mathsf{id})$, which is local and zero-sum over the three parties.

Compute $z_i = x_i y_i + x_i y_{i+1} + x_{i+1} y_i + \alpha_i$, which is three of the nine cross terms.

Send z_i to party $i-1$ and receive z_{i+1} from party $i+1$, that is, one ring element in each direction and one round.

return (z_i, z_{i+1}).

Crossing between arithmetic and Boolean domains. Comparisons, ReLU and argmax are not arithmetic circuits and they require the bits of a shared value. We denote by `a2b` the bit-decomposition protocol, which takes $[x]$ to sharings of its bits, and by `b2a` its inverse. Bit decomposition is a carry-propagation problem, so its round complexity is decided by the adder: a ripple-carry adder needs depth $O(k)$, whereas a Kogge–Stone prefix adder [44] needs $O(\log k)$ at the price of more AND gates. Under replicated sharing the three additive components can be re-shared as Boolean values for free, since each one is already known to exactly the two parties that a replicated Boolean sharing requires, and therefore decomposition reduces to a three-input addition followed by a two-input one.

Malicious security by a multiplicative MAC. As we observed above, replicated sharing defends itself wherever a value is recomputable by another party. Auditing the semi-honest protocol leaves exactly one point that is not, namely the value z_i of Theorem 6.2, which depends on the pair (x_i, x_{i+1}) that no other party holds. Following Chida et al. [18] in the ring adaptation of Eerikson et al. [30], we carry every wire as a pair $([x], [r \cdot x])$ under a global MAC key r. Linear operations apply to both halves for free and a multiplication costs a second multiplication, so that a batched check at the end detects any deviation except with probability determined by the unused high bits of the ring. It means that the statistical security of the check is $128 -$ used bits, and consequently

an actively secure execution cannot also narrow the ring wire to the width of its values: the check and the encoding compete for the same slack.

Fixed point and truncation. Weights are real numbers, so a value x represents $x/2^s$ for a scale s. A product of two such values carries the scale 2^{2s} and must be truncated by s bits. Truncation is the awkward primitive of this area. Local truncation of each share is cheap, introduces a one-bit error, and fails outright with small probability. Exact truncation opens a masked value and corrects the borrow, and it is therefore governed by Definition 6.2: the mask must be wide enough for correctness and uniform enough for privacy, which are not the same requirement.

6.3 Garbled circuits

In this section we describe Yao's construction [67] and the three optimizations that determine its modern cost. We use all three, and we remark that the third one is the reason why circuit design in this paradigm reduces to counting AND gates.

Definition 6.4. A ***garbling*** of a Boolean circuit assigns to each wire w two labels W_w^0, W_w^1, drawn uniformly from $\{0,1\}^\lambda$, and to each gate a table of ciphertexts such that the labels of the input values decrypt exactly the label of the correct output value. The ***garbler*** produces the tables, the ***evaluator*** obtains one label per wire, and by construction it cannot decide which value that label encodes.

Definition 6.5 (Point-and-permute)**.** Each label carries a random ***pointer bit***. The pointer bits of the input labels index the row of the gate table that must be decrypted, and therefore the evaluator performs one decryption instead of trying all the rows.

Definition 6.6 (Free-XOR [45])**.** Let $\Delta \in \{0,1\}^\lambda$ be a global offset with least significant bit 1, and let $W_w^1 = W_w^0 \oplus \Delta$ for every wire w. Then the output label of an XOR gate is the XOR of its input labels.

It means that XOR gates cost no ciphertexts and no hashing, and that negation is a relabelling: to compute NOT it is enough to flip which of the two labels denotes zero. Only AND gates consume bandwidth. We remark that the price of this optimization is a correlation among the labels, since all of them differ by the same Δ, and consequently the hash used inside a gate must remain secure when its inputs are correlated; a plain pseudorandom function is not sufficient.

Theorem 6.3 (Half-gates [68])**.** Under free-XOR, an AND gate can be garbled with two ciphertexts, by writing it as the XOR of a ***generator half-gate***, whose value depends on a bit that the garbler knows, and an ***evaluator half-gate***, whose value depends on a bit that the evaluator knows. Garbling one AND gate costs four hash evaluations and evaluating it costs two.

Figure 6.2 shows the resulting cost structure of a circuit. Together with Definition 6.6, Theorem 6.3 gives the accounting we use throughout: a circuit's communication is 2λ bits per AND gate and nothing else.

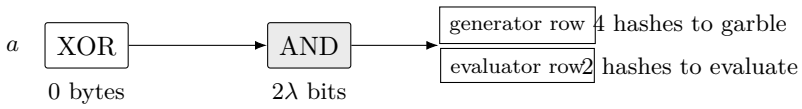


Figure 6.2: Cost of a garbled circuit under free-XOR and half-gates. XOR and NOT are free, so only the AND gates appear in the transcript.

Three parties without oblivious transfer. In the two-party setting the evaluator must obtain the labels for its own input without revealing that input, which requires oblivious transfer. With three parties and an honest majority the oblivious transfer can be removed: two parties garble the same circuit from shared randomness and the third evaluates, so that input labels are delivered by the agreement of the two garblers, and consistency between the two garbled copies replaces the guarantee that oblivious transfer would provide. The protocol that results needs one round for the entire circuit. We remark that this is the structural reason why garbled circuits win under high latency, since the cost model is rounds $\times$ RTT + bytes/bandwidth and this paradigm minimizes the first term while secret sharing minimizes the second.

6.4 Lattices and homomorphic encryption

In this section we recall the lattice definitions that we need, we state the ring learning with errors problem, and we describe the two encodings, BFV and BGV, that our artifact implements. We keep the lattice material to what the cost discussion requires and we point the reader to our previous report [54] for the mathematical apparatus.

Definition 6.7. Let $B = [b_1, \dots, b_n]$ be a basis of linearly independent vectors in $\mathbb{R}^n$. The ***lattice*** generated by B is the set $\mathcal{L}(B) = \{Bx \mid x \in \mathbb{Z}^n\}$, and its ***fundamental domain*** is the parallelepiped $\mathcal{P}(B) = \{Bx \mid x \in [0,1)^n\}$. The determinant $\det(B)$ is independent of the choice of basis and corresponds geometrically to the inverse of the density of lattice points.

Figures 6.3 and 6.4 illustrate the two facts about lattices that the encryption schemes use. In Figure 6.3 a point $w \in \mathbb{R}^n$ decomposes uniquely as a lattice vector v plus an element t of the fundamental domain, and this decomposition can be interpreted as a modular reduction. In Figure 6.4 a ciphertext is a lattice point displaced by a small error e, so that decryption is the recovery of the nearest lattice point; it succeeds precisely while the error remains inside the shaded region, in which that point is unambiguous, and it fails for the ciphertext on the right, whose error has left it.

Definition 6.8 (Ring learning with errors)**.** Let $R = \mathbb{Z}[X]/(X^n + 1)$ for a power of two n, and let $R_q = R/qR$. Given a secret $s \in R_q$, the RLWE distribution outputs pairs $(a, as + e)$ where a is uniform in R_q and e is drawn from a narrow error distribution. The ***decision*** problem is to distinguish such pairs from uniform ones, and the ***search*** problem is to recover s.

The hardness of Definition 6.8 reduces to worst-case lattice problems, and concrete parameters are obtained from lattice-reduction estimates [2, 3]. We remark that the security level depends jointly on n, on q and on the error distribution, and in particular that raising q at a fixed n *weakens* the scheme. It means that depth and security are coupled: a construction that needs less depth may use a smaller modulus and is therefore secure at a smaller ring dimension.

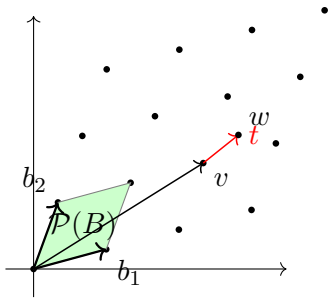


Figure 6.3: Reduction modulo $\mathcal{P}(B)$

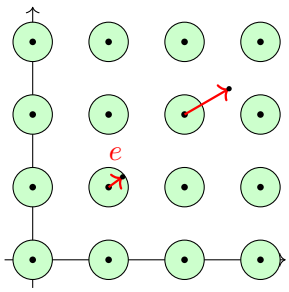


Figure 6.4: Noise budget of a ciphertext

Definition 6.9 (BFV and BGV encodings)**.** Let t be the plaintext modulus and $\Delta = \lfloor q/t \rfloor$. A ciphertext c encrypts m under s with

$$\text{BFV [32]:} \quad c(s) = \Delta \cdot m + e, \qquad \text{BGV [10]:} \quad c(s) = m + t \cdot e.$$

Everything else follows from that single line. BFV places the message in the high-order bits and decrypts by scaling and rounding, so its tensor product carries a factor Δ^2 that must be scaled away; BGV places the message in the low-order bits and decrypts by reducing modulo t, so its noise grows multiplicatively and is controlled by *modulus switching*, that is, by dividing the ciphertext modulus in order to divide the noise. We implement both, and we did so in order to answer a specific question by substitution not by argument: how much of the cost we measure belongs to RLWE itself and how much belongs to a particular encoding.

Noise, depth and bootstrapping. Every homomorphic operation enlarges the error of Figure 6.4, mildly for addition and sharply for multiplication. When the error leaves the region where the nearest lattice point is unambiguous, decryption fails, and therefore a parameter set supports a bounded multiplicative *depth*. Gentry's bootstrapping [35] evaluates the decryption circuit homomorphically and resets the noise, which makes unbounded depth possible and is the dominant cost of full homomorphism. It means that the practical question for a pipeline is not how fast a bootstrap is, but whether it occurs at all; in Section 10.4 we describe a pipeline whose depth is one by construction and which therefore performs none.

Packing. The ring R_q has n coefficients, so a single ciphertext carries many values. In *slot* packing the plaintext space is decomposed by the Chinese remainder theorem and operations act component-wise, and moving data between slots requires rotations. In *coefficient* packing we exploit the fact that polynomial multiplication is a convolution: if a vector is laid out forwards and a kernel is laid out *reversed*, then one ciphertext product places an inner product in a coefficient of the result, and with suitable strides it places an entire matrix–vector product or convolution there. This is the Gazelle [40] and Cheetah [37] line, and it is the reason why the cost of our convolutions does not depend on the image size.

Relinearization, and how to avoid it. A product of two ciphertexts decrypts under s^2 instead of s, and implementations normally *relinearize* back to s using an evaluation key. We remark that if the pipeline never multiplies again, then relinearization is pure overhead, since it is enough to decrypt under the squared key. Our construction does exactly that, and consequently evaluation

keys disappear from the trusted setup and from the network. This simplification is available only because the depth is one.

Threshold decryption. When the secret key must not exist in a single place, it is shared and decryption becomes a protocol. We implement threshold decryption, and the key it decrypts under is generated distributively: no party holds it and no dealer produces it (Section 15). Both fresh and once-multiplied ciphertexts are covered, the latter without an evaluation key, for a reason the depth-1 design supplies and which we return to below.

6.5 A composability theorem, and what it costs

In this section we state what we can prove about composition, and we describe the two protocol changes that were required in order to be able to state it. We remark that the exercise was not bookkeeping: attempting the proof located a leak in Π_{PC} that had survived every correctness test, and we describe it in Section 15.

Definition 6.10. The ***arithmetic black box*** $\mathcal{F}_{\mathsf{ABB}}$ over $\mathbb{Z}_{2^k}$ is the ideal functionality that keeps a store of values indexed by handle and accepts the commands input, add, mul, lt, trunc_d and reveal. Only reveal produces output, and the functionality leaks to the adversary the sequence of commands together with the handles they name, and nothing else.

The leakage clause of Definition 6.10 is the one that carries weight, and it is why the control flow of a program matters. Our gadget library never branches on a secret, so its command sequence is a function of the public layer shapes alone; the interpreter, in contrast, implements if on a secret predicate by revealing that predicate, and therefore the statement below is about the library and not about the language.

Functionality $\mathcal{F}^k_{\mathsf{ABB}}$

The functionality keeps a partial map St from handles to $\mathbb{Z}_{2^k}$, initially empty, and interacts with parties P_0, P_1, P_2 and the adversary $\mathcal{S}$. Every command is ignored unless its input handles are defined and its output handle is not. On each command the functionality leaks the command name together with its handles to $\mathcal{S}$, and nothing else.

- $(\mathsf{input}, id, P_j)$: on this command from all parties and $x \in \mathbb{Z}_{2^k}$ from P_j, set $\mathsf{St}[id] \leftarrow x$.
- $(\mathsf{add}, id_1, id_2, id_3)$: set $\mathsf{St}[id_3] \leftarrow \mathsf{St}[id_1] + \mathsf{St}[id_2]$.
- $(\mathsf{mul}, id_1, id_2, id_3)$: set $\mathsf{St}[id_3] \leftarrow \mathsf{St}[id_1] \cdot \mathsf{St}[id_2]$.
- $(\mathsf{lt}, id_1, id_2, id_3)$: set $\mathsf{St}[id_3] \leftarrow 1$ if $\mathsf{St}[id_1] < \mathsf{St}[id_2]$ as signed k-bit integers, and 0 otherwise.
- $(\mathsf{trunc}_d, id_1, id_2)$: if $0 \le \mathsf{St}[id_1] < 2^{k-1}$ set $\mathsf{St}[id_2] \leftarrow \lfloor \mathsf{St}[id_1]/2^d \rfloor$; otherwise set $\mathsf{St}[id_2] \leftarrow \perp$.
- (reveal, id, R): send $\mathsf{St}[id]$ to every party in R.
- (abort) from $\mathcal{S}$: send $\perp$ to every honest party and halt. *Present only in the malicious variant* $\mathcal{F}^{k,\mathrm{abort}}_{\mathsf{ABB}}$.

Figure 6.5: The arithmetic black box our MPC layer realizes

Figure 6.5 makes two design decisions explicit, and both of them are load-bearing.

The first is the ***leakage clause***. The functionality tells the adversary which commands were issued and on which handles, and therefore a protocol realizing it is allowed to have a public control flow and nothing more. It means that a program whose sequence of operations depends on a secret

cannot be simulated, which is why Corolary 6.2 is a statement about our gadget library, whose command sequence is a function of the layer shapes, and not about our language, whose if on a secret predicate reveals that predicate.

The second is the guard on trunc_d. We could have written truncation as a total function on $\mathbb{Z}_{2^k}$, and then no protocol of ours would realize it, because our truncation is correct only for a non-negative argument below half the ring. Writing the precondition into the functionality instead is the honest move: the guard is exactly the bound that Section 6.2 obtains by gating before shifting, and a caller that violates it receives $\perp$ rather than a wrong answer. We remark that this is the general shape of the "no free lunch" in a UC statement — either the protocol is strengthened to meet the functionality, or the functionality is weakened to state what the protocol does, and the second is only legitimate when the weakening is written down.

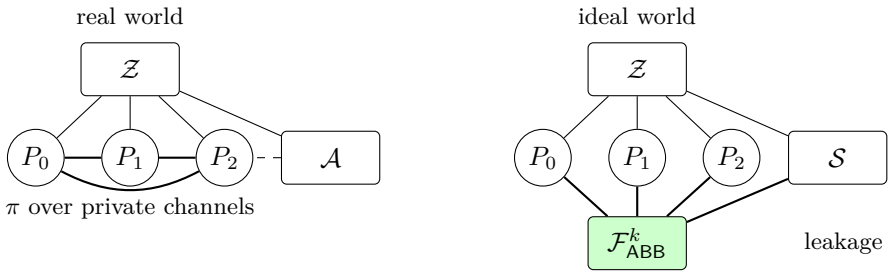


Figure 6.6: UC realization: no $\mathcal{Z}$ distinguishes the two

Figure 6.6 recalls what has to be established. The environment $\mathcal{Z}$ chooses the inputs, sees the outputs, and talks to the adversary throughout; the protocol realizes the functionality when for every $\mathcal{A}$ there is a simulator $\mathcal{S}$ such that no $\mathcal{Z}$ distinguishes its interaction with the real protocol from its interaction with $\mathcal{F}^k_{\mathsf{ABB}}$. We remark that $\mathcal{Z}$ is what makes the statement composable and also what makes it demanding: since $\mathcal{Z}$ may run other sessions and feed their results back, a simulator is not allowed to rewind, and any randomness that a protocol reuses across sessions is randomness that $\mathcal{Z}$ can arrange to collide.

Theorem 6.4 (Simulator)**.** Let $\mathcal{A}$ be a static semi-honest adversary corrupting P_c. There is a straight-line simulator $\mathcal{S}$ that, given the leakage of Figure 6.5 and the corrupted party's inputs and outputs, produces a view computationally indistinguishable from the real one.

Proof. We construct $\mathcal{S}$ command by command; in every case it holds the corrupted party's two replicated components and must produce the messages that party receives.

For input and for the linear commands there are no messages, since a replicated sharing is created locally from the dealer's value and addition acts on components.

For mul, the real protocol delivers to P_c the single element $z_{c+1} = x_{c+1}y_{c+1}+x_{c+1}y_{c+2}+x_{c+2}y_{c+1}+\alpha_{c+1}$. The term α_{c+1} is $F_{k_{c+1}}(\cdot) - F_{k_c}(\cdot)$, and P_c holds k_c but not k_{c+1}, so by pseudorandomness of F the element is computationally indistinguishable from uniform. $\mathcal{S}$ samples it uniformly. This is the step that needs the keys to be fresh per session: were the same (k, ctr) to recur in a second session that $\mathcal{Z}$ schedules, the two elements would differ by a value $\mathcal{Z}$ knows, and no uniform sample reproduces that.

For lt and trunc_d, the messages are those of Π_{WA}, Π_{PC}, Π_{SS} and Π_{Trunc}, and each of their openings is covered by Lemma 6.1: $\mathcal{S}$ samples the opened value from the distribution the lemma names — uniform on the ring, a uniform bit, or uniform on $\mathbb{Z}_p^\star$ conditioned on the predicate the functionality has already given it — and derives the remaining messages from the corrupted party's components exactly as the protocol does.

For reveal, the functionality supplies the value and $\mathcal{S}$ sends the missing component consistent with it.

Indistinguishability follows by a standard hybrid over the PRF calls, of which there are polynomially many, replacing each in turn by a uniform value. No step rewinds $\mathcal{Z}$ or $\mathcal{A}$. □

Corolary 6.1. With the MAC layer of Chida et al. [18] in the ring adaptation of Eerikson et al. [30], the same protocol realizes $\mathcal{F}_{\mathsf{ABB}}^{k,\mathrm{abort}}$ against a static malicious adversary, provided every consistency check is deferred to one point whose outcome depends only on whether the adversary deviated.

The proviso in Corolary 6.1 is not a formality. If a check can fail as a function of a secret, then the abort itself is an output channel that $\mathcal{F}_{\mathsf{ABB}}^{k,\mathrm{abort}}$ does not have, and no simulator reproduces it: the simulator learns from the functionality whether the adversary cheated, and never learns the secret that a data-dependent check would consult. Our own first attempt at this layer had exactly that defect, and we describe it in Section A; Brüggemann and Schneider [11] exploit the same shape against Trident [15], Fantastic Four [23] and Quad, and identify a gap of this kind in the proof of SWIFT [46], which is evidence that the condition is easy to violate while believing one has batched a check for efficiency alone.

On the preprocessing. The masks of Lemma 6.1 are produced by the pseudorandom secret-sharing of Cramer, Damgård and Ishai [22] and by doubly-authenticated bits in the sense of Rotaru and Wood [57], improved by the edaBits of Escudero et al. [31]. We remark that the functionality they realize is a preprocessing functionality and not $\mathcal{F}_{\mathsf{ABB}}$, so the composition is two-layered: the preprocessing realizes $\mathcal{F}_{\mathrm{Prep}}$, and the online phase realizes $\mathcal{F}_{\mathsf{ABB}}$ in the $\mathcal{F}_{\mathrm{Prep}}$-hybrid model. It means that our pool of Section 13.4 is not merely an optimization but the boundary between the two layers of the proof, and that the miss counter which refuses to let an online phase generate its own preprocessing is enforcing that boundary and not merely measuring it.

Lemma 6.1 (Opening audit)**.** In the protocols of this artifact, every value opened to the parties is either the output of a reveal command or is masked by randomness that is uniform on the domain of the opening and used only once. Consequently each opening is simulatable from the corresponding command's output.

Proof. By enumeration; the sites are finitely many and Table 1 lists them. The three interesting cases are as follows. In Π_{WA} the opened value is $a + x$ for a preprocessed x that is uniform on the whole ring, so the opening is uniform. In Π_{Trunc} the same holds, which is the reason the mask must cover the ring and the overflow be corrected rather than avoided. In Π_{PC} the opened value is $m \cdot \prod_i c_i$; the c_i carry no randomness of their own, hence the mask m must be a uniform unit *and must remain secret*, which is what forces the zero test on m to open $m \cdot s$ not m. □

Theorem 6.5. Assume private authenticated point-to-point channels, a pseudorandom function, and a setup that delivers pairwise PRF keys freshly per session. Then the protocol of Section 6.2 UC-realizes $\mathcal{F}_{\mathsf{ABB}}$ over $\mathbb{Z}_{2^k}$ against a static adversary corrupting at most one of three parties, semi-honestly; and with the MAC layer of Chida et al. [18] in the ring adaptation of Eerikson et al. [30], against a static malicious adversary, with abort.

We do not claim a machine-checked proof, and we set out the argument and its hypotheses so that the gaps are visible. Simulation is straight-line, since the simulator for a semi-honest

Protocol	Opened value	Mask	Simulatable
Π_{PC}	$m \cdot \prod_i c_i$	m uniform unit, secret	yes†
Π_{WA}	components of $a + x$	x uniform on $\mathbb{Z}_{2^\ell}$	yes
Π_{SS}	$b \oplus c$	c a fresh uniform bit	yes
Π_{Trunc}	$a + r$	r uniform on $\mathbb{Z}_{2^\ell}$	yes†
mask generation	$\delta \oplus r$	r a fresh uniform bit	yes
unit sampling	$m \cdot s$	s uniform on $\mathbb{Z}_p$	yes†
`reveal`	the command's output	—	by definition
FHE $\to$ MPC	$m + \sum_i r_i$	r_i uniform on $\mathbb{Z}_{2^k}$	yes
`a2b`	nothing	—	by definition

Table 1: Every opening, and why it can be simulated. † three of the nine required a protocol change before the entry could be written; see Section 15.

minority samples the corrupt party's view directly: linear operations are local, the multiplication of Theorem 6.2 sends one element masked by an α_i the corrupt party cannot compute, and every opening is handled by Lemma 6.1. No rewinding appears, which is what distinguishes a UC argument from a standalone one. Keys are drawn afresh per session, so a one-time mask indexed by an operation counter is never reused across the concurrent sessions that the UC environment may schedule; this is a property of our transport and we state it as a hypothesis because it is not enforced by a type. For the malicious case the abort must be simulatable, and it is: the checks are deferred to a single point whose outcome depends on whether the adversary deviated and not on any secret. We remark that this is exactly the property whose absence Brüggemann and Schneider [11] exploit against Trident, Fantastic Four and Quad, and whose specification gap they identify in SWIFT; deferring a check hides *when* an abort happens and not *whether* it happens, and only the latter is what security requires.

Corolary 6.2. Any network assembled from the gadget library, whose control flow is determined by public layer shapes, UC-realizes the functionality that reveals only its declared output.

Figure 6.7 shows the shape of the argument, and we draw it because the layering is what makes each step small enough to check. Reading upwards, the preprocessing realizes $\mathcal{F}_{\mathrm{Prep}}$ from private channels and the key setup; the online protocols realize $\mathcal{F}_{\mathsf{ABB}}$ in the $\mathcal{F}_{\mathrm{Prep}}$-hybrid model, which is the step Lemma 6.1 and Theorem 6.4 establish; and the gadget library realizes $\mathcal{F}_{\mathrm{NN}}$ in the $\mathcal{F}_{\mathsf{ABB}}$-hybrid model, which is Corolary 6.2 and needs only that the control flow be public. It means that the boundary between the second and third layers is the pool of Section 13.4: a warm server has run the bottom layer in advance, and the miss counter that refuses to let an online phase generate its own preprocessing is enforcing the layer boundary and not merely reporting a latency.

We remark that $\mathcal{F}_{\mathrm{NN}}$ is the weakest of the three to specify and the easiest to get wrong, because it is where the application decides what may be public. Ours reveals the predicted class and the network's architecture, and nothing else: not the logits, not the activations, and not the input. Writing it down is what makes "the pipeline reveals only the class index" a checkable statement not a description of the code.

What the theorem does not cover, and what it would cost. The hybrid pipeline is outside $\mathcal{F}_{\mathsf{ABB}}$, because $\mathcal{F}_{\mathsf{ABB}}$ has no ciphertexts. Extending the statement to it needs three things, and we have two. The conversion of Section 6.6 opens $m + \sum_i r_i$ with each r_i uniform on the plaintext space, so it satisfies Lemma 6.1. Against a malicious adversary an input ciphertext must also be

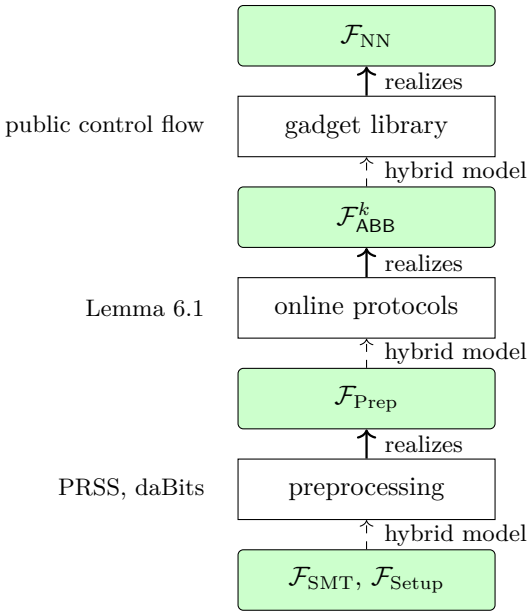


Figure 6.7: The layers the composition theorem is applied through

shown well-formed with bounded noise, because replication cannot detect a malformed one — both honest co-holders compute on it faithfully and agree — and that proof is implemented.

Key generation was the last part of this argument to rest on a setup assumption, and it no longer does: the key is sampled distributively, so the hybrid realizes its functionality without a dealer rather than in a model with one. The construction follows the line Damgård and Nielsen [24] established for an arithmetic black box from threshold homomorphic encryption and which Asharov et al. [5] carry to threshold FHE, with one adaptation that the replicated setting forces and one dividend that it pays; both are described in Section 15. The adaptation is that components are ternary, not uniform, because independent sampling cannot deliver a small key and uniform shares at once, and the resulting leak is one the standard assumption already absorbs. The dividend is that a product ciphertext needs no evaluation key at all, since replicated shares yield a sharing of s^2 with no communication.

What remains is not a setup assumption but a parameter one, and we state it in Section 15: the boundary proof's soundness slack is subtracted from the smudging budget bit for bit, which leaves a smaller λ than an honest-input analysis reports.

6.6 Crossing between the paradigms

A ciphertext and a secret sharing are two representations of the same hidden value, and the possibility of converting between them is what makes a hybrid protocol possible. In this section we describe the two conversions, which are illustrated in Figure 6.8. We remark that both are linear, and consequently neither consumes multiplicative depth, which is the property that Section 10.4 depends on.

To pass from a ciphertext to a sharing, we add a fresh random mask homomorphically, we decrypt the masked ciphertext, and we let the mask itself be secret-shared. The plaintext that appears is uniform by Theorem 6.1, and subtracting the shared mask leaves a sharing of the original value. To pass from a sharing to a ciphertext, each party encrypts its own share and the ciphertexts are added homomorphically, which is correct because the sharing is additive.

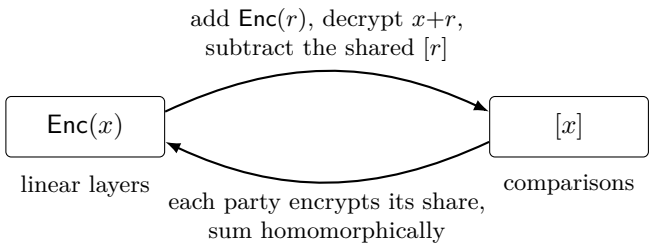


Figure 6.8: Scheme switching. Both directions are linear, so a hybrid pipeline can alternate between them without consuming multiplicative depth.

Paradigm	Secret from	Expensive in	Cheap in	Rounds
Secret-sharing MPC	computing parties	**rounds**, bandwidth	local arithmetic	O(depth)
Garbled circuits	computing parties	**bandwidth**	rounds	$O(1)$
FHE	computing server	**compute**, ct. size	rounds (none)	none
ZK proofs	the verifier	**prover compute**	verifier, proof size	none

Table 2: The paradigms organized by cost structure, not by security property. Three trade-offs follow directly, and drive most of this paper.

6.7 Succinct proofs

A zero-knowledge argument allows a prover to convince a verifier that it knows a witness satisfying a public relation, revealing nothing else, and it is *succinct* when the proof size and the verification time are sublinear in the computation being proved. We remark that these systems answer a different question from the one that MPC and homomorphic encryption answer: they establish the *integrity* of a computation performed by somebody else, and not the privacy of a computation performed jointly. They enter this survey because a deployment of private inference may need both properties at once. Our artifact targets Binius64 [27], which operates over towers of binary fields so that operations on 64-bit words are expressed without bit-blasting them.

7 Four paradigms, four cost structures

In this section we organize the four paradigms. They are usually taxonomized by the security property that they provide, but for a practitioner who has to choose among them we consider that the more useful axis is *what each one is expensive in*, and this is the axis of Table 2. Three trade-offs follow from it directly, and they drive most of the remainder of this survey.

Secret sharing and garbled circuits trade rounds against bytes. This is the classical trade between the constructions of Yao [67] and of GMW, and neither of them dominates the other. The winner depends on the round-trip time and on the bandwidth, and therefore the location of the boundary is an empirical question. We measure it in Section 13.4, where we also report the three answers that we published before the current one.

Homomorphic encryption and secret sharing trade compute against communication. Homomorphic encryption sends much data once and then computes hard, whereas secret sharing sends little data many times. On a fast local network secret sharing usually wins, and on a thin pipe

the balance moves. We remark that this framing omits *preprocessing*, and in Section 13.1 we show that the omission is precisely what makes most published crossovers between the two paradigms fragile.

Succinct proofs are orthogonal, and conflating them with the others is a category error. Secret sharing and homomorphic encryption keep data secret *from the parties that compute on it.* A succinct proof establishes that a computation was performed correctly while hiding the witness *from a verifier*, and the prover holds everything in the clear. It means that a deployment which needs both confidentiality and integrity pays for both, and that a proving time and a protocol latency do not belong in the same column. For this reason we keep them in separate tables, and our benchmark prints the caveat next to its numbers so that they cannot travel without it.

7.1 The hybrid, and why it exists

Inference in a neural network alternates linear layers with nonlinearities. Homomorphic encryption is good at the former and bad at the latter, since a comparison is not a low-degree polynomial; secret sharing is the reverse, since a comparison costs a bit decomposition whereas a linear layer costs one round independently of its width.

Consequently the natural design is to evaluate the linear layers under homomorphic encryption and the nonlinearities under secret sharing, with a *scheme switch* between them in the sense of Section 6.6. This is the design that Gazelle [40], Delphi [52] and Cheetah [37] instantiate. We implement it and we add two elements, which we describe in Section 10.4: the weights are encrypted as well, and the switch is used as a *noise refresh*, which removes bootstrapping from deep networks entirely.

8 The Baru artifact

8.1 Baru is a CEK machine

In this section we describe the abstract machine at the centre of our artifact, and we argue that this choice is what makes the comparisons of this survey possible. Our artifact is not a circuit compiler with a front end. It is an abstract machine for a Lisp-like language, and the cryptography is placed behind a single interface that the machine calls.

Definition 8.1. A ***CEK machine*** [33] evaluates a program by rewriting a triple $\langle C, E, K \rangle$, denominated a ***configuration***, where C is the expression under evaluation, E is an environment that maps variables to values, and K is a ***continuation***, that is, a representation as data of what remains to be done once the value of C is known.

Figure 8.9 shows one step of the machine. Evaluation is a loop over a step function: in order to evaluate an application $(f\ a)$, the machine pushes onto K the instruction “evaluate the argument and then apply f” and moves to f; when a value is finally produced, the machine pops the topmost frame of K and proceeds. It means that no recursion takes place on the stack of the host language, and consequently a deep program consumes heap not stack, and the entire state of the machine is an ordinary value.

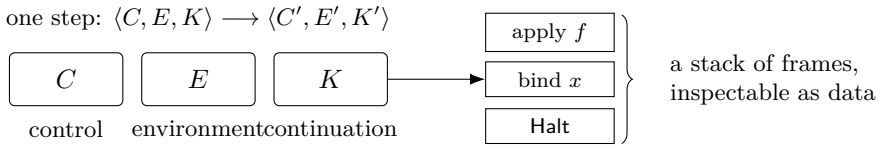


Figure 8.9: One step of the abstract machine. Because the continuation is data and not the host stack, it can be inspected, serialized, and used to batch independent protocol calls.

Why an abstract machine and not a circuit. The dominant design for secure computation compiles a program into a circuit and then evaluates that circuit under one protocol, as in ABY [26], HyCC [12], CirC [55] and Silph [16]. We consider that this is the correct design when the protocol is fixed, and the incorrect one for a survey, because a circuit has already made the choices of the protocol. A comparison compiled for garbling is a Boolean circuit, and the same comparison compiled for arithmetic sharing is not the same object. It means that comparing two paradigms would reduce to comparing two compilations, and any difference between the two compilations would contaminate the measurement.

An interpreter, in contrast, keeps the program fixed and changes what a primitive *means*. Our evaluator manipulates secret values only through opaque handles and a fixed set of operations, which is the Arithmetic Black Box of the SPDZ line [24, 25]. Therefore the expression `(> z 0)` is one call to `lt`, and whether that call becomes a bit decomposition, a garbled comparator or a `wrap3` sign extraction is a decision taken by a backend at run time. In other words, the same source text, the same syntax tree and the same sequence of calls to $\mathcal{F}_{\mathsf{ABB}}$ reach every paradigm, and consequently a difference that we measure between two paradigms is attributable to the protocol and not to the program.

What the continuation buys. We use three consequences of Definition 8.1 in this work.

- **Non-local control is data.** Condition handlers are frames on K, so that a `guard` installs one and leaving its body pops it. Therefore error handling inside a private computation requires no exceptions of the host language, and it cannot accidentally unwind past a point at which a protocol must synchronize.
- **The abstraction admits batching, although the machine does not perform it.** A backend primitive receives a *vector* of operands, so an entire layer of `lt` calls can be one protocol round not thousands. We are precise about where that batching comes from, because it is not from the continuation: our evaluator issues primitives one at a time, and the parallelism is supplied either by a gadget that calls the batched primitive explicitly, or by the compiler of Section 13, which recovers it automatically by levelling a dependence graph. What the abstract machine contributes is the seam that lets both routes exist behind one program. We remark that this distinction matters more than it appears to, since almost every cost result in this survey depends on the difference between a cost paid per element and a cost paid per call.
- **The state is serializable.** A configuration is an ordinary value, so an execution can be checkpointed, replayed, or, as we use it here, run three times in lockstep by three parties whose operation identifiers are required to agree. We derive protocol randomness from a counter that every party advances identically, and this is sound only because the three machines take the same steps in the same order.

The cost of this choice. Interpretation is slower than compilation, and we make no claim about absolute performance (Section 15). An interpreter also cannot perform the whole-circuit optimizations that a compiler would find, and therefore when a compiler-based system reports fewer AND gates for the same function, that difference is real and it is not a difference between protocols. What our design buys is not speed but *commensurability*, and in Section A we record cases in which commensurability is what caught an error that the numbers alone did not reveal.

8.2 One program, many backends

In this section we describe the interface through which the machine of Section 8.1 reaches the cryptography. As we said above, our language manipulates secret values only through opaque handles and a fixed set of primitive operations, which is the Arithmetic Black Box abstraction of the SPDZ line [24, 25]: an ideal functionality $\mathcal{F}_{\mathsf{ABB}}$ that stores values by handle and exposes `add`, `mul`, `lt`, `share_public` and `reveal`. A backend implements those primitives, and the evaluator never inspects a representation.

```
; private ReLU on one neuron -- every value is secret-shared
(let ((w :sint8 -1) (x :sint8 3) (b :sint8 0))
  (let ((z (+ (* w x) b)))
    (reconstruct (* z (> z 0)))))          ; => 0

; a value computed under MPC crosses into FHE without being revealed
(let ((keys (fhe-keygen)) (x (share 6 32)) (y (share -7 32)))
  (fhe-dec keys (share->fhe keys (* x y))))   ; => -42
```

The consequence for a survey is immediate. *The same source text* is the workload for every paradigm, and therefore a comparison between paradigms cannot be confounded by two implementations that compute subtly different functions. We remark that this is not a hypothetical concern, since in Section A we record a case in which it caught exactly that.

8.3 Composability

Every gadget of our library, namely `relu`, `max2`, `maxpool2d`, `argmax`, `dense` and `conv2d`, is a pure composition of $\mathcal{F}_{\mathsf{ABB}}$ primitives: no gadget opens a value and none branches on a secret, so the sequence of primitive calls a gadget makes is a function of the public layer shapes. Consequently the composition theorem of the universal composability framework [13] carries security from the primitives to any network assembled from the gadgets, and a search over architectures requires no new security argument for each architecture. It means that the research question becomes which architecture is *fastest* under the cost model, rather than whether a given one is secure.

We remark that the hypothesis of that sentence is stronger than it looks, and we state it properly in Section 6.5. The primitives must *UC-realize* $\mathcal{F}_{\mathsf{ABB}}$, which is not implied by their being secure in a standalone sense, and every value the primitive protocols open internally must be simulatable. Two of our own security failures were failures of precisely that hypothesis rather than of composition, so we treat the gadget-level claim as conditional and prove the condition separately.

8.4 What is implemented

Table 3 summarizes what the artifact contains. Unless we note otherwise, all the figures that follow were obtained on the same machine, an Apple M1 Pro with 8 cores, in a release build.

Layer	Contents
MPC	ABY3 [53] replicated shares over $\mathbb{Z}_{2^{128}}$; **triple-free** multiplication from pairwise PRFs; bit decomposition (carry-save $3 \to 2$ plus Kogge–Stone [44]); batched primitives; **malicious security with abort** (≤ 1 of 3 corrupt), following Chida et al. [18] in the ring adaptation of Eerikson et al. [30]; in-process and TCP transports
Garbled	Half-gates [68], free-XOR [45], fixed-key AES; comparator and fused ReLU $\circ$ rescale circuits; OT-free label delivery in the three-party honest-majority setting
Falcon	Π_{PC} private compare over $\mathbb{Z}_p$, Π_{WA} `wrap3`, Π_{DReLU}, Π_{SS}, Π_{Maxpool}, exact truncation—a reproduction of Falcon [62] inside the same harness
FHE	BFV [32] and BGV [10] (depth-one, no evaluation keys), TFHE-rs [60, 19], an exact real-arithmetic backend; multi-modular NTT with Shoup butterflies; CRT slot packing; Cheetah-style [37] convolution packing
Hybrid	MPC$\leftrightarrow$FHE switching both directions; re-encryption as a bootstrapping alternative; replicated threshold-key switch [5]
ZKP	Expression-to-circuit compiler; real Binius64 [27] prover and verifier
Compiler	Backend-aware circuit IR, rewrite rules, cost models, protocol registry (dealer / Shamir / Semi2k [41] / LowGear [42]), per-operation protocol assignment
Primitives	Chou–Orlandi [20] and IKNP [38] OT, DH-PSI

Table 3: What the artifact implements. Every row is exercised by at least one benchmark, and every engine attests its security level (Section 12.4).

9 Compilation: flattening, scheduling and backend-aware optimisation

The comparisons of this survey are possible because one source program is lowered to every paradigm rather than being written once per paradigm. That lowering is a compiler, and it is a contribution in its own right: the object it optimises is not a machine but a *protocol*, whose cost model is unlike a processor's, and several standard compiler transformations change sign under it. This section develops the intermediate representations, states what the optimiser is permitted to do, and proves the two structural facts that distinguish the problem from classical instruction selection.

9.1 Two intermediate representations, and why there are two

Definition 9.1. A ***flat program*** over a set of wires $\mathcal{W}$ is a finite sequence of instructions in static single-assignment form, $\iota_1, \ldots, \iota_m$, where each ι_j has an operation, a list of source wires and a destination wire d_j, subject to: every d_j is distinct, and every source wire either is an input or equals $d_{j'}$ for some $j' < j$. We write $\mathcal{P}$ for the set of flat programs. The operations are constants, ring addition and subtraction, multiplication, signed and unsigned comparison, equality, bitwise operations, shifts, selection and, in the non-deterministic setting, division.

Flat programs are the representation a proof system wants, because a constraint system is a conjunction of relations over named wires and SSA gives exactly that. They are also what a homomorphic evaluator wants, because a homomorphic evaluator is an interpreter over a dataflow graph. They are, however, the wrong representation for costing a secret-sharing protocol, for three reasons that Definition 9.2 then addresses.

Definition 9.2. A ***vectorised bytecode program*** is a finite sequence of instructions over registers, where each register carries a ***width*** $\omega \in \mathbb{N}$, a ***domain*** $\mathcal{D} \in \{\mathbb{Z}_{2^k},\ \mathbb{F}_p,\ \mathbb{F}_2\}$ and a ***secrecy*** tag in $\{\mathsf{clear}, \mathsf{secret}\}$. Instructions are partitioned into ***local*** ones, which consume no communication, and ***interactive*** ones, which consume one round and possibly correlated randomness. An instruction of width ω denotes ω independent applications of its operation.

The three deficiencies of Definition 9.1 for protocol costing are now visible. It is scalar, so a dot product of 784 terms appears as 784 multiplications not as as one instruction of width 784, and a cost model reading it would charge 784 rounds where a protocol pays one. It carries no domain, so it cannot express that the same multiplication costs one Beaver triple over $\mathbb{Z}_{2^k}$ and roughly k AND gates over $\mathbb{F}_2$. And it has no notion of an *opening*, which is the unit of communication in a secret-sharing protocol and does not correspond to any single arithmetic operation.

Definition 9.3. A ***lowering*** is a map $\mathcal{L} : \mathcal{P} \to \mathcal{B}$ from flat programs to bytecode programs. It is ***faithful*** when for every P and every input assignment, the outputs of P and $\mathcal{L}(P)$ agree under the plaintext semantics of both.

Our compiler keeps both representations and one faithful lowering between them, in place of a single universal intermediate representation. The reason is not expedience: the two have genuinely different notions of cost, and a representation that served both would have to carry the union of their annotations while permitting only the transformations sound for both. Keeping the flat form as the input to lowering, and confining protocol-aware reasoning to the bytecode, means that a rewrite justified by a protocol's cost model cannot accidentally be applied where the paradigm makes it unsound.

9.2 Flattening and the classical passes

Flattening turns a nested expression into a flat program. Its interesting properties are not the traversal, which is standard, but the interaction of the classical passes with the cost model that follows.

Definition 9.4. Let P be a flat program with designated output wire r. ***Constant folding*** replaces any instruction all of whose sources are constants by a constant. ***Common-subexpression elimination*** identifies instructions with equal operation and equal source wires, retaining one and substituting its destination for the others. ***Dead-wire elimination*** removes instructions whose destinations are not reachable from r in the dependence graph.

Proposition 9.1. Constant folding, common-subexpression elimination and dead-wire elimination preserve the value computed at r, and dead-wire elimination applied after common-subexpression elimination removes at least as many instructions as either order of the two applied once.

Proof. Soundness of folding is that the operations are functions of their arguments; soundness of elimination is that equal operations on equal arguments yield equal results, so substituting one destination for another preserves every later value; soundness of dead-wire removal is that SSA destinations are distinct, so an instruction whose destination is unreachable from r cannot influence

r. For the ordering claim, note that eliminating a common subexpression can only decrease the set of reachable wires: it replaces occurrences of one destination by another and never introduces a new dependence. Hence the set of dead wires after elimination contains the set of dead wires before it, and the second pass removes at least as many. □

Proposition 9.1 justifies the pass order our compiler uses. It is worth stating because the intuition from classical compilation—that elimination and dead-code removal are independent—is false here in a useful direction: collapsing duplicates is what *creates* the dead wires, so running the removal afterwards is strictly better and not merely tidier.

9.3 Scheduling: rounds as a graph height

Definition 9.5. A ***schedule*** of a bytecode program assigns to each instruction a level in $\mathbb{N}$ such that every instruction's level exceeds the level of each instruction producing one of its sources when that instruction is interactive, and is at least that level when it is local. The ***as-soon-as-possible*** schedule assigns each instruction the least level satisfying these constraints.

Proposition 3.4 established that the as-soon-as-possible schedule's number of interactive levels equals the maximum number of interactive operations on a directed path, and that no schedule does better. Two corollaries drive the optimiser.

Corolary 9.1. Under the as-soon-as-possible schedule, all interactive instructions at one level are mutually independent and may be issued as a single batched protocol invocation. Consequently a layer of n independent comparisons costs the rounds of one comparison, and the round count of a network is a function of its depth alone.

Proof. Two instructions at the same level cannot be joined by a directed path, since a path through an interactive instruction strictly increases the level. Independence permits batching by the definition of the protocols involved. The consequence for a network follows because the nonlinearities of one layer depend only on that layer's affine map and not on each other. □

Corolary 9.2. Let $\circ$ be an associative operation whose bytecode realisation is interactive, and let e be a left-nested chain $x_1 \circ (x_2 \circ (\cdots \circ x_n))$. Rebalancing e into a tree of depth $\lceil \log n \rceil$ leaves the instruction count unchanged and reduces the interactive depth from $n-1$ to $\lceil \log n \rceil$. If the realisation of $\circ$ is *local*, rebalancing changes nothing.

Proof. A binary tree with n leaves has $n-1$ internal nodes regardless of shape, so the instruction count is invariant; the height of a balanced tree is $\lceil \log n \rceil$ and of a chain is $n-1$, and by Proposition 3.4 the interactive height is the round count. If $\circ$ is local it contributes no interactive levels in either shape. □

Corollary 9.2 is the cleanest instance of the phenomenon this section exists to formalise: reassociation is a transformation that changes *only* depth, leaving every other count identical, so its value is determined entirely by whether the target protocol charges for depth. Over an arithmetic sharing, addition is local and multiplication is interactive, so rebalancing a product chain pays and rebalancing a sum chain does not. Over a binary protocol an adder is itself a circuit with AND-depth, so rebalancing a sum chain pays as well. Over a constant-round protocol neither pays. One transformation, three answers, determined by the cost model and not by a heuristic.

9.4 Backend-aware rewriting

Definition 9.6. A ***rewrite rule*** is a partial map ρ on bytecode programs that preserves plaintext semantics. A ***cost model*** for a protocol π and deployment δ is a function $\mathrm{cost}_{\pi,\delta} : \mathcal{B} \to \mathbb{R}_{\geq 0}$. A rule ρ is ***profitable*** for (π, δ) at P when $\mathrm{cost}_{\pi,\delta}(\rho(P)) < \mathrm{cost}_{\pi,\delta}(P)$.

Definition 9.7. The ***backend-aware optimiser*** takes a program P, a protocol π and a deployment δ, and returns $\rho_{i_k} \circ \cdots \circ \rho_{i_1}(P)$ obtained by considering candidate rules and applying each only when it is profitable for (π, δ) at the current program, with semantic preservation checked by differential evaluation against the original on random inputs.

The definition is greedy and makes no optimality claim; we state its limitations below not leave them implicit. What it does provide is that the target protocol is a *parameter* of optimisation rather than a fixed assumption, and the consequence is that the optimiser emits different circuits for different protocols from the same source. We give the phenomenon a statement, since it is the contribution.

Proposition 9.2. There exist a rewrite rule ρ, a program P and protocols π_1, π_2 such that ρ is profitable for π_1 at P and unprofitable for π_2 at P.

Proof. Take ρ to be strength reduction, replacing multiplication by a public constant c with a sum of shifts, and let P multiply a secret by c. Under a binary protocol, a $k \times k$ multiplier is $\Theta(k^2)$ AND gates while an adder is $\Theta(k)$, so replacing a general multiplication by a small number of shifts and additions reduces the gate count and hence the cost; ρ is profitable. Under an arithmetic sharing over $\mathbb{Z}_{2^k}$, multiplication by a public constant is a local scaling of each share and costs nothing, whereas the rewritten form is a sequence of local operations that is no cheaper and adds instructions; ρ is not profitable. The two protocols therefore assign opposite signs to the same rule at the same program. □

Remark 9.1. Proposition 9.2 concerns one rule at one program. A subtler point is that profitability varies *within* a program: on $(x \cdot 2^a) + (y \cdot c)$ with c not a power of two, strength reduction is profitable at the first site and not at the second under a protocol where the shift is free and the general constant is not. An optimiser that decides a rule once for the whole program must therefore take both or neither, and will be wrong at one site. Deciding per site costs a full cost evaluation per candidate, which is affordable because evaluating the model is cheap, and it is what makes the emitted circuits differ between protocols by a *subset* relation rather than arbitrarily.

Remark 9.2. The optimiser of Definition 9.7 is greedy with a fixed rule order and evaluates candidate sites independently. It therefore misses any pair of rewrites that is profitable only jointly, and it offers no approximation guarantee. We state this not characterise the loss, because we have not measured it; the honest claim is that the optimiser improves the cost model's prediction relative to no optimisation and relative to a backend-blind choice, not that it is optimal.

9.5 Protocol assignment, and why the objective is not separable

The final transformation chooses *which* protocol evaluates each part of a program, paying for conversions where the choice changes.

Definition 9.8. Let Σ be a finite set of protocols and let K be the set of operation kinds occurring in P. An ***assignment*** is a map $a : K \to \Sigma$. A ***crossing*** is an edge of the dependence graph whose endpoints are assigned different protocols; realising it requires a ***conversion*** instruction. The ***assignment problem*** is to minimise $\mathrm{cost}_\delta(a)$, the cost of the program under a with conversions inserted at every crossing.

Proposition 9.3. The objective of Definition 9.8 is not separable over operations: there are programs and deployments for which the cost of an assignment is not equal to any sum of per-operation terms, even allowing pairwise terms for crossings. Two mechanisms suffice, and either alone is enough:

1. a level's latency is the *maximum* over the interactive instructions it contains, not their sum;
2. conversions at the same level are *batched*, so the marginal cost of a crossing depends on whether another crossing already occurs at that level.

Proof. For (1), consider a deployment with latency L and unbounded bandwidth, and a program consisting of m mutually independent interactive operations. By Proposition 3.4 the schedule has one interactive level, so the latency contribution is L regardless of m; any separable objective assigning a positive term $c > 0$ to each such operation would report mc, which is not constant in m. Moving one of the m operations to another protocol changes the cost by zero if that protocol's level is already occupied and by L if it is not, so the marginal cost of an individual assignment decision depends on the rest of the assignment. For (2), let two crossings occur at one level; their combined conversion cost is that of a single batched conversion, which is strictly less than twice the cost of one whenever the conversion has a fixed component. Then the cost of the assignment that makes both crossings is less than the sum of the costs of the two assignments each making one, contradicting separability with pairwise terms. □

Corolary 9.3. An integer-linear formulation of the assignment problem with per-operation and per-crossing variables and a linear objective cannot express the cost of Definition 9.8 exactly.

Proof. A linear objective over such variables is by construction a sum of per-operation and per-crossing terms, which Proposition 9.3 shows is not equal to the cost function. □

Corollary 9.3 places our approach with respect to the prior work on protocol assignment, which formulates the problem as an integer program and therefore commits to a linear, separable cost. That formulation is a good approximation in a regime where each operation's cost is dominated by bandwidth, which is separable, and it becomes inaccurate in the regime where latency dominates—which is the wide-area regime and precisely the one in which the paradigms differ most. Because evaluating our cost model on a whole program is inexpensive, we can afford to optimise the exact objective by search, giving up optimality guarantees instead of fidelity. A learned surrogate for the cost model would be the alternative if the search space grew beyond exact evaluation, and it is the direction the recent literature takes; we note it as complementary and not as as competing.

Remark 9.3. Our assignment is per operation *kind*, not per instruction. The space is therefore $|\Sigma|^{|K|}$ with $|K|$ a few dozen, in place of $|\Sigma|^m$ with m the instruction count, and the resulting assignment is interpretable: a statement such as "comparisons moved to the binary protocol" is a statement about the cost model. Per-instruction assignment is strictly more expressive and is the natural next step; we do not claim it.

9.6 Summary of the compilation pipeline

Collecting the above, a source program traverses: parsing to an abstract syntax tree; flattening to a flat program (Definition 9.1) with constant folding, common-subexpression elimination and dead-wire elimination (Proposition 9.1); lowering to vectorised bytecode (Definition 9.2) with domain and secrecy annotations; backend-aware rewriting against the cost model of the chosen protocol (Definition 9.7); protocol assignment with conversions priced at crossings (Definition 9.8); as-soon-as-possible scheduling (Definition 9.5); and execution on a protocol engine or, for the proof and homomorphic paradigms, direct interpretation of the flat program.

Two properties of this pipeline are what the rest of the survey relies on. Every paradigm receives the *same* source, so a comparison between paradigms is not confounded by a difference in the program. And the cost model that guides optimisation is the same one whose predictions we validate against measurement, so a claim that an optimisation helps is falsifiable in the same terms as a claim about a protocol.

10 The paradigms, surveyed and measured

In this section we survey and measure each paradigm in turn, in the order in which Section 6 introduced them.

10.1 Secret-sharing MPC

Our MPC layer follows ABY3 [53], so that $x = s_0 + s_1 + s_2 \bmod 2^{128}$ and party i holds the replicated pair (s_i, s_{i+1}) of Definition 6.3. Addition and multiplication by a scalar are local, and multiplication is the triple-free protocol of Theorem 6.2, in which each party sends one ring element to one neighbour and the correlated randomness is regenerated locally from a single pairwise exchange of PRF keys. There is no dealer and there is no preprocessing phase.

We remark that this layer taught us the same engineering lesson on four separate occasions, and that the lesson generalizes beyond our artifact.

> **Finding.** The arithmetic was never the bottleneck; the round structure was. Each time a segment-level measurement said “MPC is slow”, a per-phase breakdown said “one unbatched loop inside it is slow”.

A batched form costs *the same number of rounds for any batch width*. When we adopted such forms throughout, the MPC segment of our convolutional network moved from 40.8 s to 6.1 s, that is, by a factor of 6.7×, and the number of messages fell from 29,211 to 5,591. It means that a published MPC cost is highly sensitive to the batching discipline of its implementation, and consequently that comparing the round counts of two systems, without first verifying that both of them batch, compares implementations and not protocols.

The cost of active security. We measured the malicious protocol of Section 6.2 against the semi-honest one on the same workload and the same transport, and we give the result in Table 4.

The row for comparisons is the one that matters for inference, since every ReLU, every max-pool and every argmax reduces to a comparison.

> **Finding.** Active security costs bytes, not rounds: the MAC doubles what is sent but batches into the same round.

The dot-product row is a small-workload artifact—the semi-honest leg needs only four interactive calls, so a fixed pair-pool cost amortizes against nothing—and we report it instead of hiding it.

	bytes			
Workload	equal width	packed base	int. calls	wall clock (WAN, 25 ms)
Comparisons, n=32, 32-bit	**2.63**×	**9.70**×	**1.06**×	**1.26**×
Dot product, n=64, 32-bit	2.44×	2.85×	2.00×	2.26×

Table 4: Semi-honest ABY3 versus malicious-with-abort, same workload, same transport. Active security costs bytes, not rounds. *Equal width* charges both legs a 128-bit wire; *packed base* lets the semi-honest leg pack to its values' own width, which the verified leg cannot do because its statistical security is $128 - \mathit{used_bits}$. The two differ by 3.7× on the workload that matters for inference.

A second result generalizes, **with a correction we owe to Section 13.4**. The statistical slack that ring-based verification normally charges as a widening to $\mathbb{Z}_{2^{k+s}}$ [21] is already *present* when the implementation shares over a ring much wider than its values: Q16.16 values in a 128-bit ring leave **59 bits** of statistical security.

We previously added that this came at *no widening cost*. That is false, and packing the semi-honest paths is what exposed it: the 128-bit ring had been costing 3.1–5.6× in bytes on every one of them, which is exactly what narrowing to the values' own width recovered. The verified path cannot recover it—its security is $128 - \mathit{used_bits}$, so narrowing to the value width sets λ to zero. The slack was never free; it was invisible, because nothing was packed and so nothing revealed its price. The 2.63× above is therefore a fair ratio at equal width that *understates* the cost of active security against a packed baseline. Measured: 9.70× on the comparison workload—the semi-honest leg packs from 234,036 to 63,416 B while the verified leg is unchanged at 615,446. The ratio of ratios, 3.69×, falls inside the packing factors we measured on comparable paths, as predicted.

10.2 Garbled circuits

As we described in Section 6.3, in the three-party honest-majority setting the input labels are delivered by the co-garbler and no oblivious transfer is required. Since oblivious transfer is the expensive part of two-party garbling, this changes the economics substantially, and it is the reason why garbling is competitive in this setting at all.

We measured the two routes head to head over the identical transport, on 1,352 comparisons of 20-bit values. Bit decomposition needs 9 rounds and 1.29 MB, whereas the garbled circuit needs 1 round and 1.69 MB. It means that **bit decomposition is the byte-cheaper of the two**, by a factor of 1.3×, and that the advantage of garbling lies entirely in the rounds: the wall clock favours it by 18.8× on a local network and by 4.0× on a wide-area network with 80 ms of round-trip time and 100 Mbit s^{-1} of bandwidth, the ratio shrinking as bandwidth rather than latency becomes the binding term.

This result reverses a figure that we published. The same experiment previously reported bit decomposition at 5.94 MB, which made garbling appear 3.5× cheaper in bytes *as well as* faster. That difference of 4.6× was serialisation padding: a 20-bit ring share travelled inside 128 bits, and bit decomposition is almost entirely ring traffic whereas garbled tables are already compact. It means that the overhead fell on one side of the comparison only. Once the encoding was corrected, as we describe in Section 13.4, the ordering in bytes inverts and only the advantage in rounds survives. We remark that both routes remained bit-exact against the plaintext reference throughout, and consequently no correctness test could have revealed the problem.

A correction to another of our claims. The figures given above are for the *garbling primitive in isolation*. The deployable three-party protocol is smaller, once we include the $Y2A$ return leg that converts the Boolean output back into an arithmetic share: it is 7.7× on a local network, falling to 1.8× at 80 ms and 100 Mbit s^{-1}, and between 3.9× and 2.4× for the fused ReLU ∘ rescale. We record both figures, because the distance between them is exactly what is lost when a number is cited.

An earlier version of this paragraph quoted between 1.4× and 1.7× for the deployable path. That figure was taken not from this experiment but from the *MPC segment* of our convolutional network, which is a different benchmark on a different workload, and it was presented as though it corrected this one. That segment measures 6.1× today. The old figure was correct when it was written, and it became stale for a reason that we record in Section A: an unbatched scheme-switch import was then 94% of the MPC segment, which loaded both routes equally and therefore compressed the ratio toward one. We consider that both corrections illustrate a single rule, namely that one should quote the deployable protocol, on the workload about which the claim is being made, and name the benchmark that produced the number.

10.3 Fully homomorphic encryption

We use the cheapest homomorphic primitive that exists, namely a single ciphertext-by-ciphertext multiplication, with no relinearization keys, no evaluation keys and no bootstrapping, decrypting the resulting degree-two product directly under s^2 as we described in Section 6.4. Our parameters for BFV are $n = 4096$, $q = 2^{100}$ and $t = 2^{20}$, which lie inside the ceiling of approximately 109 bits on $\log q$ for 128-bit security at that ring dimension [2].

We consider that two results about packing are worth surveying.

- **Reversed-coefficient packing** turns an entire matrix–vector product into one multiplication. We find that it is *already optimal for dense layers*, since a product of degree n admits at most n/in separable outputs, and therefore $\lceil \text{out} \cdot \text{in}/n \rceil$ multiplications is the limit. The SIMD packing of Gazelle [40] matches this throughput, but it requires rotations and Galois keys, and consequently it forfeits the property of needing no evaluation keys.

- **Convolution packing** in the manner of Cheetah [37] turns an entire output channel into one multiplication, independently of the size of the image. We measured the two layouts against each other at 28×28, and 1,352 multiplications become **two**, while the time spent in homomorphic evaluation falls from 77.5 s to 264 ms.

Finding. BGV is indistinguishable from BFV at identical parameters (repeated runs put the gap at +0.5%, +0.5%, −0.1%). Where the message sits in the ciphertext, high bits or low, is not where the cost is.

On the arithmetic side, we use a multi-modular number-theoretic transform, in which NTT-friendly primes are employed as a computational device and the result is reconstructed by the Chinese remainder theorem, because a power-of-two modulus q admits no direct transform. After we moved its butterfly to Shoup-preconditioned multiplication, it became 1.8× faster than NTL together with GMP [59] at every size that we measured. We remark that we retain NTL as a *differential oracle* rather than as a dependency, and that this is what made it safe to rewrite the hottest code of the repository.

depth	exact?	total	fhe	switch	mpc	our boots.	levelled-competitor boots.
2	✓	145 ms	20	59	54	**0**	0
8	✓	539 ms	81	239	210	**0**	3
32	✓	2,119 ms	333	958	820	**0**	15

Table 5: Deep private inference, width 16, weights *and* activations encrypted, real ABY3 protocol, BFV at $n = 4096$, $q = 2^{100}$. Linear in depth and bit-exact at every depth; the exactness is the evidence for the noise claim, since accumulated noise would make deep outputs diverge.

width	constraints	prove	verify	proof bytes
1	14	6.3 ms	1.0 ms	18,544
64	589	8.0 ms	1.3 ms	47,808
784	**7,159**	**32.3** ms	**4.8** ms	**111,712**

Table 6: Constraints grow linearly (≈ 9 per input); proof size does not—a 286× larger circuit yields a 6× larger proof. Width 784 is the MNIST input dimension.

10.4 The hybrid and the no-bootstrap result

In this section we describe our hybrid construction and the consequence that we consider its main result. The central observation is that the scheme switch which we perform in order to evaluate a ReLU *also refreshes the ciphertext*. When we switch back in the direction of Section 6.6, each party encrypts its share component afresh and the evaluator adds the ciphertexts, and a sum of fresh encryptions is itself a fresh encryption. It means that the noise budget of Figure 6.4 is reset at every boundary between layers. Consequently a network of arbitrary depth runs with **zero bootstraps**, at a cost that is linear in the depth, on an engine that supports only a *single* multiplication. We give the measurements in Table 5.

We remark that our construction also encrypts the **weights**, and that this is a stronger privacy model than the one of Gazelle, Delphi and Cheetah, in which the server holds the model in plaintext. The exactness column of Table 5 is the evidence for the claim about noise, since accumulated noise would cause the outputs at large depth to diverge from the plaintext reference, and they do not.

10.5 Zero-knowledge proofs

In this section we measure the succinct-proof paradigm of Section 6.7. The statement that we prove is a quantized neuron in the Q16.16 format, namely $\mathrm{ReLU}((\sum_i w_i x_i) \gg 16 + b)$, with public weights and private inputs, and we prove it on a real Binius64 prover not on a placeholder. We give the results in Table 6.

We consider that two of our implementation findings generalize to anybody who builds arithmetic circuits over 64-bit words. The first is that comparison gates are **unsigned**, and therefore a signed comparison must flip the sign bits beforehand; without that step $\mathrm{ReLU}(-1)$ returns -1, which is to say that the circuit is perfectly sound about the wrong function. The second finding is sharper, and we state it as follows.

Finding. Division cannot be checked the obvious way. A circuit cannot divide, so the prover supplies quotient and remainder as witnesses; but checking $a = bq + r$ with $|r| < |b|$ is *unsound*, because circuit multiplication wraps mod 2^{64}. For $a = 1, b = 3$ a cheating prover sets $r = 0$ and $q = 3^{-1} \bmod 2^{64}$ and both checks pass. Requiring the 128-bit product's high word to be the sign extension of the low word closes it.

route	domain	B/triple	triples/s	rounds
LowGear (SHE) [42]	$\mathbb{F}_p$, 60-bit prime	**192**	97,304	2
Semi2k (OT) [41]	$\mathbb{Z}_{2^{64}}$	3,074	2,681	3

Table 7: Two parties, semi-honest. **The two rows are over different domains**; see the caveat below for the apples-to-apples figure.

We remark that we drive that forgery through the real prover as a test, so that the soundness of the repair is demonstrated and not merely asserted.

10.6 Preprocessing: OT versus SHE

In this section we measure the preprocessing that Section 6.2 described, because it is where the comparison between homomorphic encryption and secret sharing is usually decided, and because it is usually decided implicitly. We give the two routes in Table 7.

A caveat about comparability that the literature routinely drops, and that we dropped ourselves. A prime field of approximately 60 bits does *not* represent every 64-bit value, and therefore Table 7 compares a triple over a 61-bit field against a triple over a 64-bit ring. At **equal width of** 64 **bits**, LowGear requires two limbs under the Chinese remainder theorem, which gives 384 B per triple against the 3,074 of the oblivious-transfer route. It means that the advantage in communication is **8.0**× and not the 16× that a naive reading suggests. Our own experimental record quoted 16.3×, and we correct it here. We remark that this is precisely the class of error that a single-harness study is positioned to catch, and that it is the reason why Section 12 exists.

The measured cost of the oblivious-transfer route also validates our analytic model, since we measured 3,074 bytes against 3,136 predicted, an error of 2 %.

11 Related work, and where this study sits

In this section we position our work with respect to the literature. We remark that organizing this literature by primitive would reproduce Section 6, and therefore we organize it instead by *what a system is for*, because in our view that is what determines whether the numbers of one system can be compared with those of another.

11.1 Frameworks that run many protocols

MP-SPDZ [41] is the closest system to this one in ambition and the natural point of comparison. It implements dozens of protocols — dishonest and honest majority, fields and rings, semi-honest and malicious, with the SPDZ [25], SPDZ2k [21], Semi2k and LowGear [42] families among them — behind a Python-like front end that compiles to a bytecode executed by a virtual machine. It is more complete than Baru on the MPC axis by a wide margin, it is the source of the triple-preprocessing figures in Table 7, and any practitioner choosing an MPC protocol should start there rather than here.

We consider that three differences matter for a survey, and we emphasize that none of them is a claim of superiority.

The first difference is the **scope of the paradigm**. MP-SPDZ is a framework for secret-sharing MPC, in which somewhat-homomorphic encryption appears as a means of generating triples. Our

	MP-SPDZ [41]	Baru
Purpose	production-grade MPC across many protocols	one harness in which paradigms are measured against each other
Paradigms	secret sharing, garbled circuits, OT	the same, plus RLWE-based FHE, FHE↔MPC switching, and succinct proofs
Execution	compiler to bytecode → VM	CEK interpreter, protocol chosen per call at run time (Section 8.1)
FHE role	preprocessing (SHE-based triples)	preprocessing *and* a first-class evaluation paradigm
Breadth vs. depth	many protocols, one paradigm family	few protocols, four paradigm families

Table 8: Baru is not a smaller MP-SPDZ; it answers a different question. MP-SPDZ optimises *within* the secret-sharing paradigm and treats FHE as a preprocessing device. This study needs FHE, garbling and secret sharing to be interchangeable *evaluation* strategies for the same program so their costs can be subtracted.

survey requires homomorphic encryption as an evaluation paradigm in its own right, because the question of whether a given layer should be computed under homomorphic encryption or under secret sharing has no answer inside a system in which one of the two alternatives is only ever preprocessing.

The second difference is **compilation against interpretation**. A compiler is obliged to commit to a protocol in order to emit code, whereas an interpreter can leave the choice to a backend and consequently hold the program fixed across the paradigms, as we argued in Section 8.1.

The third difference is **maturity of the optimizer**, and here we were until recently describing our own artifact inaccurately. We had written that MP-SPDZ optimizes protocol performance while we optimize the comparability of measurements. The second half is true and the first half understates what we have: our compiler carries an instruction graph, an as-soon-as- possible levelling pass that collapses independent operations into one batched round, common subexpression elimination, reassociation of dependent reductions, per-protocol cost models, and an assignment pass that chooses a protocol per kind of operation. Those are the same categories of pass that make MP-SPDZ's compiler effective, and Table 9 measures what they buy against an expert baseline.

Finding. Automatic scheduling reaches *parity* with the expert, and the metric that shows it is neither of the two we reached for first. The compiled route makes half the primitive calls (12 against 24) and sends 0.61× the messages, yet blocks the same number of times (33 against 32 to 33) and takes the same wall clock. Calls and messages do not predict cost here; waits do.

We should be precise about what "the same number of times" is doing in that sentence, because we were not at first, and we corrected ourselves twice. Waits are *almost* deterministic, not deterministic. The mechanism is visible in the instrumentation: a receive is counted as a wait only when it actually polls the transport, and one that finds its message already delivered does not, so the quantity is the number of times a party *had* to stop, and that depends on how the three threads interleave.

Our first correction was to withdraw the phrase "exact parity", which we had written on the strength of two equal readings of 33. Four repetitions showed the expert's route reading 32 or 33 while every other row held. Our second was to withdraw the accompanying claim that only that one row was unstable: a later run that swept a second round-trip time moved three rows by one count each. The network profile perturbs the interleaving, so the jitter is a property of the counter and not of one route. Every wait figure we report should be read as ±1.

route	calls	**waits**	messages	bytes	ms (RTT 0)	ms (RTT 20)
hand-written gadgets	24	**33**†	61	33,466	53.2	493.2
hand-written, gc cmp	21	**12**	46	30,191	28.4	279.2
compiled, serial	15,304	18,362	18,367	3,822,372	26,344	245,972
+ ASAP levelling	15	43	47	881,324	195.1	757.3
+ CSE	15	43	47	33,504	61.0	620.7
+ reassociation	12	33	37	33,262	48.3	486.3
+ gc comparisons	9	**12**	22	29,987	32.4	269.9

Table 9: One network, $x(16) \to \text{dense}(8) \to \text{ReLU} \to \text{dense}(4) \to \max$ with secret weights, both routes over the same transport and both checked against the plaintext reference. The gadget route is written by an expert who called the batched primitives deliberately; the compiled route is scalar source with no batching hint. *Calls* counts backend primitives, *waits* counts the times a party blocked for a peer. The last rung is a protocol assignment in place of a compiler pass, so the hand-written route is given the same assignment in the second row — the two rows in bold are the comparison that means something. Bytes are one run and are stable; the millisecond columns carry the harness's run-to-run variance of roughly 13 %. †Waits carry a ±1 of their own, and the marker is on this row because it is where we first saw it: over four repetitions at RTT 0 it read 32, 33, 32, 33, while every other row was identical in all four. A run that also swept RTT 20 then moved three rows by one count, so the jitter is not confined to this row — it is a property of the counter. A receive that finds its message already delivered does not block, so the quantity is how many times a party *had* to stop, and that depends on thread interleaving, which the network profile itself perturbs. Read every wait figure here as ±1.

What survives is the argument, because none of it rests on a single count. Parity is parity within the counter's resolution, and the effects we measure against it — a factor of 136 for levelling, 2.7 for the protocol assignment, 214 for the local-cost constant — are not one-count effects. We record the two corrections because the temptation each time was to quote the equality we had happened to observe.

The reason is that a primitive call is not a round. A `mul_batch` blocks once; a `lt_batch` performs an entire bit decomposition and blocks once per stage of the prefix adder. Counting calls therefore weighs a comparison and a multiplication equally, and a route that trades many multiplications for few comparisons looks better than it is. We had written this section twice on the strength of the call count, once claiming the compiler beat the expert and once attributing the residual to executor overhead; both readings were artifacts of the counter, and neither survived adding one that counts blocking.

We read the corrected table as good news of a narrower kind. Automatic levelling over a dependence graph, with no hint from the source, recovers what a careful author achieves by remembering to call the batched primitive — which is the premise the rest of this section needs. It also shows which pass does the work: levelling alone leaves the compiled route at 43 waits against the expert's 33, and it is reassociating the dependent reduction that closes the gap to 32. Common subexpression elimination contributes nothing to waits at all, and a great deal to bytes (881 against 33.5 kB), which is why the ladder is reported per pass not as as one number.

The cost of a primitive, measured. Once blocking is the unit, the cost of each primitive can be measured and not assumed, and Table 10 does so at two batch widths. The width columns agree, which is the property that makes levelling worth doing: a batched primitive blocks the same number

of times whatever its width, so collapsing a layer into one call is free in rounds.

primitive	waits, width 1	waits, width 64	modelled rounds
`mul_batch`	2	2	1
`lt_batch`	9	9	7
`a2b_batch`	9	9	7
`reveal`	1–2	1–2	1

Table 10: Blocking cost per primitive at $\ell = 32$, against what our cost model charges

A comparison costs about four and a half multiplications, which is the ratio any scheduler trading one against the other has to know, and our model's structure already reflects it: it prices a comparison as an opening plus $\lceil \log_2 \ell \rceil + 1$ multiplication rounds, against one for a multiplication. The constants have not been fitted — the model charges 1 where the implementation blocks twice, and 7 where it blocks nine times — so the ratio is the right order (7× modelled against 4.5× measured) while the absolute figures are not yet calibrated. Now that blocking is counted, they can be.

We also record a place where the model and the executor are defined differently, together with what that costs, because the second part is easy to assume and was not what we assumed. The model charges a level the *maximum* over the primitives it contains, on the reasoning that a level is one logical round; the executor issues one batched call per *kind* per level and each blocks, so a level holding a multiplication and a comparison pays their *sum*. We expected this to be the largest item outstanding. Counting the mixed levels in the network of Table 9 shows there are *none*: the schedule has 8 interactive levels and issues 8 batched calls. The reason is structural not lucky — a ReLU written as $z \cdot (z > 0)$ puts its comparison and its multiplication at consecutive levels by construction, and so does a mux-style max — so the two definitions differ and this class of program does not pay for the difference. We keep the counter (`ExecStats::mixed_levels`) so that a schedule which does pay is visible not inferred, and we do not claim the optimisation.

Which primitive to attack, and the assignment that follows. Attributing the 32 waits to levels says where the budget goes: three of the eight interactive levels hold a comparison, and at nine blocks each they account for 72 % of the total, against two blocks for a multiplication level. The comparison is therefore the primitive worth changing, and Baru has three of them. Table 11 measures what each costs.

comparison backend	waits	messages	kilobytes
`a2b` — batched bit decomposition	10	16	127.1
`falcon` — Π_{DReLU} over $\mathbb{Z}_p$	11	44	**49.1**
`gc` — three-party garbled circuits	**3**	11	109.9

Table 11: 64 comparisons at $\ell = 32$, online phase, preprocessing filled in advance. Bytes are the whole run including preprocessing.

The spread is 3.7× in blocking, which is what makes an assignment worth deriving at all: if the three agreed there would be nothing to choose between them. Two things follow. The first is that `gc` ***strictly dominates*** `a2b` here, in blocking and in bytes both, and `a2b` was the default the compiled route used — so the assignment pass had a free improvement available that no correctness test and no byte count would ever have surfaced. The second is that `falcon` is the byte-cheapest by

2.2× while being the block-dearest, which is the same trade our crossover measurements report at the network level (§13.4) arriving here at the level of a single primitive. An assignment cannot be read off one column.

Finding. Assigning the comparison levels to the constant-round backend cuts the compiled route from 33 blocks to 11 to 12, a reduction of 2.75 to 3.0 times and the largest single lever after levelling itself. It is also *not a compiler result*: giving the hand-written route the same assignment takes it from 32 to 33 to 11 to 12 as well, so measuring the assigned compiler against the unassigned expert would report roughly a 0.36× win of which the compiler contributed nothing. Compared like with like the two are at parity, and they are at parity in the strong sense that their observed *ranges* coincide over eight runs not that two particular readings happened to agree.

Keying the choice on the deployment, and the third term that decides it. An assignment that is right for one link is not an assignment. Baru's precompile registry previously held one champion per gadget, set by a maintainer on the strength of a benchmark; it now also answers *which entry for this link*, ranking the measured candidates under the latency model. We built that expecting Table 11 to demonstrate it: `falcon` is 2.2× cheaper on the wire and 3.7× dearer in blocking, so the champion ought to flip between a thin pipe and a slow one. We then measured all three backends across the grid, and report the result because it is not the one we expected.

backend	network free	RTT 0, 10 Mbit s^{-1}	RTT 40, 100 Mbit s^{-1}
`a2b`	33 ms	206 ms	334 ms
`falcon`	79 ms	130 ms	1,056 ms
`gc`	**13** ms	**105** ms	**201** ms

Table 12: The same 64 comparisons at $\ell = 32$, wall clock, across three links. The first column is `NetworkProfile::INSTANT` — zero latency, unlimited bandwidth — so it isolates local cryptographic work.

Finding. Garbled circuits are fastest at every point of the grid, and the reason is a term our cost model does not contain. With the network free, Falcon's comparison costs 6× what the garbled one does at this batch size — that is local arithmetic over $\mathbb{Z}_p$ together with its preprocessing — and a model of the form waits × RTT + bytes/bandwidth charges every candidate zero for it. The true ratio is larger still, and we arrive at it below: measured on a whole diagnosis and not on 64 comparisons, the factor is 16.4 (Table 13).

The omission is not cosmetic, and it failed in two distinct ways. On a fast link the model predicts 0 s for all three candidates and cannot rank them at all, so the choice fell through to a tie-break which returned the *dominated* entry — a wrong answer instead of an arbitrary one. On a thin pipe it mispicked outright, preferring Falcon at 130 ms to garbled circuits at 105 ms: the 78 kB Falcon saves is worth 62 ms at 10 Mbit s^{-1}, and it spends 66 ms more in local work to save it. A measured local term restores agreement with the wall clock to 1.00× at all three points.

The microbenchmark understates the spread, and we nearly built on it. Table 12 measures 64 comparisons in a fresh session, which is the natural thing to measure and the wrong denominator. Measuring the same comparison work inside a whole diagnosis — 7,137 comparisons, the workload the assignment exists for — gives the nonlinearity compute directly, and it does not agree.

backend	in a diagnosis	implied marginal, 64 wide	microbenchmark, 64 wide
`gc`	137 ms	1.2 ms	13 ms
`a2b`	1,063 ms	9.5 ms	33 ms
`falcon`	2,248 ms	20.2 ms	79 ms
spread	1 : 7.8 : 16.4	1 : 7.8 : 16.4	1 : 2.5 : 6.1

Table 13: Local comparison cost measured two ways. The middle column divides the first by the comparison count and rescales; the right column measures 64 comparisons on their own.

A 64-wide call costs roughly ten times its own marginal cost, because most of it is session setup and share distribution and not comparison. The absolute inflation would not matter — a ranking does not read absolute numbers — but it is not uniform across the candidates, and the ratio it distorts is exactly what a ranking does read. It makes garbled circuits look 2.5× cheaper than bit decomposition where the marginal figure is 7.8×.

This is the same slope-against-intercept error we corrected in the analytic model's local-work constant, in a different instrument, and we record it as a pattern and not as as two incidents: ***a cost measured at one scale and applied at another is an intercept masquerading as a slope***, and the symptom is that the fixed part is common while the ratio is not.

We then made the correction, and it was wrong. Substituting the marginal figures for `local_secs` while leaving the byte term alone flipped the selection to Falcon on a thin pipe, where measurement says garbled circuits win — caught by the validation test not by inspection. The error was making one term marginal and not the other, and the byte term does not decompose the same way: Falcon's traffic includes *its own* preprocessing, which is a protocol-specific cost and not a fixed one common to the candidates. There is no common term to strip. We reverted to validated totals and kept the marginal measurement as a statement about direction: every correction to the ratio favours the champion already selected, so the assignment is more right than the registry's numbers imply and never less.

We keep the link-keyed machinery and withdraw the example. The machinery is the right shape, the byte-against-blocking trade it expresses is one we measure at the level of whole routes (§13.4), and the assignment it selects is worth 2.75 to 3.0 times in blocking, the range being the counter's own ±1. What we no longer claim is that the comparison gadget is where the champion flips: it is not, and the test that asserted a flip now asserts the measured fact, so a change that produces one will fail it and be looked at.

The same term in the analytic model, and what fitting it changed. Our analytic cost model does not have the omission just described — it carries a local-work term and reports a *compute* bottleneck when that term dominates. Its problem was narrower and worse: the constant feeding it had carried the comment *"an estimate; calibrated in P1"* since it was written, and that calibration never happened. Our own calibration command validates predicted rounds and predicted online bytes against a measured run and reports zero error on both; it never touched compute. So we fitted it, and we describe the method because the number is machine-specific and the method is not: wall clock on a free network is per-lane work *plus* a fixed cost — thread spawn, share setup — so dividing one measurement by its width charges the fixed part to every lane. We measure four batch widths from 1,024 to 65,536 and fit the ***slope***, reporting the intercept separately; Table 14 gives the result together with the two quantities that bracket it.

quantity	measured
`mul_batch`, real transport, network free	**1,072** ns per lane
local-only `add`, no communication at all	145 ns per lane
the estimate it replaced	5 ns per lane

Table 14: Fitting the per-lane local cost of an ABY3 multiplication

Finding. The assumed constant was 214 times low for a multiplication lane, and 29 times low even for the arithmetic floor: an addition, which communicates nothing whatsoever, already costs 145 ns in share bookkeeping and a ring addition. A multiplication is 7.4× an addition, the remainder being serialization and the channel round.

Neither reading rescues the constant, so we replaced it and marked it fitted, leaving every other protocol's at its literature estimate and marked as such. The correction is not cosmetic. On a three-instruction program compute is 3 µs against 878 µs of latency and nothing moves; on a *batched* program — 513 interactive operations over two dependence levels, which is the shape of a dense layer — the predicted total moves from 169 µs to 716 µs, a factor of 4.2, and the reported bottleneck flips from latency to compute. That is the regime every neural-network claim in this paper lives in.

We are obliged to add that the resulting table is *not* a ranking, and we have made the artifact say so. ABY3 is now the only protocol whose compute term is measured; the others carry estimates that are presumably low by a comparable factor, so reading a cross-protocol ordering off a compute-bound table would be the same error of comparing incommensurable quantities that this section has already recorded three times. The comparison output now emits a note whenever any row is compute-bound, naming which constant is fitted and declining the ordering. We also record that our three-party *simulator* is unusable for timing — it has no batched multiplication and spawns three operating-system threads per multiplication, measuring 58 µs per lane against the real backend's 1.07 µs — because it is the instrument one would naturally reach for, and the calibration test now says so at the point of temptation.

We labour the control because this benchmark's own history is three readings that compared quantities which were not comparable, and the test that fixes the fourth now asserts that the mismatched comparison is the flattering one — so the reading that would overstate the compiler is pinned in place as a thing to avoid and not described in a comment. What the result does establish is the premise the next section needs: the comparison backends differ enough to be worth choosing between, the choice is invisible to correctness, and the compiler is a good enough consumer of such a choice that it will not squander it.

We remark that this is a narrower claim than MP-SPDZ's, and deliberately so. MP-SPDZ implements tens of protocols against our handful, its compiler vectorizes where ours does not, and its engineering is years ahead; a practitioner choosing an MPC protocol should still start there. What Table 9 supports is only that automatic scheduling over a dependence graph recovers, and slightly exceeds, what a careful author achieves by remembering to call the batched primitive — which is the premise the rest of this section needs, because a per-operation protocol assignment is worth deriving only if the code that consumes the assignment is not itself leaving rounds on the table.

Frameworks based on circuit compilation. ABY [26] introduced mixed-protocol two-party computation with explicit conversions between arithmetic, Boolean and Yao sharings. HyCC [12] automated the decision of *partitioning* a program, compiling ANSI C into a mixed-protocol program. CirC [55] unified circuit compilation across MPC and proof back ends, and Silph [16] treats the

assignment of protocols as an optimization problem. Our own cost model, which we describe in Section 13, is a small instance of this line, and we report there that the hybrid split for a neural layer can be *derived* from measured per-operation costs not annotated by hand. We remark that the distinction from those systems is again interpretation: they choose an assignment and emit a circuit, whereas we choose per call and keep the program.

11.2 Three-party honest-majority protocols

ABY3. The work of Mohassel and Rindal [53] is the protocol layer that our artifact reimplements, namely replicated sharing over rings, the triple-free multiplication of Theorem 6.2, conversions among arithmetic, Boolean and Yao representations, and fixed-point primitives for machine learning. For our purposes its central insight is that with three parties and one corruption the *online* cost of a multiplication falls to one ring element per party and requires no preprocessing at all, and it means that honest-majority secret sharing becomes competitive with homomorphic encryption on linear algebra.

Falcon. On that base, Wagh et al. [62] construct a set of protocols for the nonlinearities, and we reproduce them in detail: private comparison over a small prime field $\mathbb{Z}_p$, the `wrap3` computation of the wrap of three summands, Π_{DReLU} and Π_{ReLU}, the share selection Π_{SS}, max-pooling with a one-hot indicator, and exact truncation. Section 13.4 is largely a reproduction study of these protocols, and we consider it the part of this survey in which reproduction produced the most, because matching a published protocol at its own ring width and on its own network obliged us to separate the offline cost from the online one, and that separation changed our conclusion twice.

We flag two details for anybody who reproduces this work. The first is that the ReLU of Falcon carries a one-hot indicator because Falcon also *trains*, and consequently charging an inference-only pipeline for that indicator is not a fair comparison. The second is that its exact truncation states a bound on the mask which is necessary for correctness and insufficient for privacy, in the sense of Definition 6.2; we describe the consequence in Section 15.

Other work in the honest-majority setting. SecureNN [61] preceded Falcon with three-party protocols that avoid garbled circuits for comparisons. ASTRA [14] and the works of Chida et al. [18] and of Eerikson et al. [30] supply the machinery for malicious security that we described in Section 6.2, and SING [7] continues the thread of nonlinearity protocols. We implement the MAC construction of Chida and Eerikson and we measure its cost against the semi-honest baseline instead of assuming it.

11.3 FHE-based and hybrid inference

The space of hybrid designs is where the largest differences that we measure in this survey are found. Gazelle [40] established the pattern, namely linear layers under lattice-based homomorphic encryption with packing, nonlinearities under garbled circuits, and conversions between the two. Delphi [52] moved work into an offline phase and traded accuracy for speed by means of polynomial activations, and MiniONN [50] had taken an earlier mixed approach. CrypTFlow2 [56] contributed protocols for comparison and truncation with substantially better constants, and it is the reference point toward which our own findings repeatedly point. Cheetah [37] contributed the coefficient-packed convolution of Section 6.4, which requires no rotations, and it is the technique whose exactness we verify at 5×5 in Section 14. We remark that PEGASUS [39] and CHIMERA [9]

address switching *between homomorphic schemes* not between homomorphic encryption and secret sharing, which is a different problem from ours.

Compilers for homomorphic deep learning, and how we use one. A second line builds *compilers* not protocols. Fhelipe [47] automates data packing; Orion [29] accepts PyTorch networks, contributes single-shot multiplexed packing and automatic bootstrap placement, and reports the fastest published CKKS inference figures we are aware of, including 0.23 s on MNIST and the first high-resolution homomorphic object detection; HE-LRM [34] extends Orion to recommendation models with encrypted embedding lookups, and reports a single bootstrap consuming 75 % of a 24.22 s inference—the discontinuity our no-bootstrap hybrid exists to avoid. HEIR [4] is different in kind: it is MLIR-based compiler *infrastructure* spanning BGV, BFV, CKKS and CGGI with exits to OpenFHE, Lattigo, tfhe-rs and several hardware accelerators, and its paper claims generality rather than performance.

We use HEIR in Section 13.2 as a *baseline generator*: it supplies a homomorphic arm running on tuned C++ and not on our own arithmetic, which is the confound our own threats section concedes. We note two practical findings from doing so, because they are not in its documentation. Its Python front end compiles through Numba and cannot express a dense layer at all—tensor subscripting, matrix multiplication and reductions are each rejected—so the route that works is to emit `linalg` MLIR directly, which is the level its machine-learning pipeline targets and which the PyTorch route reaches by way of torch-mlir. And its emitted code sets the multiplicative depth and the plaintext modulus but never the security level, so the level in force is the backend library's default; we read it back out of the generated code instead of assuming it.

Our contribution relative to this line is not a faster hybrid. It is that the hybrid route, the pure homomorphic route, the pure secret-sharing route and the garbled route are all present in a single artifact over a single transport. It means that the question of what the hybrid actually buys, and of where the crossover lies, can be answered by measurement not by comparing numbers taken from four papers with four experimental setups.

System-level comparisons of the paradigms. Huang, Maeng and Suh [36] characterise secret sharing against homomorphic encryption for private inference across latency, throughput, memory, energy and monetary cost, and report the shape we reproduce in Section 13.2: secret sharing wins on latency over a fast link, homomorphic encryption wins on throughput and is insensitive to the network, and the offline phase dominates secret sharing's real cost. Their study assembles systems from the literature, which is precisely the citation-assembly problem of Section 1; ours holds the computed function, the machine and the accounting fixed and varies only the paradigm. The two are complementary, and that their conclusions agree is evidence for both.

11.4 Homomorphic encryption schemes and libraries

BFV [32] and BGV [10] are the two schemes that we implement, in the encodings of Definition 6.9. CKKS [17] targets approximate arithmetic over the reals and we do not use it here, because our results depend on bit-exactness against a plaintext reference. TFHE [19, 60] targets fast Boolean bootstrapping and is present as a backend for comparison. SEAL [58] and NTL [59] are the reference implementations against which any work of this kind is measured, and we make no claim of competing with them on constants, as we state in Section 15. Our selection of parameters follows the standard for homomorphic encryption [2] and the line of work on lattice estimators [3, 28, 43].

We remark that our one negative result in this area is a scheme that we *removed*. It was an NTRU variant in the style of LTV and YASHE [51, 8], it was fast, and at the parameters that our

construction requires it was not secure; we describe the episode in Section A.

11.5 What is distinctive here

No individual protocol in our artifact is novel, and we claim none. We believe, however, that three elements are unusual in combination.

1. **Four paradigms behind one abstraction, in one artifact and over one transport**, with a single program text reaching all of them, as we argued in Section 8.1. The existing frameworks are deep in one family of paradigms and they treat the others as preprocessing or omit them entirely.

2. **Comparability treated as the object of study.** The uniform accounting, the bit-exactness gate and the validated cost model for the network are themselves the contribution, and we remark that several of our findings are corrections to comparisons and not new speedups; we present them in Sections 12 and A.

3. **Negative and withdrawn results kept in the record**, with the component deleted in the cases where it lost. We report three optimizations that were measured and rejected, four successive wrong answers to a single question about a crossover, and two gaps in the security of our own code, one of which we created while implementing a published protocol faithfully enough to inherit an under-specified precondition.

12 Methodology

In this section we present our methodology. It is the part of this work that we would most like other artifacts to adopt, and consequently we state each element as a rule together with the error that the rule exists to prevent.

12.1 Identical transport, uniform accounting

Every protocol, whether arithmetic secret sharing, garbled circuits, the Falcon reproduction or the hybrid, runs over the same transport stack, which is wrapped in counting and latency decorators that observe the traffic without touching the code of the protocol. It means that both legs of any comparison pay a real serialization, a real passing of messages and a real counting of bytes, and consequently there is no asymmetry in which one side is simulated and the other is not.

We remark that this matters more than it appears to. Our in-process simulator *is not the protocol*, since its multiplication of shares reconstructs and reshares locally, which is free and insecure, and it understated the pure secret-sharing baseline by approximately 2.7×, namely 6.7 ms simulated against 18 ms real. Every number that we report for secret sharing in this survey is the real wire protocol, with three party threads that exchange serialized messages.

12.2 A validated network model

Our cost model charges rounds × RTT + bytes/bandwidth, which allows a benchmark to move the parties between continents without deploying anything. We **validated it against a real loopback TCP socket**, and we measured a worst-case error of 13.6 % and an error below 1 % through the middle of both sweeps, with the constants of the link measured and not assumed. It means that the

cost really is linear in the rounds and linear in the bytes, with no missing cross-term, and this is what licenses us to compute a wall-clock figure from measured counts.

Constructing this validation revealed two defects in our harness that had been silently distorting results. The per-message latency timers could *reorder* back-to-back messages, and the bandwidth model did not serialize bulk transfers cumulatively, which *undercharged precisely the byte-heavy phases* that the crossover experiments are about. Both defects now have regression tests. We mention them because a cost model that has never been checked against a socket should be assumed to contain defects of this kind.

We state the limitation openly: the constants are those of loopback, with a round-trip time of 23 µs, and not those of a wide-area network, and therefore a wide-area deployment requires its own two constants.

12.3 Bit-exactness as a gate on quoting cost

Finding. A cost comparison between routes computing different things is meaningless.

Consequently we check every backend against a common plaintext reference before any number is reported, and our benchmarks refuse to present a clean table when a cell disagrees. The encrypted convolutional network reproduces the plaintext prediction on 600 of 600 embedded medical images, and we ran it twice, before and after two optimizations, with both passes agreeing on all 600 images. It means that those optimizations are *exact* and not merely close.

12.4 Machine-checked security attestations

Every engine attests the security level that it actually implements, in a single auditable table, and asking a session for more than an engine attests is an *error not a silent downgrade*. We remark that exactly one of our engines attests malicious security. The editing rule is that an attestation records what the code *does*, never what the roadmap intends: raising one is the last step of an upgrade, after the cheating-detection tests pass.

We consider this the most transferable idea in the artifact. Security level is the dimension most often lost when numbers move between papers, and making it a machine-checked property of running code—rather than prose in a limitations section—is cheap to implement and hard to get wrong by accident.

12.5 Gating an external baseline the way we gate our own

Section 12.4 describes machine-checked attestations for our own engines: each states what it implements and asking for more is an error. When we adopted an external baseline (Section 13.2) we extended the same discipline to it, because the failure mode is documented not hypothetical. A community write-up of a homomorphic MNIST perceptron through HEIR reports roughly 30 s at a *security level of “not set”* and roughly six minutes at the secure ring dimension—a factor of twelve sitting between an insecure and a secure baseline. An insecure baseline is worse than no baseline: it manufactures a speed-up out of a parameter choice.

Our harness therefore refuses to report a latency unless three conditions hold: the run declares one of the standard security levels, with the *provenance* of that level recorded; the homomorphic evaluation reproduces the plaintext classification; and both arms computed the same function. The third is checked first and is the one we would not have thought to add: a mismatch between arms otherwise presents as a classification disagreement, which reads as approximation error. It caught a

real wiring fault of ours, in which the reference network used ReLU while the homomorphic arm was told to evaluate x^2.

12.6 Recording withdrawn results—and deleting the component

We keep negative, superseded and withdrawn results in the experimental record beside the successful ones, together with the reasoning that produced them, and we summarize what this practice caught in Section A.

We remark that the half of the rule which is easy to omit is the other one: **when a measurement condemns a component, the code is removed and the number is retained**. An artifact that accumulates every paradigm it has ever measured becomes unmaintainable, and worse, it becomes misleading, because a reader assumes that everything present is recommended. On this basis we removed the following components. Single-server private information retrieval, which loses to downloading the whole database by approximately 13× and admits *no* crossover in the database size N. The SEAL and CKKS backends, whose approximate comparison costs seconds per operation and carries a real error, against milliseconds and bit-exactness under secret sharing, and which carried 3.4 MB of vendored C++. An NTL polynomial backend, which our hand-written transform beat by 1.8× at every size. And an unsound placeholder for the proof system, which we describe in Section A.

Two consequences follow. The first is that **the figures given here for a removed component are not reproducible from the current artifact**, and our experimental record says so at the point of measurement. The second is that where a removed measurement still feeds a live claim, it survives as the smallest object that can carry it. In other words, the bootstrap counts attributed to a levelled competitor in Table 5 come from the level arithmetic of the CKKS sign approximation, which is now a module of forty lines with no dependencies, and whose figures are pinned by a test precisely because the apparatus that produced them has been deleted.

13 Cross-paradigm findings

In this section we present the findings that motivated the single-harness approach, in the sense that the literature of no single paradigm could have produced them. In Section 13.1 we show that the crossover between homomorphic encryption and secret sharing belongs to the preprocessing of the baseline. In Section 13.4 we locate the crossover between the two routes for the nonlinearities, and we report the four answers that we gave before the present one.

13.1 The FHE-versus-MPC crossover belongs to the baseline's preprocessing

The standard claim is that homomorphic encryption overtakes secret sharing beyond some width of the layer, because the preprocessing of secret sharing grows linearly whereas a packed ciphertext is flat. Our cost model predicts this behaviour and our measurements confirm it, but we remark that they confirm it *against a baseline preprocessed by oblivious transfer*. We give the figures in Table 15.

Against secret sharing preprocessed by **somewhat-homomorphic encryption**, the crossover disappears from this range entirely, and LowGear tracks the lower bound given by a trusted dealer to within 5 ms at width 64.

Finding. The honest statement is not "the hybrid beats dishonest-majority MPC past width 64" but "the hybrid beats *OT-preprocessed* dishonest-majority MPC past width 64, and loses to SHE-preprocessed MPC at every width measured." The mechanism survives—preprocessing volume decides it—but the boundary is a property of the baseline, not of either cryptosystem.

width	dealer (lower bound)	Semi2k (OT)	**LowGear (SHE)**	hybrid/BFV
64	600.6 ms	694.5 ms	**606.2** ms	692.1 ms
1024	603.0 ms	942.8 ms	**623.4** ms	692.1 ms
4096	610.9 ms	1.74 s	**678.4** ms	733.7 ms

Table 15: One private layer with both operands secret plus a ReLU; WAN, two parties, preprocessing charged per triple *consumed*.

A second boundary moves with the **threat model**. Under an honest majority, that is, with the ABY3 multiplication of Theorem 6.2 from correlated randomness and no preprocessing at all, the layer remains on secret sharing at *every* width. Under a dishonest majority with preprocessing from oblivious transfer, homomorphic encryption takes it beyond approximately 256 multiplications. We consider that this explains a structural fact about the literature which is otherwise puzzling, namely that *every hybrid inference system combining homomorphic encryption and secret sharing is two-party.* Gazelle, Delphi and Cheetah are situated precisely in the regime in which the model says that the hybrid pays.

13.2 MPC versus FHE at a matched threat model, against an external baseline

Every private-inference figure in §13.1 and in the wider literature we compare against shares a weakness: the homomorphic arm is *our* homomorphic encryption. §1 concedes that our absolute figures are not competitive with tuned implementations, and the natural objection to "secret sharing beat homomorphic encryption" is that it beat an implementation written by the same hands. This subsection removes that objection in the one direction that matters, and tightens the threat model at the same time.

The comparison is only meaningful at a matched corruption model. Single-server homomorphic encryption assumes one server and nothing whatever about non-collusion. A three-party honest majority assumes two of three parties remain honest and do not collude. A three-party system beating single-server homomorphic encryption on latency is therefore not a result: it is a consequence of assuming away the hard part, and it is why every hybrid inference system in the literature is two-party (§13.1). We therefore run the comparison under **two-party dishonest-majority** secret sharing, where one of two parties may be corrupt—the regime Gazelle, Delphi and Cheetah [40, 52, 37] occupy.

One asymmetry survives the matching and we do not measure it away. The two settings share a corruption model but not an *interaction* model: homomorphic encryption lets the client upload a ciphertext and go offline, whereas two-party computation needs it online for the whole evaluation and needs multi-gigabyte correlated randomness besides. Huang, Maeng and Suh [36] report that this is where the real monetary and energy cost of secret sharing lives, and nothing below contradicts them.

The external baseline. We drive HEIR [4]—Google's MLIR-based homomorphic-encryption compiler—to OpenFHE [1], so the homomorphic arm runs on tuned C++ not on our own number-theoretic transform. HEIR is compiler *infrastructure*: its paper claims generality and reports no private-machine-learning evaluation, so this is a baseline generator and not a target. Two properties make the resulting number quotable instead of decorative.

depth	shape	FHE eval	FHE+keygen	MPC warm	MPC cold	warm	cold
3	784-32-10	212.5 ms	322.9 ms	**20.4** ms	318.8 ms	10.4×	1.0×
5	784-32-32-10	762.2 ms	1,159.9 ms	**26.6** ms	454.2 ms	28.7×	2.6×
7	784-32-32-32-10	1,230.5 ms	1,681.4 ms	**31.2** ms	588.0 ms	39.5×	2.9×
9	784-32^4-10	1,775.5 ms	2,327.2 ms	**37.4** ms	723.2 ms	47.4×	3.2×
11	784-32^5-10	*segmentation fault in key generation*		42 ms	858 ms	—	—

Table 16: Two-party dishonest-majority secret sharing (LowGear, SHE preprocessing) against HEIR→OpenFHE CKKS at `HEStd_128_classic`, LAN. Same network, same weights, x^2 activation, every row verified against the plaintext reference before any timing is quoted. Depth is chosen by HEIR.

First, it is competitive with the published state of the art. Orion [29] reports MNIST at 0.23 s single-threaded; our harness obtains 0.21 s to 0.49 s for MNIST-shaped multilayer perceptrons through HEIR. Whatever else follows, it is not a comparison against a strawman.

Second, we refuse to quote a figure that fails any of three checks, each of which fired at least once during this work. **(i)** The security level must be a real one. HEIR's emitted C++ sets only the multiplicative depth and the plaintext modulus and never calls `SetSecurityLevel`, so the level in force is OpenFHE's default; we read it back out of the generated code and report it *with its provenance*, because "128-bit because someone chose it" and "128-bit because nobody chose anything" are different claims. **(ii)** The homomorphic arm must reproduce the plaintext classification, or a cheap approximation that gets the answer wrong would look fast. **(iii)** Both arms must compute the *same function*, checked before either of the other two, because a mismatch otherwise presents as a classification disagreement and is read as approximation error.

The activation is forced, and it costs the secret-sharing arm nothing. CKKS cannot compare, so an exact ReLU is not expressible on the homomorphic arm at all. The only activation both paradigms evaluate exactly is x^2, which is also what Orion uses for its MNIST networks. A worry about that choice turned out to be unfounded: x^2 costs the secret-sharing arm *exactly* what ReLU costs—identical rounds, multiplications, decompositions and bytes. Over plain integers x^2 would be one multiplication against ReLU's masked-opening bit decomposition, but in fixed point squaring doubles the scale, and the rescale that corrects it *is* the decomposition the comparison was. Two per neuron either way. So the switch does not flatter secret sharing, and every finding measured with ReLU carries over unchanged.

Result. Table 16 sweeps depth on MNIST-shaped networks. Both arms are split warm and cold: HEIR's key generation and our correlated randomness are both per-session, query-independent costs, so warm-against-warm and cold-against-cold are the two honest comparisons and we give both.

Finding. Warm against warm, secret sharing wins by 10.4× at depth 3 rising to 47.4× at depth 9. Cold against cold the two are a *dead tie* at depth 3 and secret sharing reaches 3.2× by depth 9. The direction—its advantage widening with depth—holds on both accountings; the magnitude depends entirely on which one is quoted.

Beyond depth 9 the homomorphic arm stops working. HEIR's default pipeline is *levelled* and places no bootstraps, so depth is a ceiling, not a step: at depth 11 OpenFHE segmentation-faults inside key generation, after HEIR's MLIR-to-C++ pipeline has succeeded. Secret sharing continues linearly, at eighteen additional rounds and roughly 5 ms per layer, unbroken.

Finding. Automatic bootstrap placement, which Orion [29] contributes and which HEIR's default pipeline lacks, is therefore not an optimisation but the thing that makes depth possible at all—at roughly 20 s per bootstrap [34]. That is where secret sharing wins on both axes at once, and it is the same discontinuity our no-bootstrap hybrid (§10.4) is built to avoid.

On the wide-area profile the ordering reverses: the homomorphic arm is non-interactive and network-insensitive, at 0.2 s to 1.8 s against 7 s to 10 s for secret sharing, which pays 110 to 164 rounds of latency. This is the shape Huang, Maeng and Suh [36] report, reproduced here in one harness with both arms measured and the computed function held identical.

What this comparison is not. These are multilayer perceptrons with x^2, not convolutional networks with polynomial ReLU, and the depths reached are those a levelled scheme handles without bootstrapping. Orion's 0.23 s is a different network. The result is the set of *ratios within one harness*; the absolute figures are offered only as evidence that neither arm is incompetent. §15 states the accounting asymmetry that remains.

13.3 Protocol assignment can be derived, not annotated

Given the measured costs of each backend, a search over the assignments of protocols to kinds of operation returns `mul` → fhe and `cmp-lt` → binary for an encrypted layer of width 256 containing a comparison. In other words, the search *derives* the architecture of Gazelle, Delphi and Cheetah instead of receiving it as an annotation. We remark that the prior work on the assignment of protocols, from HyCC [12] through Silph [16] to SING [7], is **restricted to secret sharing**, and that extending the cost model so that it spans secret sharing *and* homomorphic encryption is what makes the hybrid architecture an output of the search not an input to it.

Two structural non-linearities make this objective non-separable, which is worth noting for anyone tempted to formulate it as an integer linear program: a dependence level costs the *maximum* over its instructions, not their sum; and conversions *batch per level*, so a crossing's cost depends on whether another crossing already happens there.

13.4 The nonlinearity crossover, and three wrong answers on the way to it

We implement three interchangeable protocols for comparison behind a single interface, namely batched bit decomposition, garbled circuits, and the Π_{DReLU} of Falcon [62] based on `wrap3`, and we verify that the three compute the same function bit-exactly. In Table 17 we measure them with *both* protocols charged for their preprocessing and with neither of them paying for serialisation padding.

The mechanism appears in the same run. On the party-to-party transport Falcon sends 3.4 MB against the 10.1 MB of garbled circuits, which is 2.94× cheaper, and it does so in several hundred messages against 61, which is around 7× dearer. It means that the two terms of the cost model trade against each other, and that each boundary is located where the trade balances.

We must qualify the byte figure, because we quoted it for some time without the qualification and it changes how decisive that axis is. The party-to-party transport is not all the traffic: the hybrid also moves FHE ciphertexts and scheme-switch payloads on the ***evaluator link***, which does not pass through the instrumented party transport and so appeared in no counter, while the latency column paid its bandwidth cost throughout. Fitting a diagnosis's time against the reciprocal of bandwidth exposed the gap — the fit is clean and linear, with a slope near 29 MB where the traffic table said 10.1 MB — and the missing term is 17.0 MB per diagnosis, identical for both routes because it is the same pipeline either way. Total traffic is therefore 27.0 MB for garbled circuits against 20.4 MB for Falcon.

RTT	protocol	1000	100	40	30	20	10 Mbit s^{-1}
	gc	**558**	**2983**	**6669**	8,569	12,281	23,840
0 ms	falcon (cold)	2,588	4,529	7,259	8,646	**11483**	**20235**
	falcon (warm)	995	2,760	**5069**	**6359**	**8915**	**16552**
	gc	**2414**	**4671**	**8154**	**10067**	**13940**	**25421**
20 ms	falcon (cold)	9,142	10,818	13,488	14,964	17,732	26,428
	falcon (warm)	4,998	6,431	8,807	**10087**	**12681**	**19706**

Table 17: Milliseconds per encrypted MedMNIST diagnosis, median of three repeats; winner in bold. *Cold* charges Falcon for preprocessing on the critical path; *warm* generates it per MPC segment beforehand and reports the online cost. Garbled circuits have no query-independent phase to hoist, so their row serves both. Cold crossover between 20 and 30 Mbit s^{-1} at RTT 0 and below 10 Mbit s^{-1} at 20 ms; warm, between 100 and 1,000 Mbit s^{-1} and between 30 and 40 Mbit s^{-1}. Higher round-trip time moves each boundary *down*; hoisting preprocessing moves it sharply *up*. Cells within ≈13 % are unresolved (Section 15): at RTT 0 the 30 Mbit s^{-1} cold cell and the 100 Mbit s^{-1} warm cell are ties, so each boundary is a band.

Finding. Falcon's byte advantage is 1.33× on what the deployment actually moves, not the 2.94× the party transport alone reports — a common additive term of 17 MB dilutes it by 2.2×. Because the term is common, the crossover *location* is unchanged: adding the same quantity to both curves cannot move where they cross. What changes is the magnitude of the advantage, and hence how strongly bandwidth ought to be expected to favour one route.

We remark that pooling the preprocessing also *reduces* the messages of Falcon, because a single draw per segment batches better than many inline ones, and since its bytes are unchanged, warming the server improves both terms at once. Falcon's message count is no longer a fixed number, and we give it as a range above for that reason: closing the private-comparison leak of §6.5 replaced an opening of the mask with an opening of a masked product, and the rejection sampling that needs adds a data-dependent number of retries. Two consecutive runs of the identical configuration measured 474 and 482.

We reported three other answers before this one, and we consider the sequence to be the more useful result. Each of them was reproducible, each computed the correct diagnosis, and each agreed with our cost model; the first even matched a prediction of the break-even point at 17.1 against the 15 to 20 that we measured.

1. 15–20 **Mbit s^{-1}.** Falcon's Π_{WA} preprocessing was dealt from a shared seed, free precisely because it was not secret—so this compared Falcon's *online* cost with garbled circuits' *total*.

2. **No crossover at any bandwidth.** With a real offline phase Falcon measured 32.5 MB against 10.1 MB and appeared dearer in both terms. **82% of those bytes were padding**: Baru serialised ring shares as `u128` whatever their width.

3. **A tie.** The grid stopped at 10 Mbit s^{-1}, above the boundary.

We remark that the first answer was wrong in *both* directions at once, since dealt preprocessing flattered Falcon whereas the padding penalised it. Correcting both errors moves the boundary *upward*, and consequently Falcon wins over a wider region than we originally claimed, for entirely

different reasons. In other words, the two errors had partly cancelled, and this is why the figure appeared sound.

Neither the bit-exactness gate nor the agreement with the cost model detects any of the three errors. Both of those mechanisms establish that a computation is correct and that its cost is explicable, and neither of them asks whether two measurements are *commensurable*. What caught these errors was decomposing each total into the terms that the model charges, namely the phases, the messages, the bytes and the padded share of the bytes, and then verifying that both protocols were charged for the same things.

We now apply one further rule generally: a spot check is not evidence about an encoding. Narrowing the ring wire to a fixed width of 21 bits was exact for the dataset whose activations fit in 2^{20}, and it was wrong for *every* image of the dataset whose activations reach 2^{22}, because the safe width is a property of the model and not a constant.

What we can still say. The architectural claim survives and is worth separating from the performance one:

> **Finding.** Three implementations of one comparison, bit-exact against each other behind a single interface, make the protocol backing every ReLU, max-pool and argmax a one-line choice rather than a rewrite—which is what made this error detectable not shippable.

The per-deployment claim, withdrawn and then restored by measurement. We briefly concluded the opposite of what we had claimed: that there was *nothing* to choose on this workload, because garbled circuits dominated everywhere we had measured. That conclusion rested on a limitation and not on a result. `lt_falcon_batch` ran its preprocessing inline, so every figure in Table 17 was a *total*, and Falcon is a protocol designed to put work in a query-independent phase—charging it inline is charging it for the thing its design exists to avoid.

Building the split was the open work, and it is now built: preprocessing is generated per MPC segment from a plan a cold run *measures* (sizes cannot be derived from the layer shapes without duplicating the batching decisions inside max-pooling, the fused ReLU and argmax), and a pool that fails to serve a request is a hard error instead of a silent fallback, so an "online" latency cannot quietly contain preprocessing. The warm row of Table 17 is the result, and it restores the per-deployment claim in a stronger form: a *cold* server should prefer garbled circuits above $\approx$25 Mbit s^{-1}, a *warm* one should prefer Falcon almost everywhere at low round-trip time, and the gap between those two answers is a factor of four in bandwidth.

Falcon's own claim—that its *online* phase is cheap—is now testable too, and it holds. At $\ell = 32$ on Falcon's own LeNet workload our online phase is 0.25 MB per party against a 2.87 MB total: *below* the 0.74 MB they report, while our total is several times above it. Which comparison is the fair one depends on whether their figure includes preprocessing, which we cannot determine from the paper, so we publish both and call neither the answer.

Two asymmetries keep this honest. Garbled circuits gain nothing from warming—they have no query-independent phase here—so the warm boundary must be read *alongside* the cold one and never instead of it; presenting only the warm row would reproduce exactly the error that produced our first withdrawn crossover. And our byte counters are whole-run totals, so in a warm sweep the latency column is an online cost while the traffic figures remain totals; the two are not on the same footing and our tool says so in its own output.

The scope of the withdrawal. One bit decomposition per element is an artefact of *our* construction rather than a lower bound. Writing $w_{01} = \mathrm{wrap2}(x_0, x_1)$ and $y = (x_0 + x_1) \bmod L$,

one gets $\delta = w_{01} \oplus [\, y + x_2 \geq L \,]$ (verified exhaustively at $\ell = 6$). Party 0 holds both x_0 and x_1, so it computes w_{01} and y locally and for free, leaving a single comparison between a value it knows and a value parties 1 and 2 know—a millionaires' problem, not another Π_{WA}, and one that the same no-OT three-party garbling used for the `gc` route settles in one round and $\approx \ell$ AND gates. We have not implemented it and claim no cost for it. It is enough to fix the scope: what we withdraw is *our* crossover, and the correct reading of Table 17 is "Baru's Falcon loses to Baru's garbled circuits once preprocessing is charged"—not that Falcon's DReLU fails to repay its preprocessing, which these measurements do not show and the construction above suggests is false.

13.5 The convolution crossover, and a trick that did not need its scheme

The crossover of §13.4 is about the *nonlinear* surface. The linear surface has one too, and we did not know it, because our pipeline convolves homomorphically for a reason that was never contested: coefficient packing turns a whole layer into out_c ciphertext multiplications, while the obvious secret-shared convolution spends k^2 interactive multiplications per output pixel and loses without a hearing.

What reopened it is a result from outside. CHIP [63] packs channels into CKKS slots for batch CNN inference and, among other things, reports "an efficient 2-D convolution that halves the number of HE multiplications". That component is Winograd's minimal filtering algorithm [65, 48]: $F(2, 3)$ produces two outputs from a four-element window with four multiplications in place of six, and nesting it gives $F(2 \times 2, 3 \times 3)$ at 16 multiplications against 36.

Finding. The trick does not need the scheme it was published with. Its three transforms are $B^\top$ on the data, $A^\top$ on the result, and G on the filter; the first two have entries in $\{0, \pm 1\}$ and only G is fractional, with halves. Since G acts on the *weights*, scaling it by two per axis clears them, leaving the output exactly four times the true convolution — an exact power of two that a fixed-point rescale absorbs by shifting two places further. So Winograd is plaintext algebra over the ring, it is exact in integers, and it transfers to secret sharing unchanged. We adopted it without adopting CKKS.

We should say why that mattered to us specifically. We had removed our CKKS backends (§12.6) on the strength of their *comparison* cost, and a result whose benefit was inseparable from CKKS would have forced us to weigh re-adopting several megabytes of dependency against a convolution speedup. It is not inseparable, and establishing that was worth more to us than the speedup.

Adopting it also gave us the honest baseline the comparison needed. A secret-shared convolution spending k^2 multiplications per pixel is a straw man; one spending 16 per 2×2 tile is not. Table 18 therefore contests three strategies for one 3×3 layer. All three keep the model secret — the homomorphic route multiplies ciphertext by ciphertext against an encrypted model, so this is not a comparison against plaintext weights. All three end with *secret shares* of the output, which is the boundary the following ReLU needs and which the homomorphic route reaches only after a scheme switch. All three are checked against one plaintext reference.

Two things follow. The first is that ***the fast algorithm changes the assignment, not merely the cost***: at $16 \times 20 \times 20 \to 8$ the direct shared convolution sends 6,602 kB against the homomorphic route's 4,719 and loses, while the Winograd one sends 3,284 and wins. A layer that belonged to FHE now belongs to secret sharing. Its wall clock is 2.2× the direct route's, against a multiplication ratio of 2.25, which is as close a confirmation as this kind of measurement gives.

The second is that the boundary sits further toward secret sharing than we expected. The shared route dominates outright on the small layers, and where neither dominates the break-even

layer	winograd kB	fhe kB	winograd ms	fhe ms	byte win pays below
$2 \times 8 \times 8 \to 3$	**32**	1,769	**11**	31	shared dominates
$4 \times 12 \times 12 \to 4$	**172**	2,359	**21**	42	shared dominates
$8 \times 16 \times 16 \to 8$	**1060**	4,719	90	**69**	$1{,}424\,\mathrm{Mbit\,s^{-1}}$
$16 \times 20 \times 20 \to 8$	**3284**	4,719	231	**130**	$114\,\mathrm{Mbit\,s^{-1}}$
$16 \times 28 \times 28 \to 16$	12,634	**9437**	1,022	**332**	fhe dominates

Table 18: One 3×3 stride-1 convolution, three backends, all ending with shares. The direct secret-shared route is omitted for width; it is never better than the Winograd one, being the same computation with more multiplications. The last column is the bandwidth below which the shared route's smaller traffic outweighs its larger compute.

bandwidths are 1,424 and $114\,\mathrm{Mbit\,s^{-1}}$ — above most real links. Only the largest layer we measured goes to the homomorphic route on both axes.

We report this boundary with a correction attached, because our first version of it was wrong by a factor of twenty. The initial measurement put the shared route at 5,023 ms against 135 ms, and we drew the obvious conclusion: the byte advantage was real and irrelevant, since no plausible link would repay a 37× compute penalty. The penalty was ours. Our harness opened all 2,592 outputs of the shared route *one at a time* as verification scaffolding, while the homomorphic route decrypts an entire channel per call — and in a deployment the shared route's output is never opened at all, since it passes to the next layer as shares. Charging it for those openings, while charging the homomorphic route for a per-session key generation it amortises over every layer of every query, was two asymmetries pointing in opposite directions, each of which looked like the natural thing to write. Removing both moved the shared route to 236 ms and the gap to 1.8×: 95 % of what we had reported as protocol cost was scaffolding.

We draw the general point not the local one. Bit-exactness gated this comparison and passed it — every route computed the right convolution throughout — because what differed was not the computation but *where each route's measurement began and ended*. That boundary is invisible to a correctness check and is not a property of either protocol, which is precisely why it has to be stated for each route not assumed to be shared. Our benchmark now derives its summary sentence from the measured rows instead of asserting one beside them, because we wrote that sentence wrongly three times — "bytes decide", then "compute decides", then "the homomorphic route is faster everywhere" — and each was true of the table in front of us at the time.

14 Case study: private medical image diagnosis

In this section we give a case study. MedMNIST v2 [66] distributes twelve biomedical datasets at 28×28, which is exactly the shape of MNIST [49], and consequently a pipeline for private inference runs on them without modification and our artifact ceases to be a statement about handwritten digits. We give the results in Table 19.

We consider that three points deserve the attention of a privacy venue.

Privacy costs nothing in accuracy, and we show this rather than asserting it. The integer model matches its floating-point parent to within 0.2 points, and the encrypted pipeline reproduces the plaintext diagnosis bit-exactly on every one of the 600 images.

	PneumoniaMNIST	OrganAMNIST	DermaMNIST
task	chest X-ray, 2 classes	abdominal CT, 11 organs	dermatoscopy, 7 lesions
input	$1 \times 28 \times 28$	$1 \times 28 \times 28$	$\mathbf{3 \times 28 \times 28}$ (RGB)
integer accuracy, full test set	84.78 % (624)	**82.93** % (17,778)	72.12 % (2,005)
95 % Wilson CI [64]	[81.7, 87.4]	[82.4, 83.5]	[70.1, 74.0]
balanced accuracy / AUC	80.64 % / **0.9314**	**82.21** % / **0.9605**	30.14 % / **0.8519**
sensitivity / specificity	97.18 % / **64.10** %	—	—
published ResNet-18 (28×28)	0.854 / 0.944	0.935 / 0.997	0.735 / 0.917
majority-class baseline	62.50 %	18.48 %	**66.88** %
encrypted = plaintext	**300/300**	**300/300**	**300/300**
FHE bootstraps	0	0	0
online latency / diagnosis	**268** ms	291 ms	271 ms

Table 19: Private diagnosis on real biomedical images, image *and* weights encrypted, only the class index revealed. DermaMNIST is read on balanced accuracy and AUC: its largest class is 66.88 % of the test set, which published ResNet-18 beats by under seven points. Its RGB input costs nothing extra—an RGB 28×28 convolution needs degree 3,977 against $n = 4096$ and so fits one ciphertext at one multiplication per output channel, which is why its latency matches the grayscale rows.

Accuracy alone misreads a medical classifier. The test set of PneumoniaMNIST is approximately 60% positive, and our model earns its 84.78 % largely by *over-calling the disease*, since it attains 97.2 % of sensitivity against 64.1 % of specificity, and its balanced accuracy is 80.64 %. None of this is visible while only the accuracy is reported, and we remark that none of it is a cryptographic result; nevertheless a paper that claims private diagnosis is obliged to report it.

The ceiling on accuracy is the packing and not the cryptography. We initially attributed the size of the model to the noise budget of the homomorphic scheme, and that was wrong. Three limits bind, and the noise is the third of them. Spatially, a valid convolution of size 3×3 followed by pooling takes $28 \to 13 \to 5 \to 1$, which admits at most three convolutions. By *packing*, the constraint $2 \cdot \mathrm{in}_c \cdot H \cdot W \lesssim n$ caps the first convolution at 12 channels, before the input of the second one exceeds $n = 4096$. Depth, in contrast, really is free, for the reason given in Section 10.4.

We have since lifted that limit on packing without enlarging the ring. A convolution is linear in its input channels, and therefore a layer that is too wide for a single ciphertext can be evaluated as equal groups of channels whose products are *added*, which is free homomorphically and requires neither an evaluation key nor any depth. We remark that doubling the ring to $n = 8192$ would also have worked, and that it would have charged every operation of the pipeline, whereas splitting the channels charges only the layers that overrun.

It buys accuracy on one of the two datasets, and the difference is instructive. Retraining both at conv(16, 32)—whose second convolution takes 16 channels at 13×13, degree 5,266 against $n = 4096$, hence two groups of 12—moves organa from 81.64 % [81.1, 82.2] to **82.93** % [82.4, 83.5], disjoint intervals on 17,778 test images. Pneumonia does not move: 84.78 % to 83.97 % with overlapping intervals, its validation accuracy rising while its test accuracy falls. Both remain bit-exact with zero bootstraps.

So "the packing was the accuracy ceiling" holds *conditionally*, and the condition is data: organa's 34,561 training images can use the extra capacity, pneumonia's 4,708 overfit it. Either experiment alone would have been misleading.

15 Threats to validity

In this section we state the threats to the validity of our results.

The two arms of Section 13.2 are not timed the same way. This is the sharpest threat to that comparison and we state it plainly. The homomorphic arm's latency is elapsed wall clock. The secret-sharing arm's is composed: round latency and bandwidth come from measured counts put through the network model, and local compute is measured but *divided by the party count*, because our virtual machine runs every party serially in one process. The protocols are symmetric, so the division is defensible, but it is a division and not a measurement of one party. Two consequences follow. On the wide-area profile, where round latency dominates by two orders of magnitude, the composition is harmless. On a local-area network it is the whole number, and the local-area figures should be read as accurate to a factor not to a percentage. A genuinely socketed two-party arm, with parties in separate processes, is the experiment that would remove this threat, and we have not run it.

Omitting local compute is an error we made and corrected. The local-area figures of Section 13.2 originally charged secret sharing only round latency and bandwidth, with no compute term at all—inherited from a harness that reported only wide-area numbers, where the omission is invisible. At 784-32-10 the modelled 16 ms sat against 598 ms of measured compute, so the local-area figures understated secret sharing by roughly 19×, and the ratios we first computed additionally compared a *warm* secret-sharing arm against a *cold* homomorphic one. Both halves flattered secret sharing. Compute is now measured and split warm from cold on both sides. We record the error because it is the third instance in this work of a first metric flattering the paradigm we implemented—correlated randomness charged as free, compute charged as free, and warm compared against cold—and the common cause is not a coding mistake but a habit: asking whether a number reproduces rather than what it omits.

Quality of the implementation. Our code is research Rust and not tuned C++. Consequently the absolute figures are not competitive, and the ratios measured inside the harness are the defensible output. Where we compare against a published number, as for instance our sign extraction at approximately 0.043 ms against the optimized C++ of Falcon, the difference is one of implementation and not one of protocol, and we say so.

A single machine. All of our figures were obtained on one Apple M1 Pro, and therefore the variance across machines is uncharacterized. The network model is validated on loopback, and consequently the constants for a wide-area network are modelled rather than measured.

Run-to-run variance bounds how sharply a boundary can be read. Re-running an *unchanged* configuration as a separate process moved one cell by 13 %. Repeats *within* one measurement session spread under 2 %, but they share a key-generation session, run back to back and hit warm caches, so they bound jitter not reproducibility—our sweep reports the median of three and marks cells whose ranges overlap, and that marking is therefore weaker than it looks. We read every crossover as good to about one grid step, and treat cells within ≈13 % of each other as unresolved regardless of which is nominally faster. Latencies carry this variance; message and byte counts are deterministic and reproduce exactly, which is why we prefer them for any claim that has to be sharp.

The simulated network is a model. We validated it to 13.6 % in the worst case, but it remains the expression rounds × RTT + bytes/bandwidth and not a real wide-area network with congestion, jitter or loss of packets.

The disclosed cryptographic gaps, one of which has since closed. The threshold *key generation* of our homomorphic layer was dealer-based, and we recorded it on the code itself and in the attestation table rather than only in prose. It no longer is, and the reason the dealer was there turns out to be worth more than its removal.

A dealer makes the key components ***uniform*** modulo q while their sum stays ***ternary***, by choosing $s_2 = s - s_0 - s_1$. Both properties at once are what it buys, and no protocol in which each party samples its own component independently can have all three of independent sampling, a small sum, and uniform components. Sampling each s_i uniformly makes s uniform modulo q, and BFV decryption computes $c_0 + c_1 s = \Delta m + (e_1 + e_2 s - eu)$, whose $e_2 s$ term then exceeds $q/2t$: decryption fails outright instead of degrading.

Our key generation takes the other horn. Each party samples s_i ternary and sends it to its predecessor, so the replicated layout (s_i, s_{i+1}) survives — deliberately, because replication is what makes a partial decryption checkable by recompute-and-compare, and a protocol producing only additive shares would remove the dealer at the cost of the malicious-security argument. A single corruption then learns two of the three components, hence $k = s_i + s_{i+1}$, and that $s - k$ is ternary. That is not the weakness it appears to be, because the adversary's residual problem is the one the scheme already rests on:

$$b + ak \;=\; -(a(s-k) + e),$$

an RLWE sample whose secret $s - k$ is ternary. We cannot test hardness, but we can and do test the identity that the reduction rests on — the measured residual has sup-norm exactly 3, the sum of three ternary errors — so a broken reduction would be visibly and not quietly vacuous. The cost is that $\|s\|_\infty \le 3$ rather than 1, about 1.6 bits of noise budget for a fresh ciphertext and 3.2 for a product, which we report and do not assume, because it competes with the smudging budget.

Finding. The expensive part of threshold FHE does not arise here at all. A product decrypts under s^2 because we relinearize nothing, and under replicated sharing each party forms its own share of s^2 *with no communication*: writing $s^2 = \sum_i s_i^2 + 2(s_0 s_1 + s_1 s_2 + s_2 s_0)$, party i holds (s_i, s_{i+1}) and computes $s_i^2 + 2 s_i s_{i+1}$, and the three sum to exactly s^2. It is the same identity replicated multiplication uses, which is no coincidence — it is the same sharing. Distributed generation of a relinearization key, the costly step in threshold FHE, is therefore not deferred but unnecessary, and that is a dividend of the depth-1 design rather than of the key generation.

We measured the cost of using it before proposing to, and we separate two figures that are easy to conflate. Against one local decryption, a threshold decryption is 3.5× in *total work*, which is the throughput cost; the *latency* cost is one party's partial decryption at 1.18×, plus one communication round, because the three partials are computed concurrently on three machines. Quoting the first as a latency figure would repeat the error this paper records elsewhere, so the artifact prints both with that warning and asserts that one party performs between a fifth and a half of the total, which is the premise the parallelism argument needs.

What the input-boundary proof buys, and what it costs. The boundary is the one place where replication cannot help and zero knowledge is unavoidable, and we can now say concretely why. Adding half a Δ to a single coefficient of an input ciphertext — the mildest tamper that does anything at all — turns a computed result of 42 into 43, and two honest co-holders recomputing

the evaluation *agree* on the 43, because the evaluation was performed correctly. The fault is in the input, which no amount of recomputation inspects. We draw attention to the off-by-one: a malformed input need not produce a value that a sanity check would reject, and an adversary with more room than half a Δ can choose noise large enough that whether the decryption wraps depends on the model weights, which turns a correctness failure at the boundary into a channel that leaks what the protocol exists to protect.

The proof itself is implemented: a Schnorr-style Σ-protocol over the ring, made non-interactive by Fiat–Shamir, with Lyubashevsky rejection sampling not noise flooding, so that a released response is *perfectly* hiding rather than statistically so. The interesting part is what it costs, and it is not the prover time.

Finding. Soundness slack at the boundary is paid for out of the smudging budget, bit for bit. A Σ-protocol extracts a witness whose norm exceeds the honest one by a slack factor, so a verifier accepting the proof is accepting inputs with noise up to $2^{\text{slack}} \cdot B$; that noise grows through the evaluation and arrives at the threshold decryption, where the flooding that hides the key share must cover it. Our measured 26-bit slack leaves $\lambda = 30$ at the largest modulus this implementation can represent, against the 40 that an honest-input analysis reports. The two analyses were not previously on the same footing, and the gap is entirely this term.

That is why the artifact's attestation for this scheme stays semi-honest: not because the proof is missing, but because it is a tested instead of reviewed implementation, and because $\lambda = 30$ is below what we are willing to call a malicious-security claim. Both are settleable — a tighter proof, or a wider modulus than a `u128` representation admits — and neither is settled here.

What remains open. Two things, neither cryptographic in the sense above. The first is the review and parameter question just described. The second is harness structure: our benchmark evaluates the homomorphic half single-threaded with one decryption key, because simulating one evaluator is cheaper than simulating three, so the reported figures do not yet include the threshold path even though the protocol no longer requires a party that can decrypt.

The Falcon backend's preprocessing used to be the second such gap—dealt from a common seed, so all three parties could reconstruct the mask. It is now generated by bit composition, at a cost of one bit decomposition per element offline. That cost is what the two Falcon rows of Table 17 separate: the cold row pays it on the critical path, the warm row hoists it. An earlier draft of this section said Table 17 measured Falcon's *online* cost while another paragraph said every figure in it was a *total*. Both statements were in the paper at once; the table now has a row for each and says which is which.

A third gap, found while building the pure Falcon route, and closed. Falcon's exact truncation *opens* $c = a + r$. Its stated precondition is that $a + r$ must not wrap, which is a *ceiling* on the mask, and our first Π_{ReLU} satisfied it the cheapest way, with r no wider than the value. That is correct and hides almost nothing: the posterior on a given c collapses to a point at $c = 0$, and for uniform a the leak is $1/(2 \ln 2) \approx 0.72$ bits per truncation, where the batched-bit-decomposition route it replaced opens no function of a secret at all. Making the route "purer" had therefore traded away a security property.

No correctness test could have caught it—leakage is not a wrong answer—and all three tests covering truncation preprocessing happened to pass a mask narrower than the ring, so none ever exercised a wrapping one. The fix is not statistical slack but a *wrap bit*: let r cover the whole ring, making the opened value uniform and *perfectly* independent of a, then correct the resulting overflow with $[c < r]$, which is provably the overflow indicator and is computable from a public c and a

secret r for one narrowed `a2b` and one Π_{SS} lift. The alternative—κ bits of statistical slack—needs a ring wider than our narrowed wire encoding permits and measured $1.9\times$ *dearer* in bytes than bit decomposition. The narrow-mask request is now unrepresentable and not merely documented, and the wrap identity is verified exhaustively over all (a, r) at $\ell = 10$.

A fourth gap, found by attempting the composability proof, and closed. Π_{PC} opens $m \cdot \prod_i c_i$ and reads only whether the result is zero. The c_i carry no randomness of their own — they are determined by the compared bits, the public r and the secret β — so the unit m is the only thing between the transcript and the product, and the comment on our own implementation said exactly that. Our routine for sampling m nevertheless *opened* each candidate in order to reject zeros, and returned the same shares. The justification attached to it read that opening whether a mask is zero leaks nothing, because m is drawn independently of any input and the sampling runs offline. The premise is true and the conclusion does not follow: what the code opened was m itself, and the purpose of m is to blind a *later* opening, so a public m blinds nothing.

The consequence is measurable not hypothetical. With m known, $\prod_i c_i = d/m$ is public, and at $\ell = 6$ a single predicate class contains 31 distinct products, so the opening carried $\log_2 31 \approx 5$ bits where the predicate is entitled to one. In Π_{WA} the compared secret is the wrap mask x while $r = a + x$ is public, and therefore those are bits of the activation.

The repair is to test the mask without opening it: draw a second uniform s and open $d = m \cdot s$. Then $d \neq 0$ implies $m \neq 0$, and conditioned on any accepting d the mask is still uniform on $\mathbb{Z}_p^\star$, because each candidate admits exactly one witness. We verify the counting exhaustively over $\mathbb{Z}_p \times \mathbb{Z}_p$ not statistically. The cost is one extra random share and one extra batched multiplication per attempt, both offline, and the per-attempt rejection rate rises from $1/p$ to $2/p - 1/p^2$ because either factor may vanish.

We record two things about how this was found. It was not found by testing — every correctness test passed before and after, since leakage is not a wrong answer, and the protocol computed the right comparison throughout. It was found by writing the enumeration that Lemma 6.1 needed, which forced the question "what distribution does the simulator sample this opening from?" at a site where the answer turned out to depend on a secret. Together with the truncation of Section 15 that is two of our nine openings repaired by the same question, which we take as the strongest argument we can offer for stating a composability claim in a form precise enough to be checked.

Breadth of the workload. Our findings are anchored on inference in feed-forward convolutional networks at 28×28. We do not cover transformers, larger images, or workloads other than inference.

This is not an adversarial evaluation. Our tests for the detection of cheating drive real attacks over the transport against the malicious layer, but we remark that this work is not a security audit, and that none of our formal proofs is machine-checked.

16 Open problems

In this section we list the problems that remain open, in the order in which we consider them tractable.

1. **A private convolutional network at the scale of CIFAR** requires SIMD packing in slots, since coefficient packing bounds the size of the image. We remark that the story of composition and security is independent of this choice.

2. **Transformer nonlinearities** (softmax, LayerNorm) need exponential, reciprocal and inverse-square-root subprotocols.

3. **Malicious security at the boundary with homomorphic encryption.** Replication cannot catch a *malformed input ciphertext*, because both honest co-holders compute on it faithfully and therefore agree. It means that the positioning becomes sharper: a zero-knowledge proof is needed *only at the input boundary and never for the evaluation*, whereas verifiable homomorphic encryption on a single server needs one over the whole computation. We have since implemented the boundary proof, that is, a proof that a ciphertext is a well-formed encryption with bounded noise, and what remains open is a parameter question instead of a construction: the proof's soundness slack is subtracted from the smudging budget bit for bit, so the measured 26-bit slack leaves $\lambda = 30$ at the widest modulus a `u128` representation admits, against the 40 an honest-input analysis reports. Either a tighter proof or a wider modulus recovers it. Composing the proof with the malicious secret-sharing layer, so that a single protocol carries both guarantees, remains open alongside it.

4. **Secret-index array access**, priced by the cost model, before ORAM. CirC [55] implements exactly the two cases either side of that line and does not implement ORAM at all.

5. **Per-instruction protocol assignment**, versus the current per-operation-kind granularity.

6. **Cross-machine and real-WAN validation** of the cost model's constants.

17 Conclusion

Cross-paradigm claims about performance in privacy-preserving computation are not at present falsifiable, because the numbers come from sources that cannot be compared. In this survey we have described a harness in which such claims become falsifiable, and we have reported what changes when the paradigms are re-measured under a uniform accounting.

Three of those changes are worth restating. The crossover between homomorphic encryption and secret sharing belongs to the preprocessing of the baseline not to either cryptosystem. Active security costs bytes and not rounds. And the boundary between the comparison protocols lies between 20 and 30 $\mathrm{Mbit\,s^{-1}}$ for a server that pays for its preprocessing on the critical path, and above 100 $\mathrm{Mbit\,s^{-1}}$ for a server that has generated it in advance, which is a difference of a factor of four in bandwidth between two deployments of the same protocol. We mislocated that boundary four times before locating it, and each of the four figures was reproducible and agreed with our own cost model. In Section 13.4 we report that sequence and not only its endpoint, because the failure mode which it illustrates, namely two measurements that are individually sound and mutually incommensurable, is not caught by any check that we had.

We consider that the transferable part of this work is not the artifact but the discipline: identical transports for both legs of any comparison, a cost model checked against a real socket, bit-exactness as a precondition for quoting a cost, and the security level as a machine-checked property of running code not as as a sentence of prose. Each of these is cheap to adopt, and each of them caught something here that would otherwise have been published.

A Experimental report: what the methodology caught

This appendix records, in more detail than the body requires, the errors our own methodology detected in our own work, and what each cost. It is placed here and not in the body because its

interest is methodological, not scientific: the results of the survey do not depend on it, and a reader concerned only with those results may stop at Section 16. We include it because the discipline described in Section 12 is only credible if what it caught is on the record, and because several of the errors are, we believe, ones a reader of this literature is likely to make.

In this section we record what the methodology of Section 12 caught in our own work. We consider that the credibility of a study rests partly on what its authors were willing to withdraw, and consequently each item below is recorded in the artifact not quietly fixed.

A backend for proofs that proved nothing. Until 2026-07-31 the `verify` routine of the shipped backend never checked that a witness satisfied any constraint, and it means that anybody holding the public circuit could forge an accepting proof. We retain the routine, unused, so that a soundness test can keep demonstrating precisely what it failed to do. **Every figure for the proof system that predates that date is withdrawn.**

A break of privacy in bit decomposition. Our previous `a2b` masked its input with a value derived from a *public* lockstep counter, and therefore the input was recoverable from the transcript by any observer. Everything built upon it, that is, ReLU, max-pooling and argmax, leaked its activations. We remark that the advertised property of requiring no extra communication per bit was in fact a *symptom*, since the carry chain was free precisely because every operand was already public. Repairing the protocol cost 4.4× in time and approximately 130× in bandwidth, and we later recovered both by means of the triple-free multiplication of Theorem 6.2 and of batching.

An insecure homomorphic scheme, removed not caveated. Our fast engine was a port of a scale-invariant NTRU construction [51, 8]. An analysis showed it to be insecure at the parameters that our construction requires, even at depth one [3, 43, 28]: the large LTV secret raises the fatigue point to approximately 2^{59}, but a single multiplication with a usable plaintext modulus t forces $q \gtrsim 2^{84}$. We deleted it rather than keeping it as a fast reference, every result transferred bit-exactly, and the homomorphic column of our tables became slower.

A channel of selective failure that passes every functional test. Attaching MACs to the leaves of a bit decomposition by multiplication is unsound. A corrupt party injects an offset δ, the first gate downstream then carries the MAC error δy where y is a secret bit, and consequently the execution aborts when $y = 1$ and completes when $y = 0$. It means that the abort itself carries one bit of the secret.

> **Finding.** Deferring the abort does not close this. Batching hides *when* an abort happens, not *whether* it happens; those are different channels and only the first is closed by batching. We had asserted the opposite before building it.

A crossover withdrawn three times, for three different reasons. The location in Section 13.4 has been wrong three ways: measured against dealt (hence free) preprocessing, so it compared one protocol's online cost with another's total; then measured with Falcon inflated 3.2× by serialisation padding, from which we wrongly inferred that no crossover could exist anywhere; then reported as a tie because the grid stopped above the boundary. It is at 2–4 Mbit s^{-1}. **All three passed our bit-exactness gate and agreed with our cost model**, the first even matching its break-even prediction. A related lesson we now apply: a two-image spot check is not evidence about a wire format—narrowing to 21 bits was exact for the 2^{20} dataset and wrong for *every* image of the 2^{22} one.

Comparison errors of framing. Pricing FHE's *linear* part against MPC's *whole layer* (omitting the ReLU that FHE cannot do at all); computing a crossover against a trusted-dealer baseline (asking whether FHE beats free preprocessing); and the cross-domain triple comparison corrected in Section 10.6.

Honest negatives retained. PIR loses to downloading the database by ≈13× with no crossover in N. Hybrid-TFHE does not pay. Faster ring arithmetic does *not* bring FHE close to MPC. Per-layer bit-width tuning was measured at ≈10% and not built. A fused garbled max-of-four was measured as a 1.68× *loss* and not built.

Reproducibility

Single Rust crate, no GPU. `cargo test` covers correctness; every `bench-*` subcommand emits `-json`. All numbers are produced by these commands; none are hand-entered.

Result	Command
Deep inference, zero bootstraps	`bench-deep-nn -depths 2,4,8,16,32`
Private diagnosis (Table 19)	`bench-medmnist -dataset pneumonia`
Nonlinearity comparison (Table 17, withdrawn)	`bench-medmnist -sweep`
Cost of active security (Table 4)	`bench-malicious -n 32 -bits 32`
Triples: SHE vs OT (Table 7)	`bench-triples`
FHE↔MPC crossover (Table 15)	`bench-layer`
Cost model vs real socket (Section 12.2)	`bench-net`
Proving a quantized neuron (Table 6)	`bench-zkp-layer -widths 1,...,784`
Falcon vs `a2b` vs garbled	`bench-falcon`

References

[1] Ahmad Al Badawi et al. OpenFHE: Open-source fully homomorphic encryption library, 2022. `https://github.com/openfheorg/openfhe-development`.

[2] Martin Albrecht et al. Homomorphic encryption security standard. HomomorphicEncryption.org, Toronto, Canada, 2018.

[3] Martin R. Albrecht, Shi Bai, and Léo Ducas. A subfield lattice attack on overstretched NTRU assumptions. In *CRYPTO*, pages 153–178, 2016.

[4] Asra Ali, Jaeho Choi, Bryant Gipson, Shruthi Gorantala, Jeremy Kun, Wouter Legiest, Lawrence Lim, Alexander Viand, Meron Zerihun Demissie, and Hongren Zheng. HEIR: A universal compiler for homomorphic encryption, 2025. `https://github.com/google/heir`.

[5] Gilad Asharov, Abhishek Jain, Adriana López-Alt, Eran Tromer, Vinod Vaikuntanathan, and Daniel Wichs. Multiparty computation with low communication, computation and interaction via threshold FHE. In *EUROCRYPT*, pages 483–501, 2012.

[6] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *CRYPTO*, pages 420–432, 1991.

[7] Blüml, Huppert, Khayata, Schmidt, and Schneider. SING: Hybrid MPC protocol assignment with graph neural networks. Cryptology ePrint Archive, Paper 2026/1237; to appear, USENIX Security 2026, 2026. `https://eprint.iacr.org/2026/1237`.

[8] Joppe W. Bos, Kristin Lauter, Jake Loftus, and Michael Naehrig. Improved security for a ring-based fully homomorphic encryption scheme. In *IMA International Conference on Cryptography and Coding*, pages 45–64, 2013.

[9] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. CHIMERA: Combining ring-LWE-based fully homomorphic encryption schemes. volume 14, pages 316–338, 2020.

[10] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory*, 6(3), 2014.

[11] Andreas Brüggemann and Thomas Schneider. When trying to catch cheaters breaks the MPC: Breaking and fixing delayed consistency checks in trident, fantastic four, SWIFT, and quad. In *Advances in Cryptology — ASIACRYPT*, 2026. Cryptology ePrint Archive, Paper 2026/234.

[12] Niklas Büscher, Daniel Demmler, Stefan Katzenbeisser, David Kretzmer, and Thomas Schneider. HyCC: Compilation of hybrid protocols for practical secure computation. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 847–861, 2018.

[13] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 136–145, 2001.

[14] Harsh Chaudhari, Ashish Choudhury, Arpita Patra, and Ajith Suresh. ASTRA: High throughput 3pc over rings with application to secure prediction. In *ACM CCSW*, pages 81–92, 2019.

[15] Harsh Chaudhari, Rahul Rachuri, and Ajith Suresh. Trident: Efficient 4PC framework for privacy preserving machine learning. In *Network and Distributed System Security Symposium (NDSS)*, 2020.

[16] Edward Chen, Jinhao Zhu, Alex Ozdemir, Riad S. Wahby, Fraser Brown, and Wenting Zheng. Silph: A framework for scalable and accurate generation of hybrid MPC protocols. In *IEEE Symposium on Security and Privacy (S&P)*, pages 848–863, 2023.

[17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *ASIACRYPT*, pages 409–437, 2017.

[18] Koji Chida, Daniel Genkin, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Yehuda Lindell, and Ariel Nof. Fast large-scale honest-majority MPC for malicious adversaries. In *Advances in Cryptology – CRYPTO 2018*, pages 34–64, 2018.

[19] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.

[20] Tung Chou and Claudio Orlandi. The simplest protocol for oblivious transfer. In *Progress in Cryptology – LATINCRYPT 2015*, pages 40–58, 2015.

[21] Ronald Cramer, Ivan Damgård, Daniel Escudero, Peter Scholl, and Chaoping Xing. SPDZ_{2^k}: Efficient MPC mod 2^k for dishonest majority. In *Advances in Cryptology – CRYPTO 2018*, pages 769–798, 2018.

[22] Ronald Cramer, Ivan Damgård, and Yuval Ishai. Share conversion, pseudorandom secret-sharing and applications to secure computation. In *Theory of Cryptography (TCC)*, pages 342–362, 2005.

[23] Anders Dalskov, Daniel Escudero, and Marcel Keller. Fantastic four: Honest-majority four-party secure computation with malicious security. In *USENIX Security Symposium*, pages 2183–2200, 2021.

[24] Ivan Damgård and Jesper Buus Nielsen. Universally composable efficient multiparty computation from threshold homomorphic encryption. In *CRYPTO*, pages 247–264, 2003.

[25] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *CRYPTO*, pages 643–662, 2012.

[26] Daniel Demmler, Thomas Schneider, and Michael Zohner. ABY – a framework for efficient mixed-protocol secure two-party computation. In *Network and Distributed System Security Symposium (NDSS)*, 2015.

[27] Benjamin E. Diamond and Jim Posen. Succinct arguments over towers of binary fields. Cryptology ePrint Archive, Paper 2023/1784, 2023. `https://eprint.iacr.org/2023/1784`.

[28] Léo Ducas and Wessel P. J. van Woerden. NTRU fatigue: How stretched is overstretched? In *ASIACRYPT*, pages 3–32, 2021.

[29] Austin Ebel, Karthik Garimella, and Brandon Reagen. Orion: A fully homomorphic encryption framework for deep learning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, volume 2, pages 734–749, 2025.

[30] Hendrik Eerikson, Marcel Keller, Claudio Orlandi, Pille Pullonen, Joonas Puura, and Mark Simkin. Use your brain! arithmetic 3PC for any modulus with active security. In *1st Conference on Information-Theoretic Cryptography (ITC)*, pages 5:1–5:24, 2020.

[31] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In *Advances in Cryptology — CRYPTO*, pages 823–852, 2020.

[32] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Paper 2012/144, 2012. `https://eprint.iacr.org/2012/144`.

[33] Matthias Felleisen and Daniel P. Friedman. Control operators, the SECD-machine, and the λ-calculus. In *Formal Description of Programming Concepts III*, pages 193–217. North-Holland, 1986.

[34] Karthik Garimella, Austin Ebel, Gabrielle De Micheli, and Brandon Reagen. HE-LRM: Encrypted deep learning recommendation models using fully homomorphic encryption, 2025.

[35] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *41st ACM Symposium on Theory of Computing (STOC)*, pages 169–178, 2009.

[36] Pengzhi Huang, Kiwan Maeng, and G. Edward Suh. Beyond latency: A system-level characterization of MPC and FHE for privacy-preserving machine learning, 2026.

[37] Zhicong Huang, Wen jie Lu, Cheng Hong, and Jiansheng Ding. Cheetah: Lean and fast secure two-party deep neural network inference. In *31st USENIX Security Symposium*, pages 809–826, 2022.

[38] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. Extending oblivious transfers efficiently. In *Advances in Cryptology – CRYPTO 2003*, pages 145–161, 2003.

[39] Wen jie Lu, Zhicong Huang, Cheng Hong, Yiping Ma, and Hunter Qu. PEGASUS: Bridging polynomial and non-polynomial evaluations in homomorphic encryption. In *IEEE Symposium on Security and Privacy (S&P)*, pages 1057–1073, 2021.

[40] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. GAZELLE: A low latency framework for secure neural network inference. In *27th USENIX Security Symposium*, pages 1651–1669, 2018.

[41] Marcel Keller. MP-SPDZ: A versatile framework for multi-party computation. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1575–1590, 2020.

[42] Marcel Keller, Valerio Pastro, and Dragos Rotaru. Overdrive: Making SPDZ great again. In *Advances in Cryptology – EUROCRYPT 2018*, pages 158–189, 2018.

[43] Paul Kirchner and Pierre-Alain Fouque. Revisiting lattice attacks on overstretched NTRU parameters. In *EUROCRYPT*, pages 3–26, 2017.

[44] Peter M. Kogge and Harold S. Stone. A parallel algorithm for the efficient solution of a general class of recurrence equations. *IEEE Transactions on Computers*, C-22(8):786–793, 1973.

[45] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In *35th International Colloquium on Automata, Languages and Programming (ICALP)*, pages 486–498, 2008.

[46] Nishat Koti, Mahak Pancholi, Arpita Patra, and Ajith Suresh. SWIFT: Super-fast and robust privacy-preserving machine learning. In *USENIX Security Symposium*, pages 2651–2668, 2021.

[47] Aleksandar Krastev, Nikola Samardzic, Simon Langowski, Srinivas Devadas, and Daniel Sanchez. A tensor compiler with automatic data packing for simple and efficient fully homomorphic encryption. In *Proceedings of the ACM on Programming Languages (PLDI)*, volume 8, pages 126–150, 2024.

[48] Andrew Lavin and Scott Gray. Fast algorithms for convolutional neural networks. In *CVPR*, 2016.

[49] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[50] Jian Liu, Mika Juuti, Yao Lu, and N. Asokan. Oblivious neural network predictions via MiniONN transformations. In *ACM CCS*, pages 619–631, 2017.

[51] Adriana López-Alt, Eran Tromer, and Vinod Vaikuntanathan. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In *44th ACM Symposium on Theory of Computing (STOC)*, pages 1219–1234, 2012.

[52] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, Wenting Zheng, and Raluca Ada Popa. DELPHI: A cryptographic inference service for neural networks. In *29th USENIX Security Symposium*, pages 2505–2522, 2020.

[53] Payman Mohassel and Peter Rindal. ABY3: A mixed protocol framework for machine learning. In *ACM CCS*, pages 35–52, 2018.

[54] Eduardo Morais and Ricardo Dahab. Lattice-based cryptography. Technical Report IC-2016-1, Institute of Computing, University of Campinas, April 2016.

[55] Alex Ozdemir, Fraser Brown, and Riad S. Wahby. CirC: Compiler infrastructure for proof systems, software verification, and more. In *IEEE Symposium on Security and Privacy (S&P)*, pages 2248–2266, 2022.

[56] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. CrypTFlow2: Practical 2-party secure inference. In *ACM CCS*, pages 325–342, 2020.

[57] Dragos Rotaru and Tim Wood. MArBled circuits: Mixing arithmetic and boolean circuits with active security. In *Progress in Cryptology — INDOCRYPT*, pages 227–249, 2019.

[58] Microsoft SEAL (release 4.1). `https://github.com/Microsoft/SEAL`, 2023. Microsoft Research, Redmond, WA.

[59] Victor Shoup. NTL: A library for doing number theory. `https://libntl.org/`, 2021.

[60] TFHE-rs: A pure rust implementation of the TFHE scheme. `https://github.com/zama-ai/tfhe-rs`, 2024. Zama.

[61] Sameer Wagh, Divya Gupta, and Nishanth Chandran. SecureNN: 3-party secure computation for neural network training. *Proceedings on Privacy Enhancing Technologies*, 2019(3):26–49, 2019.

[62] Sameer Wagh, Shruti Tople, Fabrice Benhamouda, Eyal Kushilevitz, Prateek Mittal, and Tal Rabin. FALCON: Honest-majority maliciously secure framework for private deep learning. *Proceedings on Privacy Enhancing Technologies*, 2021(1):188–208, 2021.

[63] Huan-Chih Wang and Ja-Ling Wu. CHIP: Efficient homomorphic encryption-based CNN batch inference using channel-interleaved packing with small rotation key set. In *IEEE Computer Security Foundations Symposium (CSF)*, 2026. Cryptology ePrint Archive, Paper 2026/1399.

[64] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[65] Shmuel Winograd. *Arithmetic Complexity of Computations*. SIAM, 1980.

[66] Jiancheng Yang, Rui Shi, Donglai Wei, Zequan Liu, Lin Zhao, Bilian Ke, Hanspeter Pfister, and Bingbing Ni. MedMNIST v2 – a large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data*, 10(1):41, 2023.

[67] Andrew Chi-Chih Yao. How to generate and exchange secrets. In *27th IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 162–167, 1986.

[68] Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole: Reducing data transfer in garbled circuits using half gates. In *EUROCRYPT*, pages 220–250, 2015.